Generative AI

2 Build a LLM From Scratch

1 Generative AI: Working with LLM(s) – LinkedIn

1.1 Course 01; Introduction to Generative AI Fundamentals

1.1.1 Lesson 01

https://www.udacity.com/

Text Generation: making computers write text that makes sense and is relevant to the topic, akin to an automatic storyteller.
Image Generation: computers to make new pictures or change existing ones, like a digital artist using a virtual paintbrush.
Code Generation: Gen AI for programming, where the computer helps writing new code.
Audio Generation: Computers can also create sounds or music, a bit like a robot composer coming up with its own tunes. Voice synthesis and text to speech (TTS): voice generation is part of audio generation.
Current models:
Chat GPT: A language model developed by OpenAI that can generate responses similar to those a human would give in a conversation, by predicting the next word in a sequence, based on context.
DALL·E: An AI program by OpenAI that produces images from textual descriptions, mimicking creativity in visual arts.
GitHub Copilot: A coding assistant tool that suggests code snippets and completes code lines to help developers write more efficiently and with fewer errors. Contextual suggestions, autocompleting code, develop faster, reduce errors and stay in the flow.
Contextual Suggestions: Recommendations provided by AI tools, like Copilot, which are relevant to the current task or context within which a user is working.
Once we start discussing tools for code generation, it's natural to wonder about whether AI will replace human jobs. The important thing to remember is that the future impact of AI is still undetermined, and we all can play a part in ensuring that it is used responsibly.
Applications of Generative AI: The applications of Generative AI span a gamut of exciting fields, broadening creativity and innovation in content creation, product design, scientific inquiry, data enhancement, and personalized experiences. The power of Generative AI lies in its ability to imagine and refine with speed, offering solutions and opening doors to future inventions.
Creative content generation, product development (design optimization, rapid prototyping, material exploration), scientific research, data augmentation, personalization.
Creative content, artwork synthesis (unique art pieces), music composition (imitate the style of composers), literary creation:application of AI in generating written content.
Scientific research: experiment simulation, data analysis and prediction, uncovering trends,
Data augmentation: it can create synthetic data: overcoming problems of privacy, overcome the problem of data scarcity: image enhancements. Augmentation to create diverse text data, .
Synthetic data : creating datasets from scratch.
Personalization: personalize experiences for customers, based on profiles.: content recommendation, bespoke product creation (e.g. clothes), experience customization.
AI and ML timeline:
1950: early work (perceptron) Rosenblatt 1958, Turing and the Turing test
1960: rule based systems for NLP and machine translation: Elisa, backpropagation
1970: statistical ML methods. (AI winter) but there was no computational capacity, HMM, Naive Bayes
1980's: backpropagation successful implementation, multilayer perceptrons, AI winter ended
1990 data driven approaches, random forests, Kasparov beated by deep blue
2000 Torch released, the first open library.
2010 deep learning explodes: GAN(s), AlphaGO , transformer.
2020 Generative AI booms
Terms
Perceptron: An early type of neural network component that can decide whether or not an input, represented by numerical values, belongs to a specific class.
Neural Networks: Computer systems modeled after the human brain that can learn from data by adjusting the connections between artificial neurons.
Back Propagation: A method used in artificial neural networks to improve the model by adjusting the weights by calculating the gradient of the loss function.
Statistical Machine Learning: A method of data analysis that automates analytical model building, using algorithms that learn from data without being explicitly programmed.
Deep Learning: A subset of machine learning composed of algorithms that permit software to train itself to perform tasks by exposing multilayered neural networks to vast amounts of data.
Generative Adversarial Networks (GANs): A class of machine learning models where two networks, a generator and a discriminator, are trained simultaneously in a zero-sum game framework.
Transformer: A type of deep learning model that handles sequential data and is particularly noted for its high performance in natural language processing tasks.
Training Generative AI models: a variety of techniques. During training, the model builds an internal representation of a vast dataset, representing the world of possibilities. After the training phase, the model is able to generate diverse outputs in various styles. The data is what the humans create and experience.
Training: large language models: variational autoencoders. How are GenAI models trained?: Different techniques. Training creates an internal representation of the dataset. Training data is what humans create and experience. Post training, in production, the model is able to create e.g a painting in the style of Van Gogh, different styles. Etc.
Many ways:

- training large models: training LLM: pick a random incomplete phrase, the model must complete the phrase. It becomes better at completing words. This allows to answer questions, write realistic and coherent passages, generate translations.
- training variational autoencoders: Encoder and decoder: a picture of a dog is squeezed into a vector of the latent space. Variational autoencoder. Use the decoder to generate images. Stable diffusion uses a variational autoencoder as one of

their main components.

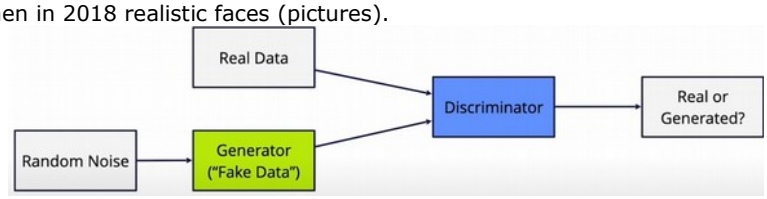
Generative algorithms: autoregressive text generation: generate sequences of words. For example, "the cow jumped over the.." the LLM provides continuation with probabilities, then it predicts the phrase based on those probabilities.. Keep repeating the process and it is possible to generate text that sounds like generated by a human being.

Latent space decoding: decoder can generate. We can generate randomly or manually generate a vector in the latent space to generate a product of the decoder. The product (image) is hidden somewhere in the latent space

Diffusion models: image diffusion model, starts with noise and removes noise. Diffusion models are trained by removing noise (denoise) in images where it was added noise, to create the dataset.

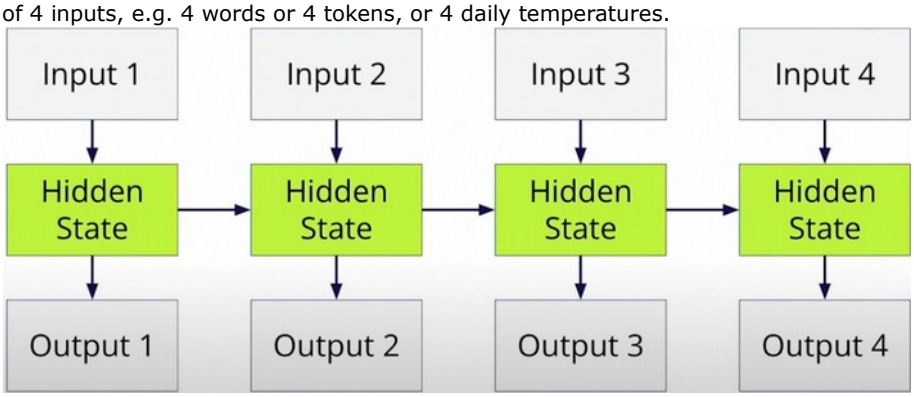
Loads a tokenizer and model (model for causal LM). Tokenizes the text and retrives token-id(s). Each token is given an ID. Subword tokenization: tokenization creates tokens from the text: each token corresponds to a word, or a part of a word.

More generative AI architectures: e.g. GAN(s). Trained on large datasets of existing data. GAN(s) Generator and discriminator proposed in 2014, then in 2018 realistic faces (pictures).



GAN architecture

RNN(s): for sequential data text and audio, learn sequential data and can predict the next element in a sequence. Here it processes a sequence of 4 inputs, e.g. 4 words or 4 tokens, or 4 daily temperatures.



Transformer based models are effective for generating text and translating languages:

- effective for text generation and translations
- can capture long range dependencies in sequential data
- they can generate new sequential data by predicting the next element in a sequence

They work with the entire sequence in parallel, rather than reading the sequence item by item. RNN(s) on the other hand, read the input one piece at a time. Sequential data: data that is connected in a specific order, like words in a sentence or steps in a dance routine.

Challenges for Generative AI

Deepfakes: Highly realistic fake videos or images created by AI, in which it seems that people are saying or doing things but they never actually did. False narratives can be spread at an unprecedented rate.

Automation: The use of technology to perform tasks without human intervention, which can increase efficiency but also may replace jobs done by people.

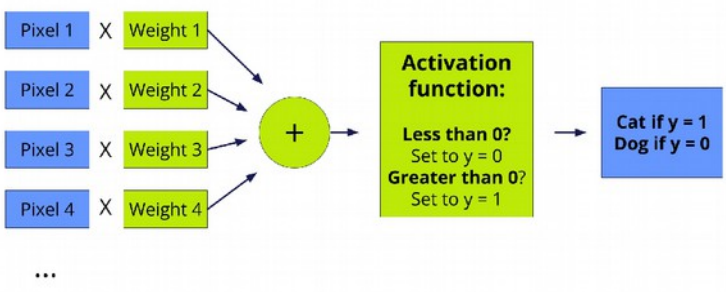
Copyright Issues: Legal problems that arise when someone uses work without permission, potentially impacting the original creator's rights.

Carbon Footprint: The total amount of greenhouse gases produced directly or indirectly by activities or entities, like running large-scale AI models.

Research pre-training datasets. Common Crawl: Size: 99.25TB

1.1.2 Lesson 02: Deep Learning Fundamentals

1.1.2.1 What is a perceptron?



The activation function in the first perceptron was a threshold. Sigmoid can also be used. It is a model of a single neuron used for binary classification. How do we choose the weight? The process is called learning. Before weights are trained , answers will likely be wrong.. After each forward pass, the training nudges the weights to get the correct answer more often.

It needs labelled data. The activation function: it can be a threshold = a step function. Today, the ReLU (rectified linear function) is a popular choice in the field.

Perceptron: A basic computational model in machine learning that makes decisions by weighting input data. It's like a mini-decision maker that labels data as one thing or another. Binary Classifier: A type of system that categorizes data into one of two groups. Picture a light switch that can be flipped to either on or off.

The input and output are vectors of numbers: a sequence of numbers arranged in order, which together represent one piece of data.

Activation Function: A mathematical equation that decides whether the perceptron's calculated sum from the inputs is enough to

trigger a positive or negative output.
The multi layer perceptron (MLP): Multiple layers. Each node is a perceptron. The input layer receives the raw data, the output layer produces the prediction or decision. Between the input and output, a set of hidden layers perform complex transformations on the data. In the MLP each neuron is connected to each neuron in the preceding layer. The each connection has a weight. Classification: the number of output neurons is equal to the number of classes in the problem.
Multi-Layer Perceptron (MLP): A type of artificial neural network that has multiple layers of nodes, each layer learning to recognize increasingly complex features of the input data.

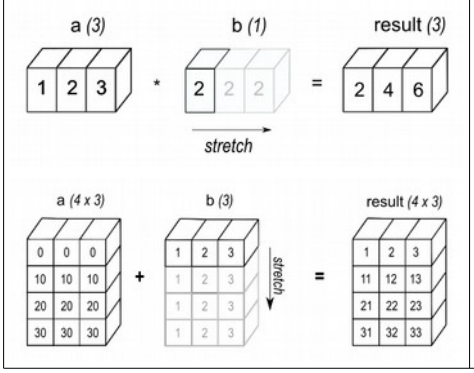
- Input Layer: The first layer in an MLP where the raw data is initially received.
- Output Layer: The last layer in an MLP that produces the final result or prediction of the network.
- Hidden Layers: Layers between the input and output that perform complex data transformations.

Training Neural networks:

- Labeled Dataset: This is a collection of data where each piece of information comes with a correct answer or label. It's like a quiz with the questions and answers already provided.
- Gradient Descent: This method helps find the best settings for a neural network by slowly tweaking them to reduce errors, similar to finding the lowest point in a valley.
- Cost Function: Imagine it as a score that tells you how wrong your network's predictions are. The goal is to make this score as low as possible.
- Learning Rate: This hyperparameter specifies how big the steps are when adjusting the neural network's settings during training. Too big, and you might skip over the best setting; too small, and it'll take a very long time to get there.
- Backpropagation: Short for backward propagation of errors. This is like a feedback system that tells each part of the neural network how much it contributed to any mistakes, so it can learn and do better next time.

Backpropagation.. The first step is a forward pass: input data is fed into the network and the corresponding prediction is produced. For each output is compared to the expected result. After training, the model is tested with a test (or hold out dataset). To check if the model performs well with data it has never seen.
Pre trained models: By using pre-trained models and the magic of transfer learning, the hard work of training an AI model from zero can be bypassed,making it easier and quicker to get the job done. By leveraging the knowledge a model distills from a large dataset,we can reduce the amount of training needed to get a performant model.Transfer learning: The process where knowledge from a pre-trained model is applied to a new, but related task.
Examples of datasets: Common Crawl for NLP, ImageNet for image classigfication, Librespeech: 1000 hours of audiobooks to train TTS systems. At the end of the training process, using a pre-trained model, there is a new model. Often the last layer will change and it will be trained from scratch during training with a pre-trained model, because we need a number of output nodes different from the original network. MobileNet v3 is a model suitable for mobile devices, it can be used as a pre-trained model.
In trasfer learning, we use supervised learning, so images are labelled. Different example: spam filter. Use DistilBERT as pre-trained model.DistilBERT is a smaller, but effective version of BERT.

NumPy Numpy broadcasting: how NumPy treats arrays of different shapes in arithmetic operations. The smaller array is broadcast across the larger array, so that shapes match. Broadcasting is done without creating copies of data, in a way that operations can be vectorized. NumPy opertions are usually done on pairs of arrays.
Links (Numpy broadcasting) NumPy absolute basics for beginners NumPy quickstart

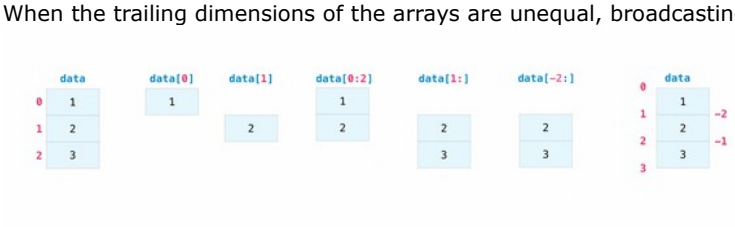


When operating on two arrays, NumPy compares their shapes element-wise. It starts with the trailing (i.e. rightmost) dimension and it works its way left. Two dimensions are compatible when:

- they are equal, or
- one of them is 1.

In case input arrays do not have the same number of dimensions, the result will be an array with the greatest number of dimensions in the inputs. The missing dimensions are assumed to have size one.

For example a color image (256,256,3) is multiplied by an array of three elements, equivalent to (1,1,3). The broadcasting result is (256,3,3). Other example A:(8,1,6,1) B:(1,7,1,5). Result (8,7,6,5)
Suppose A:(5,6), if C is a vector of 6 element, it is considered (1,6), so it is broadcasted to (5,6).
When the trailing dimensions of the arrays are unequal, broadcasting fails.



1.1.2.2 PyTorch

Tensors are multidimensional arrays . The first axis is the index of the sample; in the case of the image, the next two axis are the height and the width. PyTorch Tensors tutorial PyTorch tensors documentation. Every epoch uses the entire dataset one time.

1.1.2.3 Hugging Face

Hugging face components: tokenizers (text to tokens), pre-trained models, datasets (they are textbooks for computer models) collections of information that models study to learn and improve, trainers: the coaches for computer models, they implement the PyTorch training loop for you. (https://huggingface.co/) The Hugging Face datasets is built on top of Apache Arrow.

Using pre-trained models – transfer learning. By using pre-trained models and the magic of transfer learning, the hard work of training an AI model from zero can be bypassed, making it easier and quicker to get the job done. By leveraging the knowledge a model distills from a large dataset, we can reduce the amount of training needed to get a performant model.
Transfer learning: The process where knowledge from a pre-trained model is applied to a new, but related task.
Examples of datasets: Common Crawl for NLP, ImageNet for image classification, Librispeech: 1000 hours of audiobooks to train TTS systems.
At the end of the training process, using a pre-trained model, there is a new model. Often the last layer will change and it will be trained from scratch during training with a pre-trained model, because we need a number of output nodes different from the original network. MobileNet v3 is a model suitable for mobile devices, it can be used as a pre-trained model. In trasfer learning, we use supervised learning, so images are labelled. Different example: a spam filter. Use DistilBERT as pre-trained model.DistilBERT is a smaller, but effective version of BERT.

1.1.3 Lesson 03: Foundation Models

A foundation model is trained on vast amount of data at scale. It can perform a number of tasks, with minimum additional training. It is foundational because it can serve as the basis for additional applications. For example, ChatGPT, a model from OpenAI, it is trained on a vast amount of data covering human knowledge. Foundation models are versatile, they are trained on text, images and other types of information. FM are a paradigm shift, in the past the focus was to build a large dataset and training a model from scratch, now we start from a FM and build on top of it.
Foundation Model: A large AI model trained on a wide variety of data, which can do many tasks without much extra training.
Adapted: Modified or adjusted to suit new conditions or a new purpose, i.e. in the context of foundation models.
Generalize: The ability of a model to apply what it has learned from its training data to new, unseen data.

Foundation Models	Traditional models
Trained on vast, general datasets, e.g. the entire Wikipedia Designed to generalize across many tasks Require immense computational resources Examples: GPT, Bard (Gemini), LLaMa, BERT, DALL-E They are a new paradigm. Large scale in terms of parameters, training data and computational resources.	Trained on smaller, task specific datasets, often trained from scratch on a meticously curated dataset Task specific and domain specific: document classification. They are useless outside of that application. Examples: Linear Regression, decision trees, CNN(s), etc.

Architecture and scale Text based models are currently based on the transformer model, they are trained using sequential data at scale. The novel idea of the transformer is the self attention mechanism, the model focuses on the relevant parts, this is useful for language modeling tasks. Relevance of words can vary, depending on the context. Llama is trained on 4.7TB of data. The latest foundation models have tens of billion of parameters, the batch size is 4M.
Why benchmarks matter: they are an objective metrics for evaluation of algorithms. ImageNet: Benefits of benchmarks:

- comparability: direct comparison of diffent algorithms and models
- reproducibility of results, for scientific progress, verification of results
- focus: they provide a well-defined problem
- democratization: open access to high quality data, they level the plane field, multiple groups can access the same data
- acceleration of progress: as progress advances, more advanced benchmarks are proposed

ImageNet revolutionized the field of computer vision. In 2012 AlexNet outperformed the current methods by a significant margin in a challenge involving ImageNet.
Robustness: The strength of an AI model to maintain its performance despite challenges or changes in data.
Open Access: Making data sets freely available to the public, so that anyone can use them for research and develop AI technologies.
The GLUE Benchmark: GLUE Tasks: General Language Understanding Evaluation benchmark: a collection of language understanding tasks to evaluate the perfomrnace of models in understanding Human Language in a range of linguistic phenomena

- CoLA Grammatical Acceptability: Measures the ability to determine if an English sentence is linguistically acceptable.
- SST-2: sentiment analysis, e.g. movie review. Consists of sentences from movie reviews and human annotations about their sentiment.
- MRPC: paraphrase identification. Focuses on identifying whether two sentences are paraphrases of each other.
- STS-B- Semantic textual similarity. Involves determining how similar two sentences are in terms of semantic content.
- QQP: question pairs equivalence on Quora. Aims to identify whether two questions asked on Quora are semantically equivalent.
- MNLI Natural Language inference: understanding the relation between two sentences. Consists of sentence pairs labeled for textual entailment across multiple genres of text.
- QNLI Question natual answering inference. Involves determining whether the content of a paragraph contains the answer to a question.
- RTE: textual entailment recognition: whether an hypothesis is true, or false, given a premise.. Requires understanding whether one sentence entails another.
- WNLI: pronon disambiguation. Tests a system's reading comprehension by having it determine the correct referent of a pronoun in a sentence, where understanding depends on contextual information provided by specific words or phrases.

Textual Entailment: The relationship between text fragments where one fragment follows logically from the other.
Semantic Equivalence: When different phrases or sentences convey the same meaning or idea.
The SuperGLUE benchmarks: SuperGlue is designed as a successor to the original GLUE benchmark. It's a more advanced benchmark aimed at presenting even more challenging language understanding tasks for AI models. Created to push the boundaries of what AI can understand and process in natural language, SuperGlue emerged as models began to achieve human parity on the GLUE benchmark. It also features a public leaderboard, facilitating the direct comparison of models and enabling the tracking of progress over time. (https://super.gluebenchmark.com/)
Data Used for Training LLM(s): The capacity of a LLM to generate human-like text, is related to the quality of the training data:

- Websites: articles, blogs and forums
- Scientific Papers: academic and research papers
- Encyclopedias: comprehensive and factual entries
- Books and literature: classic and modern books, various genres

- Conversational data: transcripts from conversations, scripts from plays, text from chatbord
- Social media posts: abbreviations
- Legal documents:
- Multilingual text: text i various languaes are included: for the most part the current models are trained on English-only data.

The data need careful preprocessing to ensure that the text are useful for training. Cleaning, anonymizing personal information.
Preprocessing: This is the process of preparing and cleaning data before it is used to train a machine learning model. It might involve removing errors, irrelevant information, or formatting the data in a way that the model can easily learn from it.
Fine-tuning: After a model has been pre-trained on a large dataset, fine-tuning is an additional training step where the model is further refined with specific data to improve its performance on a particular type of task.
Data scale and volume The scale of data for Large Language Models (LLMs) is tremendously vast, involving datasets that could equate to millions of books. The sheer size is pivotal for the model's understanding and mastery of language through exposure to diverse words and structures. 1GB of text is roughly 1000 books of content. An LLM is trained on millions of books. Llama was trained on more than 4.5TB of data. About 4.5M books. The scale of training data is related to the ability of the model to learn and understand language, so it becomes more accurate. Because the model is exposed to more words, phrases and sentence structures. Such models may also be continuously trained and fine tuned with new data, it means that the training data grows over time, as more text is generated and collected. -
Among the datasets used is Common Crawl: an open repository of web crawl data. Essentially, it is a large collection of content from the internet that is gathered by automatically scraping the web.
Biases in Training data. Biases in training data deeply influence the outcomes of AI models, reflecting societal issues that require attention. Ways to approach this challenge include promoting diversity in development teams, seeking diverse data sources, and ensuring continued vigilance through bias detection and model monitoring.
Need for vigilance: Institutions can have interests that are not aligned with those of the public. Training data can have biases. If biased data is used to train LLM, a reflection of societal inequallisites or historical prejudices. It perpetuates the reinforces the biases. Sources of biases in data:

- selection bias the selection process is biased => only a subset of the whole data is used to train the model. The collected data is not representative of the populaton of phenomenon.
- Historical bias: historical prejudices are reflected in the data: Gender, race or econominc bias that reflect the culture where the data is collected.
- Confirmation bias: data is selected to confirm pre-existing beliefs.

Effect of biased data:

- discriminatory outcomes: model produces discriminatory outcomes.
- Echo chambers: the model creates a feedback loop that reinforces existing biases
- Misrepresentation and exclusion: model misrepresents or excludes cartain groups. If a model is trained with a set of languages; it will have trouble with other languages or dialects, that are not present in its training set.
- LLM play a role in job selection , loan approval and criminal profiling, there are consequences in biases on people. .

How to mitigate biases? Multi faceted challenge

- organizational diversity: ensuring that the teams involved are diverse. The leadership in organization reflects the biases in society
- Diverse data collection: actively seeking out diverse data sources.
- Bias detection and correction: employing algorithms and humani oversight
- Transparency and accountability: being transparent about the sources and nature of training data:
- Continuous monitoring, regularly and updating models

Biased data are a mirror of the boader societal issues that we face.
Disinformation and Misinformation disinfomational is intentional, misinformation is accidental/inadvertent. The result is false or inaccurate information. In ChatGPT site is written: ChatGPT can make mistakes. Consider checking important information.
LLM(s) can fenerate realistic, yet false content, as people can write finctional stories. How to mitigate the risk:

- robust ethical guidelines
- technical safeguards: Content Provenance Tools: Tools designed to track the origin and history of digital content. They help verify the authenticity of the content by providing information about its creation, modification, and distribution history.
- education and training: educate about the possibilities and limitations of LLM(s)

Environmental and human impacts: Environmental impacts:

- high energy consumption for training and inference; the carbon footprint is the same as saveral cars
- use of non renewable resources used to product GPU(s) and hardware
- electronic waste and resource depletion

Human impacts

- economic distruption: loss of jobs due to automation of tasks, it creates also new jobs
- bias and fairness: amplify biases in the datasets
- privacy: datasets can include sensitive personal information, ensuring privacy and data protection is important
- misinformation and disinformation:
- security: security challenges, new cyberattacks
- existential threat to humanity: autonomous weapons

1.1.4 Lesson 03: Adapting Foundation Models

Adaptation is the process of costumizing a model to better suit an application or address up-to-date information. Fine tune or adaptation: it means taking a pre-trained model and training it again on new data, for tasks like: question anwsering, sentiment analysis, information extraction, image captioning, object recognition instruction following. It can align the model with requirements on company guidelines or privacy concerns. Adaptation in AI is a crucial step to enhance the capabilities of foundation models, allowing them to cater to specific tasks and domains. This process is about tailoring pre-trained AI systems with new data, ensuring they perform optimally in specialized applications and respect privacy constraints. Reaping the benefits of adaptation leads to AI models that are not only versatile but also more aligned with the unique needs of organizations and industries.

- Chatbot adaptation for banking: adaptation for the specific domain, accounts, loans and other topics
- structuring medical recordss: the model can extract key data and ideas from medical records (i.e. from unstructured records to structure data?)
- general instruction-based adaptation: for example translation from English to French
- Fine-tuning using private data: internal, domain spcecific data and content

Fine-tuning: This is a technique in machine learning where an already trained model is further trained (or tuned) on a new, typically smaller, dataset for better performance on a specific task.

Why do we need to adapt foundation models? Adapt for specific use-case, why? Foundation models have demonstrated versatility but there is still the need to fine-tune them. Moreover, we may need information updated to 2024, but GPT-3 is trained with data until 2022. Adapting foundation models is essential due to their limitations in specific areas despite their extensive training on large datasets. Although they excel at many tasks, these models can sometimes misunderstand questions or lack up-to-date information, which highlights the need for fine-tuning. By addressing these weaknesses through additional training or other techniques, the performance of foundation models can be significantly improved.

Retrieval Augmented Generation (RAG). Who won the NBA championship this year? The idea is quickly searching a database for snippets that may contain the answer, and to send this pieces of text, along with our question, to the LLM. How does it happen?

- Given the answer "Who won the NBA this year", we create an embedding of the qustion. It produces a semantic embedding (a vector of numbers)
- Using the embedding, we perform a similarity search on a database; there are vector databases that can perform this search quickly.
- The search produces a set of relevant snippets of text.
- We create a prompt that contains our question and also the retrieved snippets of text.
- We collect the answer

There are a number of variations in RAG, but the concept is the same.

Semantic-embedding: A representation of text as a vector in a high-dimensional space where distances between points correspond to semantic similarity. Phrases with similar meanings are closer together.

Cosine similarity: A metric used to measure how similar two vectors are, typically used in the context of semantic embeddings to assess similarity of meanings. $\frac{\vec{a}\cdot\vec{b}}{\|a\|\|b\|}=\cos(\theta)$ Cosine similarity: the cosine of the angle between two vectors; it does not depend on the magnitude of the vectors.

Vector databases: Specialized databases designed to store and handle vector data, often employed for facilitating fast and efficient similarity searches.

Prompt design techniques:

- Prompt tuning: guide the model prediction in a domain specific task: *place the question at the very end*
- Few-shot prompting: provide a handful of examples to the model (*e.g. <= 5*)
- Zero-shot -prompting
- Chain of thoughts: provide a series of logical reasoning to guide the model. For example in math problems, you may provide examples of math problems broken down into steps, or even just say "*think in steps*".
- In-context learning: allowing the model to learn from the context provided in the prompt. Providing instructions or examples in the prompt.

These techniques can be applied together, you can use few shot prompting with chain of thoughts.

Domain-Specific Task: A task that is specialized or relevant to a particular area of knowledge or industry, often requiring tailored AI responses.

Prompt-tuning: a technique used in gen-AI, where task specific are optimized to guide a model prediction. The templates are called prompts, to steer the model predictions to our task. For example, the Task is Restaurant Review Sentiment Analysis. The input is the text of the review. Prompt: In AI, a prompt is an input given to the model to generate a specific response or output. Prompt Tuning: This is a method to improve AI models by optimizing prompts so that the model produces better results for specific tasks.

Hard Prompt: A manually created template used to guide an AI model's predictions. It requires human ingenuity to craft effective prompts. Soft Prompt: A series of tokens or embeddings optimized through deep learning to help guide model predictions, without necessarily making sense to humans, but it may be superior to a hard-prompt. The soft prompt steers the LLM in the desired direction.

One and few shot prompting. One-shot: providing the model with one example, before asking the question, few-shot: providing few examples (*5 or less*). Examples are provided before the questions, to demonstrate how the model should reply.

```
Q: What is (25+1)*2?
A: The calculation is simple. You add 25 to another 1, which equals to 26. Then , you multiply the 26 by 2, which equals 52.
So, (25+1)*2 is 52.

Q: What is (3+2)*2?
```

Few shot prompting. Example:

```
1. Q: What is the capital of France?
   A: Paris
2. Q: What is the capital of Peru?
   A: Lima
3. Q: What is the capital of the Philippines?
   A: Manila

Q: What is the capital of Algeria?
```

The Chat-GPT 3.5 replied: **`Algiers`** (which is correct).

One and few-shot prompting represent cutting-edge techniques that enable AI to adapt and perform tasks with minimal instructions. Instead of relying on extensive databases for learning, these methods guide generative AI through just one or a few examples, streamlining the learning process and demonstrating its ability to generalize solutions to new problems. This innovative approach marks a significant advancement in machine learning, empowering AI to quickly adjust to specialized tasks and showcasing the incredible potential for efficiency in teaching AI new concepts.

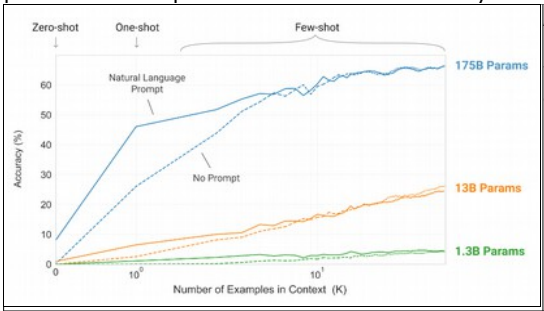
One-shot prompting: Giving an AI model a single example to learn from before it attempts a similar task.
Few-shot prompting: providing an AI model with a small set of examples, such as five or fewer, from which it can learn to generalize and perform tasks.
Zero-shot prompting: the foundation model handles tasks without any task-specific training data provided in the prompt; the models relies completely on its previous training Zero-shot prompting is a remarkable technique where a generative AI model can take on new tasks without the need for specific training examples. This process leverages the AI's extensive pre-existing knowledge gained from learning patterns across vast datasets. It empowers the AI to infer and generalize effectively to provide answers and solutions in contexts that were not expressly covered during its initial training. Zero-shot prompting: This refers to the capability of an AI model to correctly respond to a prompt or question it hasn't explicitly been trained to answer, relying solely on its prior knowledge and training.

```
Here is a legal document:
This is a contract between two parties for the sale of goods. The contrace is legally binding and enforceable by law.
The category of this legal document is _________
```

In the course, the teacher terminates the prompt with "blank", with a set of underscores: _____
In-context learning: In context: the model retrieves information from the prompt. In-context learning has two components:

- task examples: concrete examples of the task that the model can learn from, like a mini-labelled dataset
- task descriptions: they are more abstract, they describe the task in words

Both: task examples and task descriptions, are considered in-context learning.
When performing few-shot, one-shot, or zero-shot learning, we can pass information to the model within the prompt in the form of examples, descriptions, or other data. When we rely on a model using information from within the prompt itself instead of relying on what is stored within its own parameters, we are using in-context learning.
As these AI models grow in size, their ability to absorb and use in-context information significantly improves, showcasing their potential to adapt to various tasks effectively.



As these AI models grow in size, their ability to absorb and use in-context information significantly improves, showcasing their potential to adapt to various tasks effectively. The progress in this field is inspiring, as these advances hint at an exciting future where such models could be even more intuitive and useful.
Summary: Larger models make efficient use of in-context information.

Chain-of-thought (COT) prompting a method of guiding a language model through a step-by-step reasoning process to help it solve complex tasks by explicitly detailing the logic needed to reach a conclusion.
COT Prompting is a vital technique for enhancing the reasoning capabilities of large language models by breaking down complex problems into intermediate steps that lead to a solution. By providing models with a line of reasoning, they can more effectively tackle problems that require more advanced problem-solving processes, enabling them to deduce information, such as the number of cookies in a box, after considering all variables. COT prompting enhances the reasoning capabilities of a large language model. Provide a sequence of intermediate natural language reasoning steps. In the execise, it worked better by providing: the task description, the chain-of-thoughts, and the examples.

```
Given a list of words, create a new word:
. starts with the first letter from the first word,
. only if there is a second word append to the new word the first two letters from the second word, otherwise do not append anything
. only if there is a third word append to the new word the first three letters from the third word, otherwise do not append anything
. only if there is a fourth word append to the new word the first four letters from the fourth word, otherwise do not append anything
. only if there is a fifth word append to the new word the first five letters from the fifth word, otherwise do not append anything

For example the list of words is: forehead vocalist ancients
. Take the first letter from the first word "f",
. Append the first two letters from the second word "vo"
. Append the first three letters from the third word "anc"
. There are no more words, so nothing more to append.
The result is: "fvoanc"

Other examples:
List of words: kitchen
Answer: k

List of words: rumbling immortal
Answer: rim

List of words: meanings climates security
Answer: mclsec

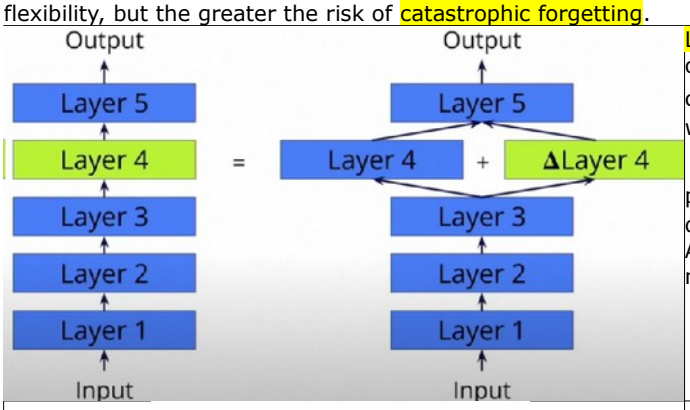
The list of words is: forehead vocalist ancients mongolia
```

Using Probing to train a classifier: Using probing to train a classifier is a powerful approach to tailor generative AI foundation models, like BERT, for specific applications. By adding a modestly-sized neural network, known as a classification head, to a foundation model, one can specialize in particular tasks such as sentiment analysis. This technique involves freezing the original model's parameters and only adjusting the classification head through training with labeled data. Ultimately, this process simplifies adapting sophisticated AI systems to our needs, providing a practical tool for developing efficient and targeted machine learning solutions.
Probing: This is a method of examining what information is contained in different parts of a machine learning model.

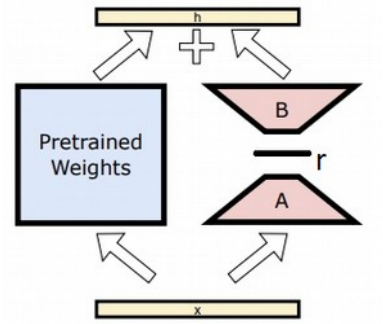
Linear Probing: A simple form of probing that involves attaching a linear classifier to a pre-trained model to adapt it to a new task without modifying the original model.
Classification Head: It is the part of a neural network that is tailored to classify input data into defined categories.
For example, we take BERT, the output is an encoding of the original message as a tensor. We attach a small neural network to the otput of BERT, for example we attach a classification head to 768 of the nodes from the final layer of BERT. We can train this classification head using our own dataset. For example, we use the IMDb dataset to train the classigfication head. Only the classification head changes during training, the layers of BERT are frozen. The model has now what can be referred to as "linear probe".
Fine-Tuning: (traditional fine tuning) is an important phase in enhancing the abilities of generative AI models, making them adept at specific tasks. By introducing additional data to these powerful models, they can be tailored to meet particular requirements, which is invaluable in making AI more effective and efficient. Although this process comes with its challenges, such as the need for significant computational resources and data, the outcome is a more specialized and capable AI system that can bring value to a wide range of applications. Fine-tuning: training the model on additional data, or changing some of the parameters to adapt it to a specific use-case. In traditional fine tuning, a pretrained model is updated using additional data, changing the actual parameters. Fine tuning is a subset (a kind of) transfer learning: traditional fine tuning updates all the parameters of the model to improve its performance on a specific task (for example, it can have 175B parameters(!)).
Challenges associated with traiditional fine-tuning.

- Gathering labeled data: e.g. extensive amount of training data, it can be expensive and time consuming.
- Computational resources, similar to the amount that was required when the model was trained in the first place.
- Storage, of the model and the data. Each used has a slightly different copy of the model, this erodes the notion of reusable foundation models
- Out-of-distribution data: the fine tuning data is different from the original distribution used when the model was trained in the first place.. This can cause catastrophic forgetting: the model forgets what it had learnt before

Fine-tuning: This is the process of adjusting a pre-trained model so it performs better on a new, similar task. It's like teaching an experienced doctor a new medical procedure; they're already a doctor, but they're improving their skills in a particular area.
Catastrophic Forgetting: This happens when a model learns something new but forgets what it learned before. Imagine if you crammed for a history test and did great, but then forgot most of what you learned when you started studying for a math test.
Parameter efficient fine tuning (PEFT):(See the paper: Hu, Edward J. et al."Lora: Low-rank adaptation of large language models." (2021)) fine tune a LLM minimizing the need for large computational resurces. One approach is to free the majority of the layers (weights) and only update a small portion of them, e.g. the last layer (aka the Head), that is shown at the top by convention on graphs). What more flexibility? Freeze all except the two last layers. The more layers are not frozen , the more flexibility, but the greater the risk of catastrophic forgetting.

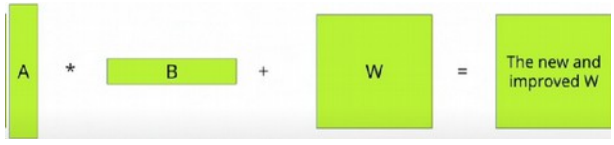


LoRA. Another technique is called LoRA: suppose we want to focus on the fourth layer: after fine tuning, the Layer 4 will have changed by an amount called $\Delta\ Layer\ 4$, which is trainable, while the rest is frozen. Thus we decompose $Layer\ 4 = Layer\ 4 + \Delta\ Layer\ 4$. $\Delta\ Layer\ 4$ has fewer parameters. How can it be? A square matrix W, of size DxD, is decomposed into two skinny matrices, A and B whose product is D. A and B are good enought to keep track of how much the original matrix would change under regular fine tuning.

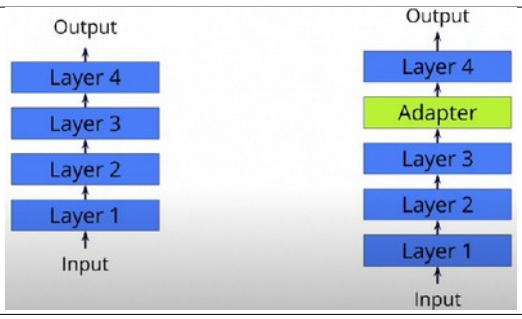


$$\Delta\ Layer\ 4 = A * B$$

We are increasing the complexity of the model, because at inference time we must calculate:
$h = Wx + (A * B)\,x$,
but the model trains quickly.
The new W is : $W' = w + A * B$



We use W' instead of W.



Another method is using an Adapter. It adds a component to the model and only adapts the parameters of that component. Can be inserted in the model at different layers. LoRA is an example of an adapter.

Parameter-efficient fine-tuning (PEFT) is a technique crucial for adapting large language models more efficiently, with the bonus of

not requiring heavy computational power. This approach includes various strategies to update only a small set of parameters, thereby maintaining a balance between model adaptability and resource consumption. The techniques ensure that models can be swiftly deployed in different industrial contexts, considering both time constraints and the necessity for scaling operations efficiently.
Parameter-efficient fine-tuning: A method of updating a predefined subset of a model's parameters to tailor it to specific tasks, without the need to modify the entire model, thus saving computational resources.
Frozen Parameters: In the context of machine learning, this refers to model parameters that are not changed or updated during the process of training or fine-tuning.
Low-Rank Adaptation (LoRA): A technique where a large matrix is approximated using two smaller matrices, greatly reducing the number of parameters that need to be trained during fine-tuning.
Adapters: Additional model components inserted at various layers; only the parameters of these adapters are trained, not of the entire model.

1.1.4.1 Parameter Efficient Fine-tuning (PEFT)

HF PEFT library

- Create aPEFT configuration
- Convert the model into a PERF model
- In inference, the PEFT model must be loaded instead of the plain model

Training with PEFT

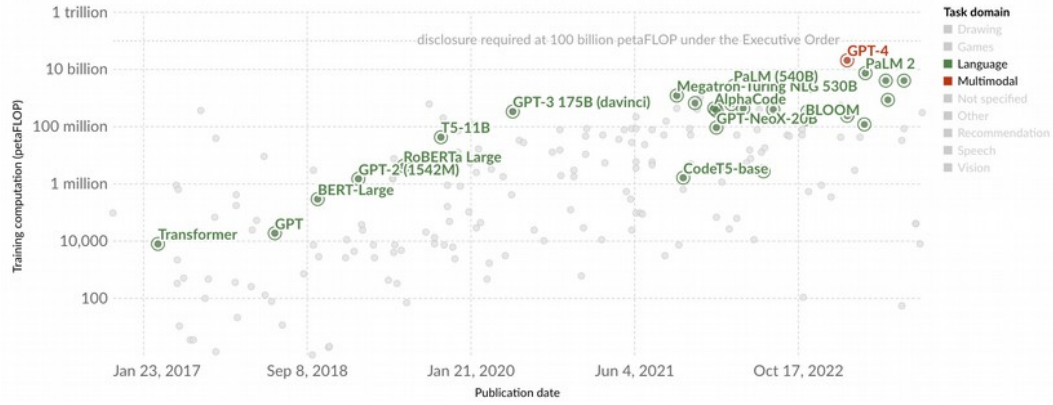
- Create a PEFT Configuration example Lora Config

https://huggingface.co/docs/peft/main/en/task_guides/token-classification-lora
Install the required libraries: !pip install -q peft transformers datasets evaluate seqeval
https://medium.com/@karkar.nizar/fine-tuning-bert-for-text-classification-with-lora-f12af7fa95e4
LoRA Config: consider the parameters:

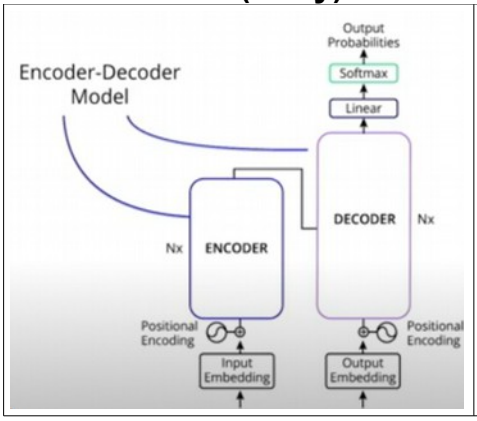
- bias= None

1.2 Course 02:Introduction to LLM(s)

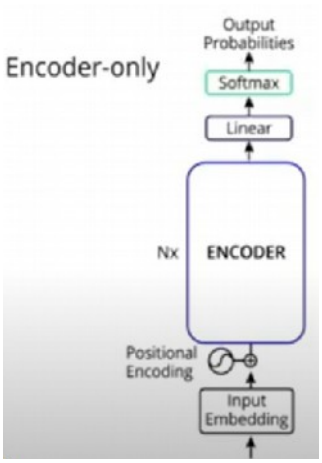
In June 2017 Google released the paper "**Attention is all you need**". It needed 10K-petaflops to train the model. GPT-3 was published on 2020, it used about 100 million petaFlops. Models continue to grow exponentialy larger with the passage of time. The Y axis is logarithmic. At the same time more an more smaller models have been published, capable of beating much larger models on specific tasks. LLM(s) graph: computation used to train notable AI systems (source: Our World in Data).



1.2.1 Decoder(-only) Models



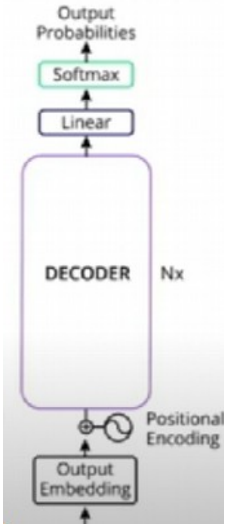
The transformer is composed of multiple layers of the encoder/decoder blocks. Encoder/decoder models are those that adopt the original architecture of the transformer. An example is T5= Text-to-Text Transfer Transformer.



The first transformer model to get broad adoption used only the encoder part of the transformer.
Using only the encoder, chopping the decoder, but taking the linear and softmax layer, we get the basic architecture for the BERT model. Bidirectional Encoder Representations from Transformers.
During encoder-only model pre-training, 15% of the dataset is masked and used for prediction. Example:

`Encoder only` **`[mask]`** `predict only the missing` **`[mask]`**`.`

The model must predict the missing word, where the placeholder [mask] is.
Given 100% of the dataset, 15% of the words are masked and must be predicted.
The criticism is that, using only 15% for prediction, is an inefficient use of the dataset.
However, this pre-trained models can be subsequently fine-tuned for a broad range of tasks. E.g. sentiment classification, etc., so maybe there is an unefficient training, but an efficient reuse of the pre-trained models.
Encoder-only models are most commonly used as base models for subsequent fine tuning with a distinct objective, e.g. text classification.



There are also Decoder-only models, obtained by chopping-off the transformer-encoder block, to achieve an architecture similar to the one used to train GPT and other decoder-only models.
Decoder-only models use an autoregressive language modeling objective, during pre-training that, unlike BERT, rules-out bidirectional attention.
Instead, the model can only attend to the tokens that have already been predicted, with all the later tokens in the sequence masked for later prediction. Example:

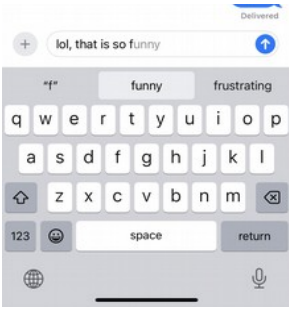
`Decoder only models predict [mask] [mask] [mask]`

The model can only attend to the preceding words, and not the subsequent ones.
Unlike encoder-only models, decoder-only models can run prediction on 100% of the tokens of the pre-training dataset, generating each next token while attending to all the prior tokens. These decoder-only models got the distinction of being called generative models. Decoder-only models are most commonly used for text generation. In distinction to encoder-only models, the Transformer's pre-training objective of next token prediction is very similar to the decoder-only model's inference-time task of text generation.
Attend = to listen or to give attention to.
References:

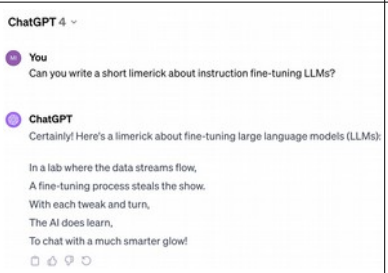
- Link to The illustrated transformer
- The annotated transformer

1.2.2 Completion vs Instruction-following models

For the rest of this lesson we focus on decoder-only models, a.k.a generative models: "*what kind of tasks are they good for?*"



They are good for: document completion task. Example: the completion of emails, or text messages, or the completion of code. Document completion is a natural use-case for generative language models, due to its similarity with the "next token" completion training objective. This use case is all around us, in mobile phones, web forms, etc.



There are also models that are fine tuned to follow instructions, process called Instruction Fine-Tuning, or Supervised Fine-Tuning (SFT), are capable of accomplishing a wide range of instruction following tasks. Other instruction following tasks include Q&A, text extraction and text summarization.
Instruction following models must be trained on datasets composed of:

`<instruction, response>` pairs.

This can still be seen as document completion, however now the document is initially just an instruction, and the "completion" portion is the response.
Instruction following training usually occurs as part of an instruction-tuning stage following pre-training. It is considered a supervised fine-tuning (SFT) stage because, unlike the pre-training corpus, the instruction-tuning corpus is often compiled with the careful attention by the model developers. Instruction fine-tuning opens LLMs up to many use-cases without the requirement for highly refined prompting.

1.2.3 Enabling the data flywheel

Maybe an LLM is not being instruction fine-tuned to handle the specific case of interest to you or your business. In this scenario, you can consider fne-tuning the model for your specific use-case. Instruction fine -tuning your own model requires the collection of a supervised training dataset, although the dataset can be significantly smaller than the original pre-training dataset, collecting such a dataset can nonetheless be a challenge. Fortunately, an existing instruction fine tuned model can be used in the creation

of the inctruction training dataset (the existing model can create synthetic in-domain data); (the pre-existing model was trained with out-of-domain use cases). Note that you must ensure that the model permits such usage. In some cases you are also going to need data from actual users.
The data flywheel is:

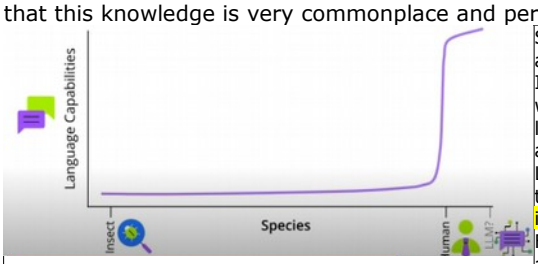
- the existing instruction following model

In general it is hard to collect user feedback for supervised training datasets, particularly explicit user preference data.
For example an e-commerce site, when you purchase an item, asks: How was your service today? You can rate from 1 to 3, with the emoticon icons. This can feel like an unrelated or useless intrusion into an otherwise productive engagement with the e-commerce website. So the user feedback can be limited or biased (?). Whereas, in many chat applications, the user is already in the flow of interacting with the chat system, and is more ready to give explicit feedback.
For example, the user asks to translate Lorem ipsum, the Generative AI site get it wrong, so the user replies with something like "No! I meant translate Lorem ipxum into English." The user answer has a negative sentiment.
So, with generative AI apps, the opportunity for gathering, explicit and high quality user feedback is often available and should not be missed. LLMs can be used for the generation of training data along multiple dimensions, including:

- Generation of responses from pre-existing queries.
  - Enabling instruction fine-tuning dataset pairs:
    `{"prompt": <existing_instruction>, "completion": <LLM_generated_response>}`
    This is probably the most straightforward usage of LLMs for synthetic dataset generation. Ensure that the LLM's license supports such usage.
- Generation of instructions from pre-existing documents. Enabling instruction fine-tuning dataset pairs:
  `{"prompt": <LLM_generated_instruction>, "completion": <existing_document_chunk>}`
  This method, known as back-translation from the development of translation systems for low-resource languages, has been given this modern spin in "Self-Alignment with Instruction Backtranslation (20230814)"). The LLM is used to generate the instructions that are applicable to generate an existing document.
- Generation of preference data from existing prompt/response LLM pairs:
  - For example, the comparison of:
    `{{"prompt": <prompt_0>, "completion": <completion_a>}, "pref": LLM_generated_preference_1}}`
    vs.
    `{{"prompt": <prompt_0>, "completion": <completion_a>}, "pref": LLM_generated_preference_-1}}`
    This can be used for Reinforcement Learning from Human Feedback (despite the feedback being non-human in this case), as explored more in "Constitutional AI: Harmlessness from AI Feedback". Multiple completions are generated, using LLM, the user preference is used in reinforcement learning.

1.2.4 Language fluency versus intelligence

Suppose that we can measure intelligence, the measure is on the Y axis. While the species are on the x-axis. The teacher proposes that LLM intelligence is somewhere in the spectrum from insect to human.
Reversal curse concept is: person A is father of Person B => Person B is child of Person A.
A paper introduced the reversal curse conpect, in which a wide range of LLM(s) could not infer the reversal curse, despite the fact that this knowledge is very commonplace and perhaps even understandable for non human species.



So, it means that the LLM(s) have an intelligence close to an insect, yet it is generally agreed that LLM(s) have human level or super-human level language capabilities.
It is accepted that human language abilities are a step function above all other species, and we humans have constructed our world to exploit our language fluency from written literature that enabled the passing of knowledge though time and space. Such language abilities are also used in modern day services and communication systems.
LLM(s) have been trained on this vast corpus of written human language. Many prompting techniques can be viewed as an effort to partially decouple LLM(s) language capabilities from its intelligence.
For example, increasing the level of context in a prompt, the user can attempt to constrain an LLM to derive its intelligence more from the prompt that from the LLM' learned weights. This requires that the LLM apply its strong language capabilities to this increasingly larger prompt context.

The failure of the reversal curse concept: there are some concepts that are so common that is little literature written about it.

1.2.5 LLM inference and decoding parameters

For example, using OpenAI, there are some parameters:

- **Model**, e.g. davinci, etc. We will use GPT 3.5 Turbo Instruct.
- **Temperature**: consider that the LLM outputs the probability distribution across tokens, for example they can be sorted from the most probable to the least probable. In order to choose the single next token to generate, a decoding mechanism must be specified. In general, decoding means sampling from the probability distribution, the token with the highest probability is the most probabile to be sampled. Increasing the temperature flattens the probability distribution so that it is more probable to sample tokens that are not at the top of the distribution. Increasing the likelihood of sampling from less probable tokens can make the LLM response more creative. An alternative to sampling is **greedy decoding**. Greedy decoding always picks the most probable token. With greedy decoding the shape of the distribution does not matter, only the most probable next token. When the temperature is zero, it causes greedy decoding. LLM inference is not deterministic, so even with low temperature, the response is not always the same, due to the nature of the auto-regressive next token prediction mechanism. When a token A is chosen instead of the token B, this conditions the generation of the next tokens, so the generation heads off in another direction.
- **Maximum length parameter**: a list of tokens is sent to the LLM, each token has an index. The list is composed of:
  - System prompt: setting the tone for the interaction
  - Chat history that may have occurred over the course of the conversation
  - User prompt (the current user prompt)
  - Number of tokens that are pre-allocated for the LLM response, to reach max_tokens. In the Python OpenAI API, maximum length is referred to as `max_tokens`. If the LLM exceeds this number, it truncates the response.
    All the tokens (system prompt, chat history, etc), must fit into the attention window of the LLM, in the order of 2k to 64k, depending on the model being used.

Why cannot the LLM generate past the attention window with some sort of sliding window once it started off the next token prediction? Each newly generated token must be able to attend to all previously generated tokens; moreover there is no mechanism in the standard transformer model, to allow a token to attend to an earlier token, that is outside of the LLM attention window.



- **Top P**: consider the probability distribution of the next token: it means chopping-off everything below some percentage P of the probability mass (remember that the area under the probability curve is 1). The remaining probability distribution is renormalized for sampling the next token. Greedy decoding is unimpacted by changes to the top P.
- **Frequency penalty** and Presence penalty: they serve to reduce the probability of sampling again tokens that have already appeared in the generated sequence, to reduce the likelyhood of repeated sequences. This phenomenon can particularly impact smaller models. These two parameters impact greedy decoding if the most probable token already appeared in the sequence . That is, if the most probable token is hit with a repetition probability.
- **Show probability**: use the legacy completion playground, rather that the chat completion playground. The legacy playground has the "Show probabilities" feature. Introspecting the probability of the generated tokens can provide insights in to the LLM generation process.

Links to OpenAI platform and Chat GPT: https://platform.openai.com/ https://chat.openai.com/
Legacy complete playground: https://platform.openai.com/playground?mode=complete
In the Demo, the teacher sets the temperature to zero (greedy decoding). Show probabilities: full spectrum.
Techniques to experiment with ChatGPT playground: it uses the Completion Playground, because it allows to return the full probabilities, moreover:

- set the temperature to zero, to make the process deterministic
- find a task where you already know the answer, and that can be programmatically evaluated (e.g. anagrams)
- show or return the token probabilties

In the demo, at the beginning, with little context, the GenAI repeated words in a loop; increasing the context, the context part weighted more thatn the already provided anwser (that is used to generate the next tokens).

1.2.6 What is a prompt?

Youtube video of the lesson chunk. At inference and training time, the LLM task is called:"next token prediction".
LLM predicts the next token Y , given X and θ : $P(Y|X,\theta)$, where:

- X is the user prompt: in traditional ML they are called features
- Y is the LLM response; in traditional ML they are called labels/predictions
- θ are the model weights, learned during training; in ML they are called model weights, usually they are frozen except during pre-training and fine tuning. The model weights are similar to the model long-term memory or intelligence. T*he weights* θ *are fixed at inference time, so they are hidden in the formulas below*.

If the task lacks context, the model is forced to rely on learned weights to fill in that missing context, increasing the likelyhood of LLM- generated hallucinations.
With proper prompt techniques, we can also rely on the model weights to provide the LLM with beneficial context. Remember that the *LLM strength is language fluency conditioned on the user prompt*: we exploit it by providing athe LLM a very detailed user prompt (detailed specification of the user request, which can be achieved via prompt augmentation), including a system prompt to give the LLM personality, the chat history to give the LLM knowledge of prior interactions.
The LLM itself can provide more context as well (e.g. the paper REACT: synergizing reasoning and acting in language models).
Chain of thought reasoning: a method can be: prepending the user query with examples of the accomplished task with step-by-step reasoning. Similar results can be achieved by triggering the LLM to think step-by-step, by appending to the user prompt: "`Think step-by-step`". Once discovered that, by appending certain instructions "think step-by-step", the LLM works better, AI engineers can can also employ instruction fine tuning to teach the LLM to perform step-by-step reasoning without any special prompting.
At inference time the LLM produces only one token at a time, while the previously generated tokens are appended to the user prompt. This continues until the end of sequence special token is generated.
At nthstep: $P(Y_{n+1}|X_{0,1,..,n})$, nth+1 step $P(Y_{n+2}|X_{0,1,..,n},Y_{n+1})$,...

nth+k step: $P(Y_{n+k+1}|X_{0,1,..,n},Y_{n+1},Y_{n+2},...,Y_{n+k})$

Revisiting the analogy to the more traditional ML, the LLM has a role in generating its own features: *that is why it is called auto-regressive model*. Such auto-regressive generation can result in both virtuous and vicious feedback loops. Chain of thought prompting serves as an example of virtuous feedback loop. When the LLM starts repeating itself, it is an example of vicious feedback loop. Paper: Chain of thought prompting elicits reasoning in large language models. Paper : Large language Models are Zero-Shot reasoners.
Together AI: used the tarpit.xyz@gmail.com account.

1.2.7 Open vs Closed Models

Open source versus Closed source models: a closed source (or proprietary) model is a black box. The prompt goes in, the response is generated, but it is unknown what is inside. An Open source model is a transparent box: the prompt goes in, the response is produced, but the user has also access to the model weights and the model architecture (you can run inference on your computer), and more rarely you can also have access to the dataset used to train the model. If you know the training dataset you are more aware of the biases that can exist in the model ; there can also be a license that allow the use of the model for commercial purpose. There can also be support to fine tune the model, unlike closed models. Close source models provide limited support for fine-tuning. With open source models you are sure that nothing is changed under the hood, including opaque versions and deprecations, etc. Running open source models require you to have access to inference time computing, which is expensive or not easy to access. Closed models usually provide an API, which is a wonderful convenience, often closed source models have superior performance to open source models. Open source models, fine tuned for specific use cases, have demonstrated superior performance to proprietary models.

We usually do not know the parameter count in close source models, but open source models are released in range of sizes, to fit different computing resources (small, medium, large, etc).

1.2.8 The Anatomy of a well-engineered prompt

Prompt engineering: programmatic construction of a prompt. The attention window of the LLM contains:

- System prompt,
- chat history,
- User prompt,
- Up to max tokens of LLM response.

In the OpenAI completion AI, you pass a list of Json elements, that map to the above prompt components. The list must be flattened into a single string to be fed into a LLM transformer model. The string provides the LLM with all the context of the current conversation, and the memory of the prior conversation.
The OpenAI completion API takes care of the chat history management for you. Triggering: appending specific tokens to the user prompt. All what is fed into the LLM (system prompt, history, user prompt) must be kept within the size of the attention window.
You can constrain the LLM to respond in a specific way by appenting specific tokens to the prompt. This method is called triggering For example Lama developers suggest to delimit a request to write some code with the tags [INST] [/INST]. and then append `'''python` Remember that an LLM is a document completion model, so `'''python` is commonly at the beginning of code blocks in markdown. This forces the LLM to generate code without explanatory text.

1.2.9 Exercise: Chatbot Memory

Video: https://youtu.be/KNOlK97iZe8

1.2.10 Understanding LLM capabilities and Using your intuition

Some tasks are not well matched to LLM capabilities: $P(Y|X,\theta)$ we are asking the LLM to predict Y, given the prompt X and the learned weights θ. If we provide limited context in X, the LLM relies more on the learned weights θ.
Example, recalling a research topic citation at the top of your head (full, with authors, date, etc.). At least a paper citation is composed of natural language, now the LLM are requested to memorize URL(s) too. Without access to external tools, the LLM often produces allucinated results At least an invalid URL is easy to detect, while other allucinated generations are heder to invalidate (automatically?), domain expertise is required to invalidate a response about cold fusion. We must give access the LLM to a search engine.
Another challenging task for LLM(s) is basic math. ChatGPT 4 has access to a runtime environment to write and execute a prompt to solve the math query. So, the ChatGPT generates a program that is then executed by a runtime Python prompt to generate the answer.
We can develop an intuition about the next token mechanism, because the LLM(s) are trained on human generated text, just like the text we read every day. What is the common cause of both: excessive newlines and excessive verbosity? For example the common cause can be the less space-constrained format. For example, newspapers and Twitter do not have excessive newlines and extra verbosity. So the hypothesis is:

- More space constrained => fewer newlines, more concise
- Less space constrained => more newlines, more verbose.

It means that if we add extra newlines or we are more verbose, the LLM makes more use of related verbose texts, so the answer will likely be more verbose as well.
https://platform.openai.com/tokenizer A token corresponds to about 4 characters of text.

1.2.11 Lesson References

- Paper: Attention is All you need
- Illustrated Transformer
- Annotated Transformer
- Paper: BERT
- GPT model
- Paper: Training Language Models
- Paper: Survey paper on instruction tuning
- Paper: Constitutional AI: Hermlessness from AI feedback
- Paper Curious case of neural Text degeneration
- Paper: Contrastive Chain-of-Thought Prompting
- Paper: A Systematic Survey of Prompt Engineering in Large Language Models: Techniques and Applications

1.3 Course NLP Fundamentals

1.3.1.1 What is NLP?

A natural language is a language that evolved naturally through human communication, such as Spanish, Mandarin, or the American Sign Language. Natural languages include pragmatic concepts, that are communicated, but also more subtle concepts that rely on the culture and social structure of the speaker. There are also constructed languages, like programming languages, called structured languages, created to reduce ambiguity, but they lack the creativity that comes with natural languages.
A structured language is an invented or constructed language, such as a computer programming language.
It is difficult to process natural language with computers. NLP is the bridge between natural languages and computers. NLP is at the intersection between: computer science, linguistics, AI.
In our world, data is almost textual, mobile devices are often voice operated, communication mostly digital.
NLP reveals structure and meaning from human language to computers and its importance has grown in the modern age.

1.3.2 NLP applications and tasks

- Speech recognition: takes audio samples and and turns it into a context. Audio is turned into text data before applying other NLP techniques.
- Text classification: e.g. classify emails as either spam or not
- Automatic machine translation, not easy because the translation must capture the full context of the original text.
- Text summarization: taking a document or a set of documents and summarize them while preserving the original essence

of the source. Summaries are reposes from the language model, that can be either extractive or abstractive. An extractive summary takes snippets of text from document sources and quotes them in the summary. Abstractive summaries are where the text is not directly quoted from the sources. The main points are summarized using novel words and phrases instead of quotes.

- Question answering is taking a question in natural language and providing a response using a document or set of documents: similar to text summarization, it can be extractive or abstractive.
- Chat bots and conversational agent systems create a dialog in natural language, for the purpose of: entertainment, support, iformation retrieval.

1.3.3 Challenges in NLP

Natural languages seem simple because we use them every day: Language is complex, nuanced, and ambiguous, making it challenging for computers to process language data. NLP Needs (some) Clean and Labeled Text Data. Data can have misspellings, unconventional words, and biases that can be difficult and expensive for humans to label.

Language is complex. It is difficult for the computer to parse. Example `The mouse moved across the desk`. "mouse" here means the device, not the animal. **Language is nuanced**: there are multiple ways to interpret the same sentence. Idioms, slangs, sarcasm, are difficult for a computer to understand, they depend on cultural knowledge or shared experiences that computers do not have.

Ambiguity: `The trophy does not fit into the purple suitcase because it is too small`. What is small? The suitcase or the trophy? The suitcase. If it was: `The trophy does not fit into the purple suitcase because it is too large`. Now it refers to the trophy. You must have some knowledge of the world to make sense of the phrase.

Availability of Clean and labeled text data:

- Misspellings: do we want to correct them or keep them? Inconventional spellings can carry context
- Decisions you make can introduce biases into datasets, even pure untreated text data can carry cultural biases from the real world
- It can be difficult and expensive to get labelled and curated data: some successful projects avoided the use of human labelers, instead using automatic strategies to label data or predict missing words. However, sometimes the human labeller is necessary

1.3.4 Encoding Text Data

Computers don't understand raw language and need to translate text into computer-usable format

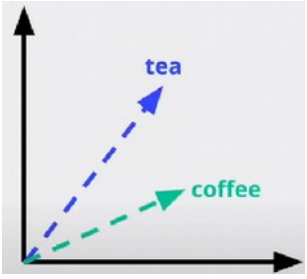
To do NLP, we have to translate text into something usable by the computer. Strategies to encode text fall into a spectrum:

- each letter encoded as a numerical value: the text is encoded close to its raw form, we have the exact information, but no context, the context must be then provided by the NLP system
- we can incorporate the context in the input encoding, for example we can have a single value representing the context of the sentence, for example the sentence sentiment, whether it is positive or negative.

Between these tho extremes, we want small pieces, but still include the context of the text. Two approaches are tokenization and embeddings.



Tokenization: tokens are chunks, that are given a numerical representation that computer algorithms can work with. The way we tokenize the text affects how easy or difficult is for later steps to extract the context. Text is broken into tokens, or chunks.
Tokenization breaks text into chunks called "tokens" and encodes these as numerical representations



Embeddings: encoding context into a vector representation. This technique goes beyond NLP, but it has been proved to be quite useful in NLP. Embeddings can incorporate context and relationships from this representation.
Embeddings encode context into a vector representation.

Tokenization and embedding are frequently used together in NLP because they complement each other. Tokenization breaks text into chunks, embeddings are a richer representation that can incorporate context of the texts.

1.3.5 Tokenization steps

Tokenization transform text into a representation that the computer can understand. During tokenization we can also extract context from the text. Steps are:

- Normalization cleans the text for consistency. Normalization makes later processing simpler, but the actual steps depend on the task, (remember that you may lose context). The output of the normalization is still text, e.g. a single string, but single characters can be different. For example:
  - set all text into lowercase
  - remove punctuation
  - replace accented characters, like è, é, ò, à and replace them with non accented letters
  - recent NLP systems like LLM(s) can deal with unnormalized text, keeping more complexity, to retain context. Example, punctuation is retained to keep more context. Do not clean information if it is relevant to the task. Example: hashtags, emoticons, etc.
- Pre-tokenization: break text into smaller pieces. Split text into words, e g Do you prefer coffee or tea?. The tokens are Do, you, prefer, etc. In some languages like English words are separated by spaces, they are futher split in later steps. Other languages need that words are separated with idfferent rules. Pre-tokenization: split text into pre-tokens separated by whitespaces.
- Tokenization
- Post-processing: it can add starting and ending special-tokens.

1.3.6 HF Tokenizers

You can use pretrained tokenizers: off the shelf, fine-tune, or train from scratch.

Note that, for example BERT tokenizer uses sub-token tokenization. Example the word Magenta is split into the tokens: `Mage`, `##nta`. Note that it is possible to have the tokenizer to return special tokens: `[CLS]`, `[SEP]`. They are used by NLP models.

```
my_tokenizer.tokenize( raw_text)                  # Split into tokens, no detailed tokens [CLS], [SEP], etc.
detailed_tokens = my_tokenizer(raw_text).tokens() # Returns the detailed tokens as strings.
print(detailed_tokens)

['[CLS]', 'Rory', "'", 's', 'shoes', 'are', 'mage', '##nta', 'and', 'so', 'are', 'Corey', "'", 's', 'but',
'they', 'aren', "'", 't', 'nearly', 'as', 'dark', '!', '[SEP]']

my_tokenizer.encode(raw_text) # Returns the detailed token ID(s) from the text string.
my_tokenizer.convert_tokens_to_ids(detailed_tokens) # Returns the token ID(s) from the detailed tokens.
my_tokenizer(raw_text)        # Returns an object containing input_ids, token_type_ids, attention_mask.
```

[UNK] means unknown token, for example with emojis, etc. Strings out of vocabulary.
If you get a lot of unknown tokens, you can:

- pick a different tokenizer that recognies those unknown tokens
- fine-tune the pre-trained model
- train a model from scratch

When we encode an unknown token, and then we decode it back, we lose the information of the original token. It is decoded back as [UNK]. The information about the emoji is lost. Links: PreTrainedTokenizer, AutoTokenizer

1.3.7 Properties of HF Tokenizers

```
bert_base=  AutoTokenizer.from_pretrained('bert-base-cased')
bert_base.model_max_tokens   # The maximum number of tokens the tokenizer can encode.
bert_base.all_special_tokens # Special tokens. Example: [UNK]', '[SEP]', '[PAD]', '[CLS]', '[MASK]
```

Max length. Different tokenizers will handle some text better, such as longer input sequences. The `.model_max_length` property of the tokenizer object will tell you the maximum length the model can handle. If the length of your data exceeds the maximum length of your tokenizer, you may need to chunk the data before tokenizing it. Or you could consider switching to a different tokenizer that has a longer maximum length.
Special tokens. Different tokenizers will have different special tokens defined. They might have tokens representing:

- Unknown token
- Beginning of sequence token
- Separator token
- Token used for padding
- Classifier token
- Token used for masking values

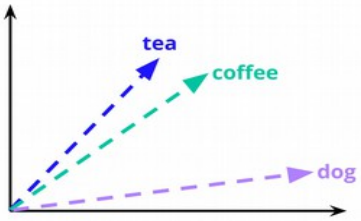
Additionally, there may be multiple subtypes of each special token. For example, some tokenizers have multiple different unknown tokens (e.g. `<unk>` and `<unk_2>`).

- CLS= Class Input. Specific for BERT.
- BOS= Beginning of Sequence
- EOS= End of Sequence
- SEP= Sentence Separator; in some models End of Sequence is the same as Sentence Separator
- CLS= Class of input: in some models the token for Class of input is the same as the begining of sequence
- UNK= Unknown token

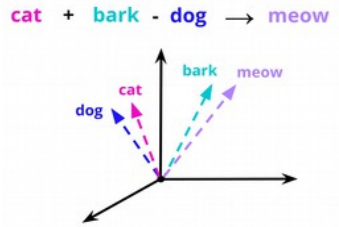
Tokenizers can be very different from one another, beware of special tokens, consider long sequences against the maximum number of tokens. Token ID: the value has no meaning, it is just a way to identify a token with a number.

1.3.8 Embeddings

Embeddings vectorize the text tokens to create representational vectors. These values aren't random, but instead they are created specifically and result in contextual information being encoded. There are classic vectorization methods, like Bag of Words, One-Hot encoding, and TF-IDF, to create vectors, but such methods lack the deep contextual information we're looking for. Vectorization methods that utilize deep learning produce vectors called **embeddings**.
The vector representation of embeddings live in a vector space. The techniques to build these embeddings can create vectors that encode context in the vector space itself. Words close to one another are more similar than those further away. Embeddings have an arbitrary number of dimensions. Embeddings below are 3D, just as an example



Vectors close to each other are more similar



Relationships between words/vectors are encoded in the vector space

$$(bark - dog) \sim (meow - cat)$$

These relationships are learned from the text when creating the embeddings, you can reuse a pre-trained embedding on a different text. Note that it might not give the best results if your text greatly differs from the text where the embeddings were learned from.

1.3.9 Encoding Text Data exercise

Adding `: str` inside the parentheses tells us that text is expected to be a string, and adding `-> str` after the parentheses tells us that the function's return value. Example:

```
def normalize_text(text: str) -> str:
  pass
```

Python documentation on typing. NOTE:

```
from typing import NewType

UserId = NewType('UserId', int)
some_id = UserId(524313)
```

1.3.10 Encoding Text Data

Encoding is composed of:

- normalization
- pre-tokenization
- tokenization
- post-processing (add [BOS_TOKEN, tokens, …, EOS_TOKEN]
- mapping tokens to ID(s)

Tokenization methods vary and have their pros and cons.

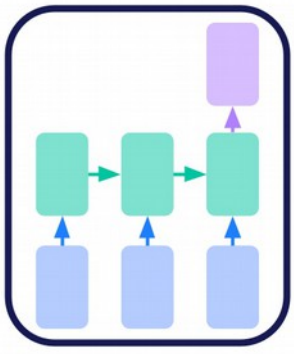
- Character tokenization results in a smaller vocabulary but can be harder for downstream tasks.
- Word tokenization results in a larger vocabulary with more out-of-vocabulary tokens.
- Subword tokenization is a balance between small and large tokens where frequent words are not split and rare words are broken down, keeping the vocabulary size manageable.

Embeddings represent tokens as vectors that incorporate the context of the text in its vector space.

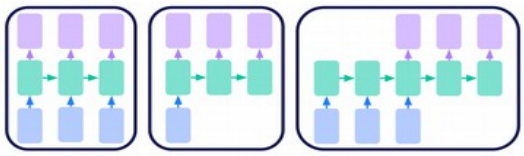
1.3.11 NLP Models and Handling sequences

So far we encoded our data into tokens and embeddings; now we can feed our encoding into the model, for a specific NLP task. There is a variety of approaches: deep learning models when you have a decent amount of data. We think of text as a sequence. In some case all the sequence is mapped to a value, for example text classification like sentiment analysis. In other cases the output is also a sequence: this is referred to as sequence-to-sequence (machine translation, chatbot, text generation are all sequence to sequence tasks).

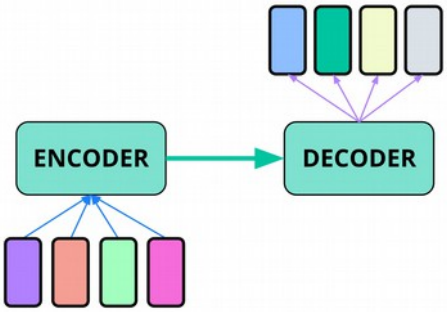
A common way to build seq-to-seq models is to take-in a sequence, and encode the input into some representation of the inputs context; that is then used (decoded) to generate the output sequence. So, the two pieces are referred-to as encoder-decoder.



Models can have an architecture where an input sequence maps to a single value.



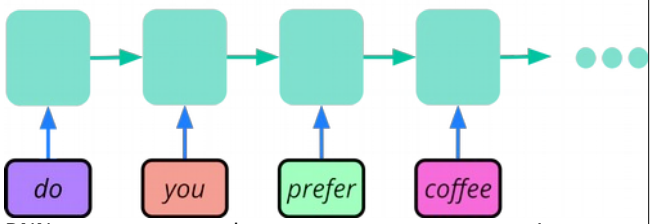
Various other models can have a "sequence-to-sequence" architecture where an input sequence maps to an output sequence.



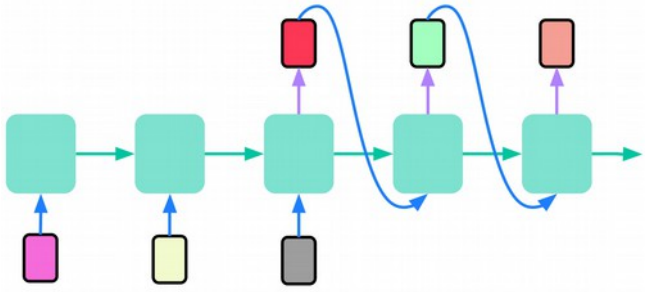
Simple depiction of an encoder-decoder model taking in an input sequence and outputting a new sequence.

1.3.12 Recurrent Neural Networks (RNN(s))

Text can be encoded as a sequence. RNN(s) take text one piece at a time. In RNN(s), there is dependence on previous inputs. On each token, there is an output and some information that feeds an hidden state, that is fed back into the model with the next input. This process continues until the stop condition is reached. In some case, the next input to the RNN is simply the output of the previous stage, so there is no new input.



RNNs process text data as a sequence, one piece at a time, with a dependence on previous inputs and using a "hidden state" to pass past information further down the chain.



RNNs can generate new text indefinitely by feeding output as input. We can have an RNN(s) that reads all the input. At the end, the hidden state contains information about all tokens in input. Thus, with the hidden state it is possible to run the decoder and generate the output tokens. In practice, RNN(s) have problems in retaining all the context in the hidden state. This is similar to the Encoder-Decoder architecture.

Disadvantages of RNNs

- The vanishing gradient problem is when the RNN has trouble "remembering" distant/earlier inputs. It is difficult to keep

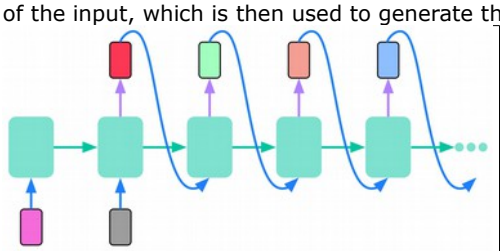
all the relevant information in the hidden state.
- RNNs are sequential in nature, which makes training and usage slow, especially when the data are large.

In the field of NLP, a better architecture, that does not suffer of the disadvantages of the RNN(s), Transformers that use the Attention mechanism have become the preferred model.

1.3.13 Autoregressive Model

Text generation in NLP is when the model produces novel text, for example:
- abstractive answer to a question
- chat bot conversation
- text generation with a particular style or tone in mind

When generating new text, the text must capture the essence of what we are trying to generate. A chatbot would be very boring if we generated the same text over and over again, no matter what we say. So, there must be some randomization in the generation process. We want interesting responses, but still relevant to the text input. One such way is by using an autoregressive model. They generate one token at-a-time, after the input is given. All the input is used to generate the context of the input, which is then used to generate the output. During generation, the output is fed-back to the input.



Autoregressive models take in one input at a time and use previously generated tokens as input to create potentially very long output sequences.
A stochastic process is used to generate the text by choosing at random the next token based on the token's probability weight.

The context, in generation, must keep track of the input context and the previously generated tokens.

The next token is selected by randomly choosing from a set of tokens. Each possible token is weighted by how likely the model thinks that that token will be the next one. The token weights are referred to as probabilities. The process of choosing a token is called **sampling**. The model thus uses a stochastic process to generate the next output. For example, we can have the tokens: the cat sat on the ….” The model can say “mat” and “rug” can be the next word, depending on how the model was trained.

1.3.14 Sampling Methods for Tokens

Sampling the next token: however, autoregressive models tend to repeat themselves, this is because the newly generated token is based on the previous tokens. When a new token is generated, the model can thus consider: the context (the previous tokens) and the last token; the model might focus on the last token, thus repeating itself. There are methods to prevent that. One method is based on the temperature. Decreasing the temperature = decreasing randomness. It means that the most likely tokens are chosen almost every time. In the extreme case of very low temperature, only the most likely token is selected. Increasing the temperature means increasing the randomness. Increate the temperature if you want the model to be more creative or generate novel text. Even at low temperature, there is still a chance that an unlikely token is chosen.
Top-k: restricts the choice to the top **k** tokens. We set the number of most likely tokens to pick from. Example k=10: only the ten most likely tokens are considered when sampling for the next token. In themost extrame case, with k=1, we always pick the most likely token every single time. Top-k and temperature are best used together. Temperature sets the randomness, top-k restricts the choice, so it limits how much creative the generated output can be.
Nucleus sampling, or top-p sampling, is similar to top-k sampling, but instead we have dynamic cutoff for the most likely tokens to sample from. **Beam search** which doesn't just consider how likely the next token is, but also a string of 2, 3, 4, or more tokens are to occur after the previous texts.
- Temperature: adjusts the randomness in choosing the next token
- Top-k sampling: samples from only the k most likely tokens
- Nucleus or top-p sampling: uses a dynamic cutoff for sampling the most likely tokens (cumulative probability is under p)
- Beam search: considers the likelihood of strings of multiple tokens instead of just a single next token

1.3.15 Recap of the course

Sequence-to-sequence models can be used in NLP since we can think of text as a sequence.
Recurrent neural networks, or RNNs, are a deep learning model that can work well for sequences by taking in one token at a time to produce a hidden state that condenses past information so it can be used further down the line.
Disadvantages of RNNs include:
- The vanishing gradient problem where tokens earlier in the sequence are "forgotten"
- Long training and slow usage because of RNNs' sequential nature

RNNs can be used as autoregressive models, which can generate new text indefinitely by feeding in input text and the newly generated tokens.
Sampling techniques can adjust how tokens are chosen by the autoregressive model during text generation.
- Temperature: Adjusts the randomness in choosing the next token
- Top-k sampling: Samples from only the k most likely tokens
- Nucleus or top-p sampling: Uses a dynamic cutoff for sampling the most likely tokens (cumulative probability is under p)
- Beam search: Considers the likelihood of strings of multiple tokens instead of just a single next token

1.4 Course: Transformers and Attention Mechanism

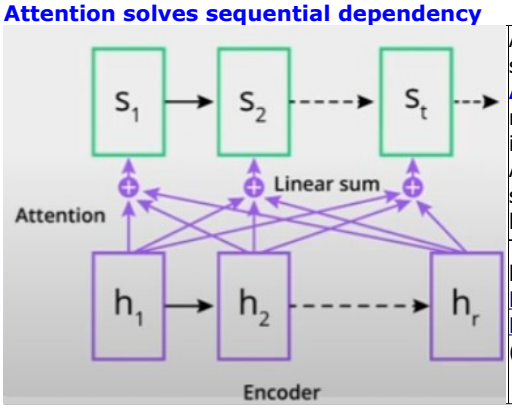
In this lesson we open-up the box of transformers and attention mechanism. Unlike other forms of predictive AI, generative AI generates content unseen before, often with unprecedented creativity. Underlying the techinology there are two fundamentals innovations: the transformer architecture, as opposed to RNN(s); the diffusion models replacing GAN(s).

1.4.1 Origin Story: Attention and RNN(s)

Limitations of RNN(s):
- they process tokens one-by-one, into a single context vector, or hidden state, so RNN(s) have a vanishing gradient problem, where the hidden state can gradually forget information over long sequences.
- LSTM (Long Short-Term Memory) were an improvement over RNN(s), by introducing a forget gate and an input gate, the neural nettwork can control what to remember and what to forget, but still it has serial dependency, which presents an

information bottleneck for very long sequences.
RNN(s) in general are not parallelizable or GPU-friendly, so they cannot take advantage of GPU(s) parallelism.
Another milestone was Embeddings from Language Models (ELMo) (*see the paper: Peters, Matthew et al. "Deep Contextualized Word Representations"* Link to paper). Idea is that a word can have different meaning based on its surroundings. This method become state of the art (2018) on many NLP tasks.
Limitations of RNNs. Recurrent neural networks (RNNs) were once the architecture of choice for many NLP tasks. However, they are limited because they can only keep track of a single hidden state (context vector), which creates an information bottleneck and causes gradual "forgetting" over long sequences. Long short-term memory (LSTM) models were developed to address the problem of gradual forgetting using a forget gate and an input gate. This manages the "memory" and "forgetting" more intentionally. LSTMs are not a complete solution for these challenges because they are still built on the underlying RNN architecture. This means that they still have information bottlenecks and can't use the built-in parallelism of GPUs.
ELMo. Embeddings from Language Models (ELMo) built on LSTMs to create a new state-of-the-art in 2018. In particular, it was able to create contextual embeddings that could differentiate between the same word being used in different contexts.
ELMo also introduced the large-scale pre-training that we see with modern LLMs.

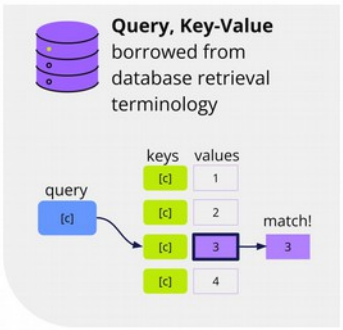
Attention solves sequential dependency

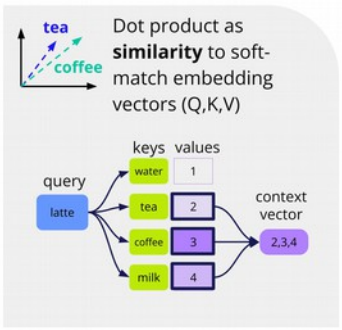


At each step, the NN learns to combine all hidden states (attends to all hidden states) to create its context vector.
Attention= weighting and combining hidden states, focusing selectively on relevant inputs. It means that the decoder can pick and choose how much of each input token can pay attention to.
Attention was first applied to language models in 2014. Instead of relying on a single context vector, attention allowed the model to combine all of the encoder's hidden states to become its context vector.
This raised the question: instead of trying to apply attention to RNNs, can we do better with just attention?
Paper §6.1.2 "Deep contextualized word representations - ELMo 201806"
Paper §6.1.1 "Neural Machine Translation by Jointly Learning to Align and Translate (Y.Bengio et al, 201409)" introduced the attention mechanism in translation.

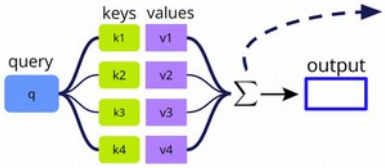
1.4.2 Attention Scores (= how to calculate attention)

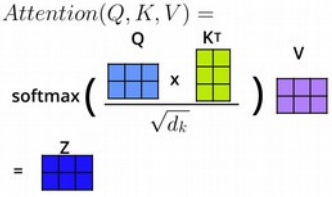
Query-key terms are borrowed from database terminology. To query the DB, you find its matching key and use it to index your value in storage. (E.g. hash map)





The dot product is used as similarity to soft-match embedding vectors (Q,K,V).
Attention is based on a similarity measure between word vectors. The same query is matched against multiple keys with different weights at once.
Match a query with multiple keys with different weights at once.

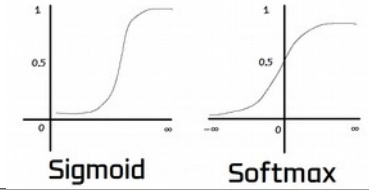




Query and values embeddings are vectors of n dimensions.
The attention weights of scores are computed as an n-by-n similarity matrix between queries and keys.
Then a softmax is applied to convert the scores into a probability distribution which is multiplied by the Values matrix.
Note that Softmax in an activation function that scales numbers into probabilities. Softmax is a generalization of the sigmoid to multiple classes.

$$Attention(Q, K, V) = softmax(\frac{QK^T}{\sqrt{d_k}})V$$, where d_k : dimension of K

The product of matrices is divided by $\sqrt{d_k}$, because gradients became small at extreme values of X applied to the softmax function, so we need to rescale QK^T, by dividing it.



When calculating attention, the entire block $softmax(\frac{QK^T}{\sqrt{d_k}})$ is just an attention score to multiply the value vectors matrix V.

Many ways to calculate attention

- Multiplicative attention: $Attention(Q, K, V) = softmax(\frac{QK^T}{\sqrt{d_k}})V$, aka dot product attention, or Luong attention, is used in transformers like BERT and GPT: it is fast and space efficient. Dimensions of Q , K , V are the same.
- Additive attention Bahdanau attention, this was the original attention , it allows for flexible query, key dimensions, valuable to neural machine translation NMT $Attention(Q, K, V) = softmax(W_2 \tanh(QW_1 + (KW_3)^T))V$
- General Attention $Attention(Q, K, V) = softmax(QW_g K^T)V$: flexible query and key dimensions, better speed and space efficiency than additive, less efficient than multiplicative attention.

1.4.3 Attention mechanisms

Attention mechanisms: it deals with "what is the source and target of attention?". In Self attention, $Q = V = K$, Q,V,K are from the same domain or distribution. Typically, in self-attention, K,Q,V come from the last layer of the output (*the previous layer output?*).
Usually attention is in a single sequence; each position attends to all positions of the same sequence. The, Q,K,V are fed into a single linear network. Note that Self-attention has interaction distance of $O(1)$, but a computational cost of $O(n^2)$, where n is the sequence length. There are other attention mechanisms that can be used or combined:
Multi-head attention: multiple instances of attention, each one can focus on a particular set of features. Each head has its set of weights.
Multi-query attention: multiple heads share the same K and V. It means that multiple Q address the same K and V. It is more efficient. It can increase the training batch size through parameter sharing. Billion parameter models all use multi-query attention.
Cross-attention: Q is not the same as K and V. This mechanism is used in multimodal models.
Question: can attention replace recurrent models,e.g. RNN(s)? Answer: not directly. There are three problems, they all have solutions.

Problem	Solution
Lack of input order	Positional encodings are fed as part of the input embeddings.
Lack of non linearity between repeated self-attention layers.	We add a feed-forward network for each processed token.
By default, self-attention can look into the future, including the "future" that it is trying to predict. The goal of a deep-learning model is to predict something "unknown" given some known information. If all the information is known, the model cannot learn the relationships properly.	We mask attention on future words during decoding.

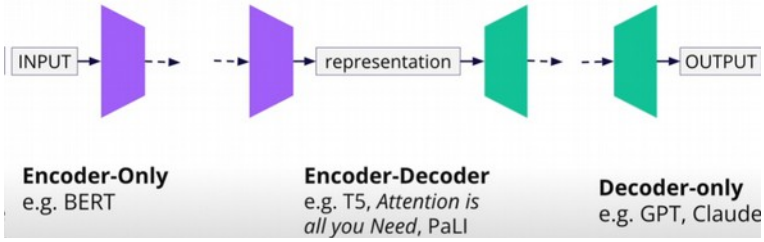
Terminology note: when some part of a neural network applies an attention score or mechanism to some information, you might think that the standard phrasing would be "A pays attention to B". Instead, we typically say that "A attends to B". This standard terminology is more concise and less anthropomorphizing.
Resources:

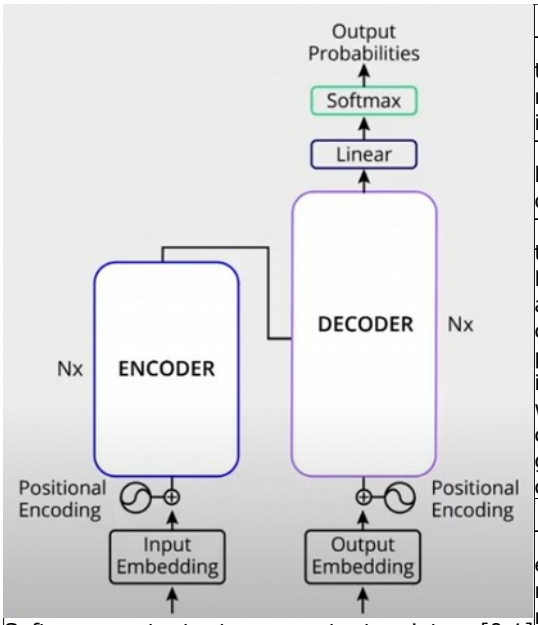
- https://arxiv.org/abs/1606.01933v2
- https://arxiv.org/abs/1706.03762
- https://arxiv.org/abs/2306.09380
- https://huggingface.co/docs/transformers/main/model_doc/falcon
- https://huggingface.co/docs/transformers/model_doc/llama2
- https://arxiv.org/abs/2303.02995

1.4.4 Transformers

Transformer architectures consist of building blocks of self-attention layers. Different types of transformer architecture:

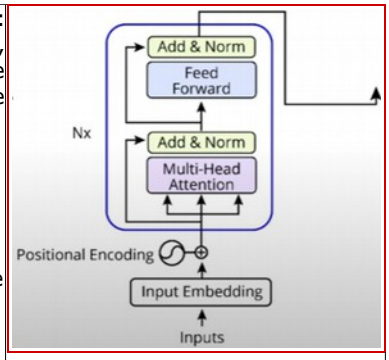
- encoder only (e.g.BERT)
- decode-only (e.g. GPT, Claude)
- encoder-decoder, e.g.T5



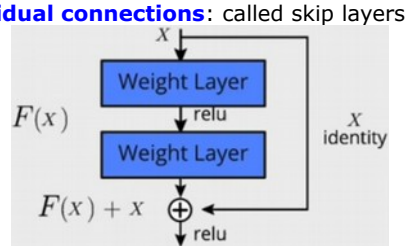


Softmax output turns outputs into [0,1] probability distribution over vocabulary.

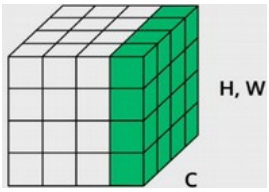
Encoder:
The input is paired with positional encoding, then multi-head self-attention module. The multi-head attention module learns the intermediate representation.
Then there is a normalization and residual layer, which smoothens optimization. The output of the encoder are embeddings.
Transformers operates on sets, by nature, there is no notion of position.
Positional embedding, is a vector of same size as the input, added to the input. The classical position encoder, called absolute positional embeddings converts the position index of each tokens with one of several ways, the straighforward one uses sine and cosine waves to parameterize the position, given the periodic nature of the sine and cosine.

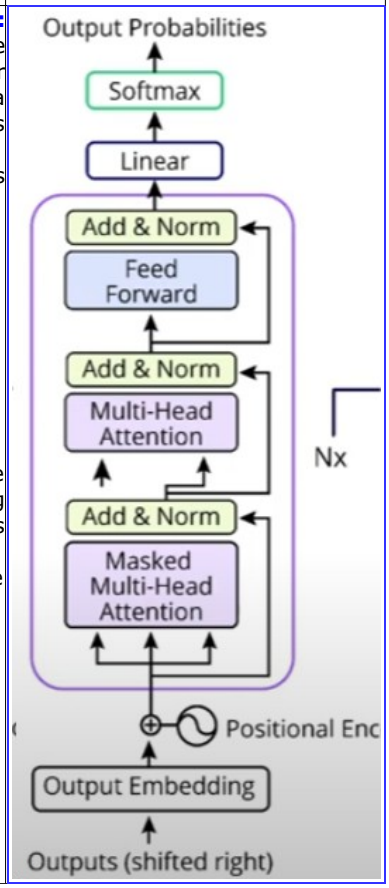


Decoder:
The decoder takes the embeddings from the encoder and decodes them using a similar mechanism. In the decoder there is a masked multi head attention that prevents the decoder from looking into the future
Other ingredients are residual connections and normallization.
Residual connections: called skip layers



The input X is added to the output after the attention layer, this addresses the vanishing gradient problem. The technique was introduced in computer vision.
Layer normalization: also helps training the model faster, it normalizes each layer to unit mean and standard dev. It is common in sequence models.





- https://arxiv.org/abs/1810.04805
- https://cdn.openai.com/research-covers/language-unsupervised/language_understanding_paper.pdf
- https://arxiv.org/abs/2112.11577
- https://arxiv.org/abs/2106.03143
- https://arxiv.org/abs/2104.09864
- https://arxiv.org/abs/1512.03385

1.4.5 Training Transformer-Based models

Each kind of transformer has it own pre-training objective, or task definitions.

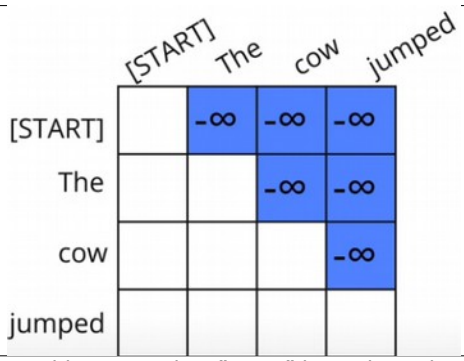
- The most common is autoregressive transformers (AR), they predict the next token using their own last output, e.g. GPT
- The second is denoising autoencoders, e.g. BERT, they corrupt the input and then they recover using their surroundings. For transformer LLM(s), this means applying mask language modeling.
- Contrastive objective, e.g. SentenceBERT, Image ↔ text clip, commonly used to aligh different inputs or views of the same input, for tasks such as calculating sentence similarity or text and image models like CLIP. Contrastive models work by training the model to separate positive and negative pairs. The models constructs (positive, negative) pairs. Trained on internet-scale data, this is what made Dall-E so effective.

Some pre-training objectives are better than others for self-supervised learning: it depends on whether the ground truth can be constructed within the data structure, or whether it requires manual annotation.

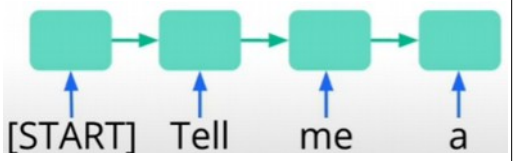
Autoregressive (AR) model.
GPT stands for generative pre-trained transformer. It has an autoregressive objective and is also called a causal language model.
Autoregressive means that it predicts the next word based on its own previous value. It assumes that there is some continuity, or dependency, between a value and its predecessors. This is a common assumption in sequential data, where the next word in a sentence is based on the previous words.



Note that, in a causal language model, the attention scores of future tokens are set to $-\infty$, because we do not want the decoder to look into the future and cheat during its prediction. When softmax is applied to negative infinity, you get zero. $softmax(-\infty)=0$



If you say "Tell me a story about family", "story about familiy" will not be visible. We select "story" by picking the word with the highest probability candidate amond: secret, joke, life, tale, etc.



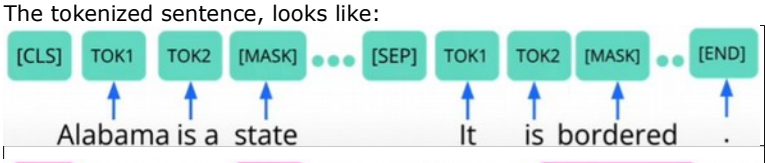
Probability if the next word:



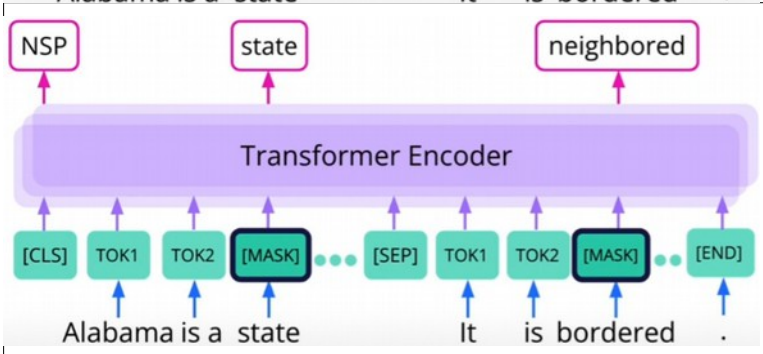
The model can take the word wrong and accumulate its mistakes. That is why, during training, we often use a technique called "teaching forcing", by using a ground truth label to predict the next token. E.g. the word joke is forced instead of story. Teaching forcing prevents the model from accumulatig mistakes and continuing on a vicious feedback loop during training.
How is BERT pre-trained, in more detail. The original BERT used objectives:

- mask language modeling
- next sentence prediction

We call them self supervised learning, because ground truth labels can be automatically constructed from within the data structure, without any manual annotation. Note that next sentence prediction is different from next token prediction. Next sentence predition is a binary task of true and false, whereas next token prediction is more about predicting the actual token.
Specifically, the BERT encoder is trained with a pair of sentences. Example: Alabama is a state in the US. It is bordered by Tennessee.
The tokenized sentence, looks like:



During BERT training, a given percentage of the tokens are masked, so the word "state" is masked, along with "bordered". So, for masked language modeling, the task of the model is to predict these masked words.



By mistake, the model can predict "neighbored" instead of "bordered". This is a common error that may occur.

The other task is sentence prediction This is asking, for example, if "It is bordered by Tennesse." comes next.. Both of these pre-training objectives mean that BERT is primed to do well at classification, because it can predict the correct class with binary classification and with fixed size vocabulary.
Modern day transformers are typicall trained with two phases.

- Pre-training, also called self-supervised learning, to produce a general purpose checkpoint.
- Fine-tuning for domain specific use-cases: further training the model on additional label data, especially useful for

classification.
For more open-ended dialogue-like task that does not require additional training, it does require the user to feed a prefix or prompt into the model. We can also use labeled data along with prompting, in a retrieval augmented generation (RAG) setup.
A large model can be prohibitely expensive due to the amount of unlabeled data needed to exceed the performance of a smalles supervised model.
https://cdn.openai.com/research-covers/language-unsupervised/language_understanding_paper.pdf
https://direct.mit.edu/neco/article-abstract/1/2/270/5490/A-Learning-Algorithm-for-Continually-Running-Fully
https://arxiv.org/abs/1810.04805

Other references from the notebook:
[1] Radford, A., Narasimhan, K., Salimans, T., and Sutskever, I. Improving language understanding with unsupervised learning. Technical report, OpenAI (2018).
[2] He, K., Zhang, X., Ren, S., & Sun, J. (2015). Deep Residual Learning for Image Recognition. 2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR).

1.4.6 The Pros and Cons of Transformers

Transformers take advantage of GPU processing power. Software optimization: software DL frameworks, like PyTorch and JAX, both use CUDA to compute matrix multiplication i parallel. Matrix multiplication is the atomic boulding block of neural networks., such operation is decomposible into small calculation units.

	Transformers	RNN(s)/LSTM(s)
Training Parallelism	**Yes**	No
Long range memory	**Yes**	No: vanishing gradients problem
Input type flexibility	**Yes**	Sequence only input
Interpretability	**Yes** (see the matrix §6.1.1.1 and §1.4.8.2) Pairwise scores between tokens can be visualized	No: the hidden state cannot be interpreted
Computational complexity	**Quadratic**	Better
Token length limit	**4k – 32k+**	No limit

1.4.7 The Future of Transformers

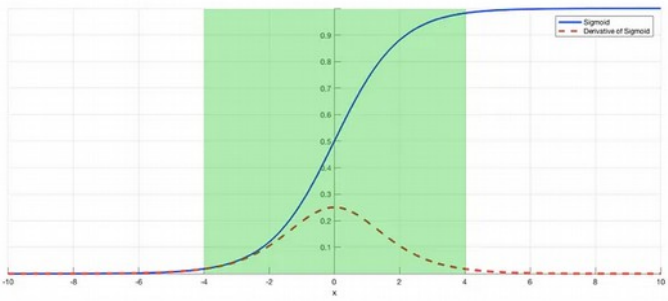
Referenced links:

- https://arxiv.org/abs/2305.13230
- https://huggingface.co/docs/peft/conceptual_guides/lora
- https://arxiv.org/abs/2312.10997v1
- https://youtu.be/dKJEpOtVgXc
- https://hazyresearch.stanford.edu/blog/2023-12-11-truly-subquadratic

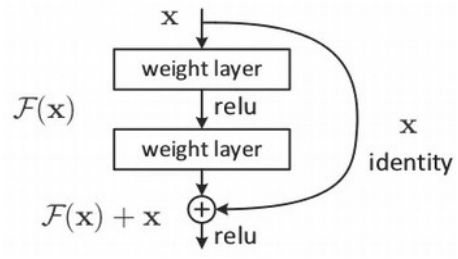
1.4.8 Referenced material

1.4.8.1 The Vanishing gradient problem

Towards Data Science



Vanishing gradient is a problem for deep NN(s).
Backpropagation finds the derivatives of the network by moving layer by layer from the final layer to the initial one. By the chain rule, the derivatives of each layer are multiplied down the network (from the final layer to the initial) to compute the derivatives of the initial layers.
However, when n hidden layers use an activation like the sigmoid function, n small derivatives are multiplied together. Thus, the gradient decreases exponentially as we propagate down to the initial layers.
A small gradient means that the weights and biases of the initial layers will not be updated effectively with each training session. Since these initial layers are often crucial to recognizing the core elements of the input data, it can lead to overall inaccuracy of the whole network.



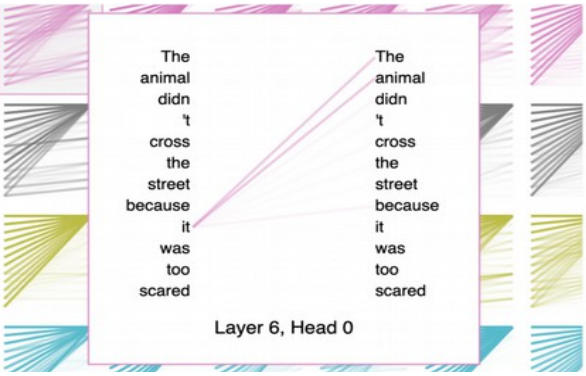
Solutions: you can use another activation function, such as the ReLU.
Other solution: residual networks. The output of a residual layer is: $ReLU\left(F\left(x\right)+x\right)$. Residual connections do not go through activation functions that squash the derivatives.
Other solution: Batch normalization. The input is normalized so that it does not reach the outer edges of the sigmoid function. In the green region of the picture above, the gradient is not small.

1.4.8.2 Explainable AI: Visualizing Attention in Transformers (Summary of transfs.&att.)

Link to Comet blog. The blog article describes **BertViz**, an open source tool that visualizes the attention mechanism. Very good summary of transformers and attention.
Transformers can be used with image, audio, and virtually any other sequential data. The key feature of transformers is the attention mechanism. Attention allows for larger context than recurrence, so transformers can craft more coherent, relevant and complex outputs.

- Encoder only: auto-encoding models: sentence classification, extractive question answering, masked language modeling
- Decoder only: auto-regressive models: text generation, causal language modeling
- Encoder-decoder: tranlation, summarization, generative QA

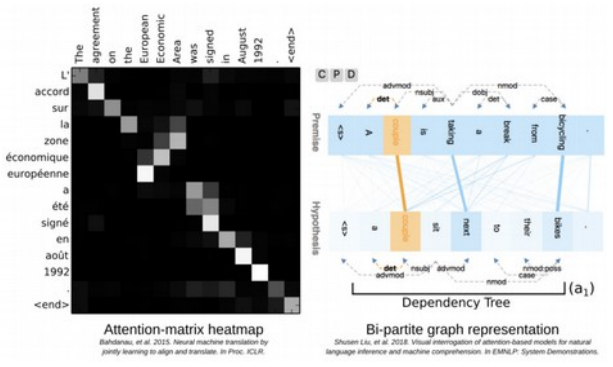
All transformers use the attention mechanism.
Attention describes the ability of a model to pay attention to the important parts of a sentence (or image, or any other sequential input). It does this by assigning weights to input features based on their importance and their position in the sequence.



We break transformers into a communication and a computation phase. Attention is the communication phase. Attention is a search-retrieval problem. Given a query, we want to find the keys that are most similar, and return the corresponging values.

- Query: the think that I am looking forgetting
- Key(s): the things I have
- Value(s): the things that I will communicate.

Self-attention: every node produces a key, a query and a value from that individual node.



Visualizing attention: previous attempts used, for example: attention-matrix heatmap(s)

1.4.8.3 A Gentle Introduction to Positional Encoding

Machine Learning Mastery site Each position/index is mapped to a vector. The output of the positional encoding layer is a matrix, where each row of the matrix represents an encoded object of the sequence summed with its positional information.

1.4.8.4 What are attention masks?

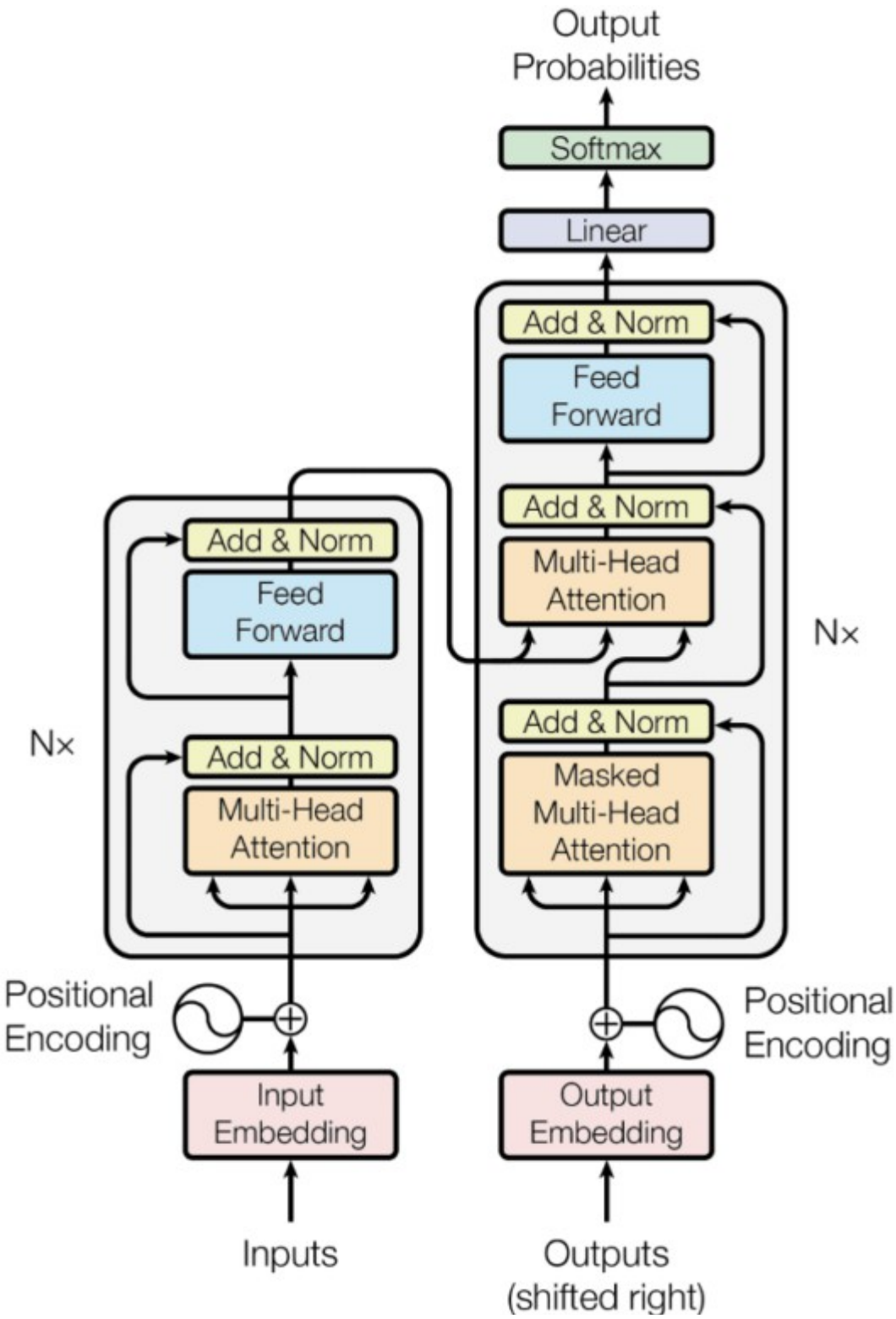
Luke Salamone Post Attention masks allow us to send a batch into the transformer even when the examples in the batch have varying lengths. We do this by padding all sequences to the same length, then using the "attention_mask" tensor to identify which tokens are padding. If you want to perform inference with transformers one sequence at a time, you can ignore attention masks. The "slow way" will be sufficient for your needs. Inference using one sequence at a time, but it is slow.
In order to be faster:

- use batching
- sequences that are shorter than the input size are padded with pad tokens
- Padding from the left or right
- The attention mask shows the transformer which tokens are padding, placing zeros in the positions of padding tokens and ones in the position of actual tokens.

The attention mask can be passed to the forward method of the trasnformer.

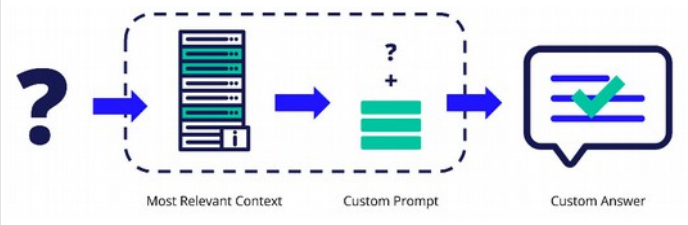
1.4.8.5 Do we need attention?

https://www.youtube.com/watch?v=dKJEpOtVgXc Slides
Input sequence in, output sequence out. Some components are repeated: attention and feed forward; they are repeated
Attention= fully connected interactions. Each component of the sequence must look over each othe possible position.
The Challenge is: do we need attention? Can we use a static routing in the system? A regular pattern instead.
Problems with transformers: training speed: cost is quadratic in length, generation speed: attention requires full loopback.
Uses RNN instead of attention: efficient linear RNN(s), effective long-range parameterization. The same form as RNN, without activation function


Output Probabilities
Softmax
Linear
Add & Norm
Feed Forward
Add & Norm
Multi-Head Attention
Add & Norm
Masked Multi-Head Attention
Nx
Add & Norm
Feed Forward
Add & Norm
Multi-Head Attention
Nx
Positional Encoding
Positional Encoding
Input Embedding
Output Embedding
Inputs
Outputs (shifted right)

1.5 Course Retrieval Augmented Generation (RAG)

1.5.1 Lesson Overview



We will create an unsupervised ML workflow, to match the input question to the relevant content in our dataset. The matching content is used to create a custom prompt, that is then sent to OpenAI, to answer our question.

1.5.2 Review: OpenAI and Prompt Engineering

The ways users can interact with OpenAI are:

- providing a text prompt: the model can produce either an image (DALL-E) or text (Chat-GPT)
- the API code can interact with the OpenAI model

You have to translate your task into a text prompt, so that the prompt is structured in a way that the model will understand:

- **Manual prompt engineering**: the user enters, and iteratively refines the prompt. Many examples of prompt engineering are primarily manual.
- **Automated Prompt Engineering**: code can augment the user original prompt to steer the model to the desired outcome. Useful whan you want the model behavior to be customized. The model by itself is general purpose, to integrate the model into a specific application, you can use automated prompt engineering to taylor the result.

1.5.3 Retrieval Augmented Generation (RAG) Steps

1. Prepare a dataset with embeddings: the dataset is initially composed of text data, before any model can make sense of any kind of data, such text must be converted into numbers, vectors of floating point numbers: for our unsupervised machine learning model, we are going to use text data that has been transformed into vectors of floating point numbers. These vectors are called embeddings and we will use an OpenAI embedding model to generate the embeddings.
2. Find relevant data with unsupervised ML: the user asks a question: we will perform a text search on the dataset, to find the most relevant data using a measure called cosine similarity. This is more sophisticated than a basic text matching search because it uses the underlying embedding vector representation of the text, not just the exact wording used in the question.
3. The selected subpart of the dataset, i.e. The relevant text, is used to compose a custom prompt that incorporates the user question and the most relevant part from our dataset. We must make sure that the number of tokens in the request do not exceed what is allowed by the completion model. We will use the results of the previous step to compose a custom text prompt that incorporates both the user's question and the most relevant context from our dataset. We will use a tokenizer to measure the length of our prompt and ensure that it doesn't exceed the limits of the OpenAI Completion model.
4. Send the prompt to OpenAI and parse the response. This will provide the user with a tailored answer to their question!

1.5.4 Getting started

Required packages: (requirements.txt)

1.5.5 Gathering Data

Custom dataset: gather dataset, anything

- new data that the model was not trained on, including events that had not happened at the time of training, private information, or information for a spefic context (specific industry or audience)
- to address a specific industry or audience you have to provide a specific dataset.
- We will focus on :
  - web sources: it is accessble via the internet, it has frequent updates, for example the wikipedia API,
  - web scraping: extracting data from th source of a web page (the HTML), usually combined with web crawling, that is following the links of related web pages. Web scraping is more challenging that accessing via API, because the content is designed to be viewed via a web browser.

In Python, web data sources can be accessed using the "`requests`" library.

1.5.6 Numeric representations for text data

To create the chatbot, we have to convert natural language data into numbers. Different ways to do that:

- Data storage representation: example, every letter is given its ASCII or UNICODE code. Since we are working on a computer, the text data already has a numerical representation. Example A=65, B=66, C=67 etc. Not a useful way to represent data, because it implies that A < B < C.
- One hot encoding: treat each word as a category, and create a one hot encoding representation for words. The size of the encoding is the same as the number of words in the vocabulary, which is huge. It creates a different set of columns for each dataset., so there is no easy way to compare them
- Vector space embeddings: described next.

1.5.7 Text Embeddings

We need to

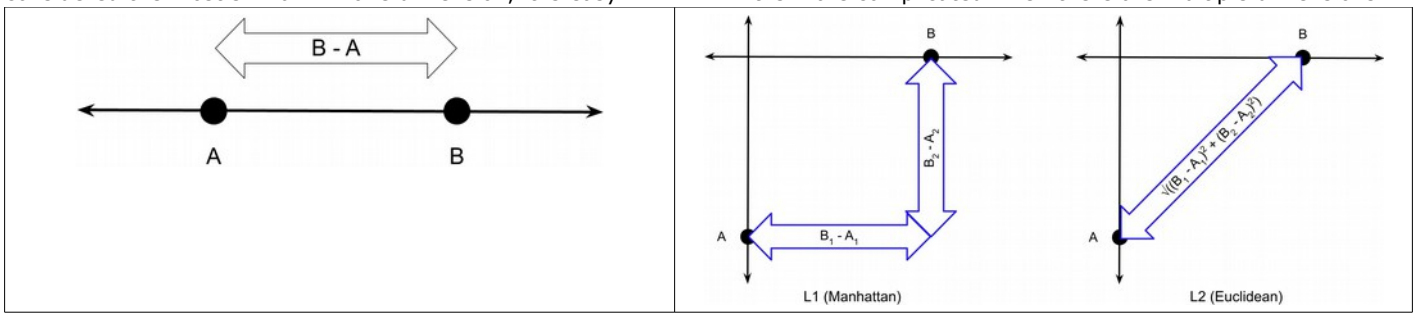
- control the size of our word representation, by limiting to a fixed size vector
- we want an embeddings in some vector space, that must have some desired property
- if two words have similar meaning, their distance should be small, compared to words that are not related.

Some relations should exist, words that are related in some way must be equally separated in the embedding space, for example, it should be true that: $(Man-Woman) \approx (King-Queen)$, so $(Man-Woman)+Queen \approx King$ so this representation becomes useful in finding synonims, analogies, etc. Why do these techniquest work so well? Does it seem magical? Embeddings rely on the distributional hypothesis.

Distributional hypothesys: words that occur in the same context, tend to have similar meanings.
For example: "Would you like a cup of _____ ?" "I like my ___ black". "I need my morning ___ before I can do anything".
The word in the blank could be for example tea or coffee, so tea and coffee are actually similar. So, when a large collection of text is used to learn an embedding, words that are similar result in vectors that are pulled together.
There are also contexts where tea and coffee are dissimilar. "Coffee grounds are great for composting". "I prefer loose leaf tea".
How do we capture those similarities and differencies with the same embedding? By adding an additional dimension. Tea and coffee can be close on one dimension, but distant on another dimension. This additional dimension can capture the variability among beverages. Most ML model architectures use embeddings. Example GPT uses embeddings for both input and output.
Each dimension in the embedding space becomes a number in the embedding vector. Current embeddings use thousands of dimensions. So, each word is stored as a single column called embedding, the cell contains a list of numbers.
Because we are using a pretrained embedding model, the resulting embeddings have the same number of dimensions, regardless of the dataset. So we can compare, combine and generate text data without need to retrain the model for each new task.
Embeddings capture semantic relationships wich enable more efficient and meaningful processing of textual information.
Embeddings are a sophisticated technique for converting text into vectors of numbers using a pre-trained machine learning model. Instead of using each number in the vector indicating the presence or absence of a word, each number in the vector represents a dimension identified by the model. All text samples result in the same number of columns and we can therefore compare any two text datasets based on their embeddings!

1.5.8 Semantic Text Search and Cosine Similarity

Now that we have embeddings, we must find relevant data for our queries. To do that, we will perform a semantic text search. What is it and how does it different from keyword search? In keyword search we look for keywords and return results that match those keywords. For example we look for: "*Leaky faucet repair*", we perform keyword search and we return results that include those exact keywords.
With semantic text search, we look for text that is most similar to our query across a number of dimensions. We have our text with embeddings. We must define a measure of distance, so, whichever piece of text that has the shortest distance, it is considered the most similar. In one dimension, it is easy: $B-A$. It is more complicated when there are multiple dimensions.



What does happen with thousands of dimensions? How do we compare vectors in thousands of dimensions.
The OpenAI recommendation is to use **cosine similarity**. Note that OpenAI embeddings are normalized to length 1, which means that:

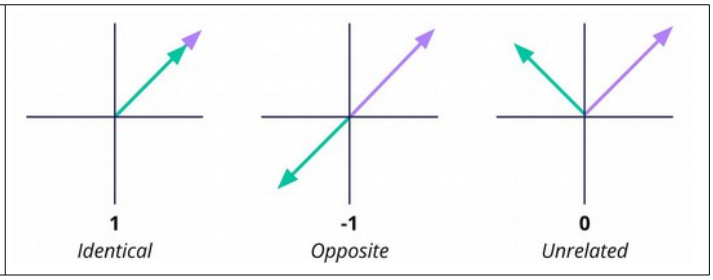
- cosine similarity can be computed with just the dot product
- cosine similarity and Euclidean distance with result in identival rankings

Cosine similarity: $\frac{\sum_{i=1}^{n} A_i B_i}{\sqrt{\sum_{i=1}^{n} A_i^2} * \sqrt{\sum_{i=1}^{n} B_i^2}} = \frac{A \cdot B}{\|A\|\|B\|}$: A and B are vectors of dimension n . Their elements are A_i and B_i .

$$\|A\| = L^2 norm = \sqrt{(\sum_{i=1}^{n} a_i^2)}$$

Cosine similarity ranges from -1 to +1:

- +1 indicates that the two vectors are identical in direction
- -1 indicates that their direction is opposite
- 0 they are orthogonal to each other, it means that they are unrelated..



If two word are similar, their vectors will be close to each other, their angle will be small, resulting in a cosine similarity close to 1.
We talked about cosine similarity, but we want to find a text that has the shortest distance, cosine similarity is not a metric.

Properties of a metric (Wikipedia on Distance, Wikipedia on Metric space)

- The distance between an object and itself is always zero.
- The distance between distinct objects is always positive.
- Distance is symmetric: the distance from x to y is always the same as the distance from y to x.
- Distance satisfies the triangle inequality: if x, y, and z are three objects, then: $d(x,z) \leq d(x,y) + d(y,z)$

To create a metric, that is a distance, the cosine distance is defined as: **Cosine distance = 1- Cosine Similarity**

	Identical	Opposite	Unrelated
Cosine Similarity	1	-1	0
Cosine Distance	0	2	1

Return to semantic text search:

- we start with our embeddings,
- we calculate the semantic distance between our question and each row in the dataset
- the smallest semantic distances will become the top results in our semantic text search

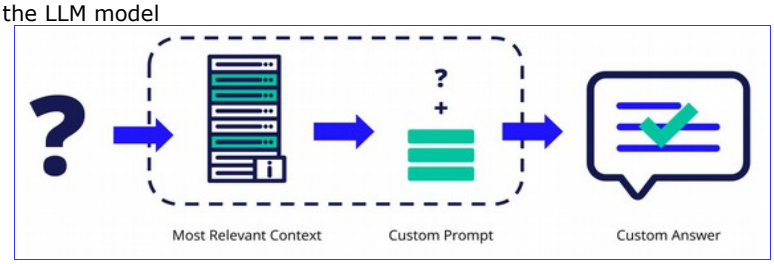
Semantic Search Sorting by cosine distance works for any kind of data that can be vectorized and works especially well for multimedia data (text, images, videos) that produce vectors with many dimensions.

1.5.9 RAG Notes

How much data should we include?
Our data is sorted from most to least relevant -- but how many of those rows can we include?
While we could choose arbitrary number, e.g. the top 5 or top 50 most relevant rows, a better approach is to count the number of tokens we use as we compose our text prompt and use all of the available tokens for each prompt.
Review: A token is the basic unit of text processing in a NLP model. It represents a sequence of characters that the model uses to understand and generate language.
A token is the base unit of text processing, in an NLP model. Example "This is nine words but it isn't nine tokens"
Each token is mapped to a unique integer ID.
Open AI has an online tokenizer interface: https://platform.openai.com/tokenizer
Counting Tokens Our Goal: Maximizing Context
What is the maximum amount of context we can include? The model limit minus the number of tokens in the prompt.
If the limit is 4097 tokens and the prompt contains 24 tokens, the maximum tokens for the context is `4097 - 24 = 4073`.

1.5.10 Resource Augmented Generation (RAG) Workflow

1. Gather data, for example from Wikipedia
2. Wrangle and clean the data, to create a list of text paragraphs, each containing a date and an event
3. For each chunk of data, create the corresponding Data Embedding Vector
4. Given the Question, create the corresponding Question Embedding Vector
5. Establish a metric for Embeddings, for example cosine similarity
6. Sort the data, by their distance from the Question Embedding Vector
7. Establish the maximum number of tokens to use in the prompt : max_used_tokens
8. Compose the prompt, using:
    1. The template:
    2. The question
    3. Add as many context data from the data chunks, without exceeding the maximum number of tokens
    4. Format the prompt, separating the data chunks, with e.g. ### lines.
    5. Calculate the maximum_answer_tokens= model_max_tokens - max_used_tokens
9. Send the prompt to the LLM model



1.6 Course: Build Custom Datasets for LLM(s)

Suppose you have a task, and you want to tran a LLM to do that. Example generating book summary, etc. To fine tune a LLM, you need a dataset to do it.

- Zero shot question answering. Many LLM are good at zero shot question answering: the model was not explicitly trained to anwser a given question; the model has zero "shots" to learn the relevant information. Example: many LLMs are good at arithmetics, even if they were not trained to do that. Called emergent properties. These zero shot capabilities are needed in Open Domains: Wikipedia, the Pile, github. This is sufficient for many users and tasks. Narrow domains: specialized fields like law, cybersecurity, medicine. The model must have deeper context on the nuances of the field, so the model must be trained on specialized data.

Lesson outline:

- How to collect data for a task, in particular from the internet
- How to evaluate and clean the data. Garbage in, garbage out
- Modeling tasks, and how they influence the dataset
- Build the dataset

1.6.1 Collecting data

Ideally, we have our internal data sources, with clear, well formatted input and output, that allow us to craft the dataset easily. In practice, these data sources are extramely rare, most of the time that data is not well formatted and may be missing context or information, and we need to supplement such data with externally collected data, or simply using external data from the get-go. Once we collected that data, we want produce a nicely-formatted dataset. How to do it:

- **identify potential data sources**, to do this, you should know what the model must accomplish. What is the task that you are looking to solve with your model? This is important, becouse sometimes, Machine Learning might not be the right tool, expecially in language modeling. Sometimes there is a need for a given precision that it is not possible to accomplish with a machine learning model; in that cases, step back and maybe use another tool/technique to solve the problem. Once we have deciced that our problem is best suited with language modeling, we want to know what the typical input looks like. Is it structured data, plain text, maybe from a webpage? What the typical output looks like? Summaries, answers to questions, text classification, etc. Moreover, what data resources people use to do their job: internal resources, can it be collected easily? More often we will be looking to external data sources that people access for use in their day to day. This lesson is about collecting external data sources, from the internet.

How to collect data? How to collect data from the internet?

- Via **Application Programming Interface** (API), intended for programmatic access to the hosted data, typically the user obtains some key, makes a request, and retrieves structured data, for example JSON or XML documents. API(s) are a great way to collect data, and they are frequently integrated into larger systems, and have the benefic of being supported for data collection.
- **Scraping**: making a series of HTTP requests to pull back the HTML documents hosted on the webpage. It is a costly operation for the site that is the target of the scraping (the web site that is scraped). Due to this cost, it is important to remember that scraping is not always welcome. You may be blocked for scraping sites that do not permit it. There are ways around this. There are legal an ethical considerations that we must consider.
  - some data may require licensing, especially when used for commercial purposes, consult a legal consult if not sure. Code licensing can be particularlt complicated.
  - Scraping is legal with rare exceptions, but not always welcome: scrape as little as you need.
  - Not all data on the internet is safe to use: we call it data toxicity.

As long as we are operating within a legal anethical framework, the internet can still be a great place to collect data for our task.
In the exercizes, he uses Beautiful Soap, and he also describes Scrapy and Selenium.

1.6.2 Evaluating Data Quality

Most ML engineers deal with tabular data, tables and spreadsheets. Data quality concerns with missing values, inconsistencies, missing fields, and how to deal with categorical values, etc.
However, LLM are different, because they primarily deal with non numerical inputs and outputs.
We are looking for two major properties in general:

- whether the text is syntactically and semantically normal for the task: syntax of a sentence: the structure that make it a grammatically correct sentence
- the text is relevant to the task

Example: "Erik Galikin is a hacker and artificial researcher whose passion is in finding ways to both apply AI to secirity and apply security to AI." Incorrect.
"Erick Galikin's passion is in finding ways to both apply AI to security and apply security to AI,and is a hacker and artificial intelligence researcher." Weird syntax.
"Erik Galinkin was a hacker and artificial researcher whose passion was in finding ways to both apply AI to secirity and apply security to AI.". By changing is → was, we now mean Erick is no longer any of these things. The change in the meaning is the change in the semantics of the sentence. In specialized fields, like law or medicine, the syntax and semantics of normal text can be different from the one we get from books, forum posts, web pages on the internet. So, it is important to know what looks like for your task. The relevance of individual pieces of text is time consuming to evaluate, requiring experts to review each bit of text. It is not scalable to hundreds of pages, so it is important that you trust the source of your data.. For example, traning an AI assistant for a medical doctor. This assistant would require substantial amounts of training data about health conditions. Forum posts giving medical advice are abundant, but we cannot use them, because we cannot individually evaluate those posts; we could get non related posts, bad anwsers, and other forms of internet toxicity, etc.
We must partner with healthcare networks and try to use messages between patients and their foctors, subjecto to HIPAA in the US, or use trustworthy public data sources like PubMed to find scientific medical papers.
Despite the best efforts of researchers and developers, we find that word embeddings consistently contain meaningful gender, racial, and other social biases.
Researchers have found that word embeddings, in general, contain male-centric biases(Link) -- something that has even been found in ChatGPT(opens in a new tab).
In some cases, we can accept the risk of these biases not impacting our use case. Even then, especially if we are using data from

the internet, we must examine our dataset for the presence of hateful language.
In more sensitive cases, we may want to address the bias within our model by modifying our dataset to use more neutral language with respect to some categories.
We note, however, that there is a cost to fairness -- debiased models have been found to be marginally less accurate in general. If fairness in machine learning interests you, the Fair ML book(opens in a new tab) by Solon Barocas, Moritz Hardt, Arvind Narayanan is a great resource.
See the Fairness book (https://fairmlbook.org/)

1.6.3 Review: Language modeling tasks

LLM(s) usually use a transformer architecture. Use the attention mechanism to perform natural language tasks. BERT, GPT, T5.
Faundation models: they are a foundation for models that are fine tuned on downstream tasks. Tasks:

- text generation: a question is given, it generates an output. This falls into either:
  - masked language modeling
  - causal language modeling: the sequence is started, and the model must continue the phrase. The most probable token that follows. It is a subtask for other tasks:
    - Abstractive summarization
    - Question answering: extractive or abstractive
- Classification: apply a label to input text
- Translation: for most time it was the motivating task
- Clustering: lookup similar sentences

There are so many models and tasks: how do we choose the one to use?

- Autoregressive models: GPT. Decoder only: tasks:
  - instruction based text generation
  - summarization
  - abstractive question anwsering
- Autoencoding: BERT: classification also extractive question answering when precision matters, also clustering
- Sequence to sequence, like T5 (encoder+decoder), used in translation

Any model can perform any other task, especially when the model is extremely large.
When we use smaller models, we should pay attention, and play to the strengths of each architecture.
There are a variety of language modeling tasks, including statistical tasks like tf-idf(opens in a new tab) that do not apply directly to transformer-based large language models. In pre-trained models, the pre-training method has a significant impact on its performance against specific tasks. We highlight three particular model types:

- Autoregressive (decoder-only)
- Autoencoding (encoder-only)
- Sequence-to-sequence (encoder-decoder)
-

Typically, these models are optimized against particular pre-training tasks. Kalyan et al.(opens in a new tab) survey these pre-training tasks in depth, but we primarily concern ourselves with the two most popular pre-training tasks:

- Causal language modeling - predicting the next token in the sequence
- Masked language modeling - filling in the [MASK] token

With these types of pre-training tasks and models, we can perform several popular language modeling tasks:

- Text generation
- Question answering (both abstractive and extractive)
- Summarization
- Classification

1.6.4 Structuring and Storing Raw data

The `Dataset` Python library allows to store and retrieve structured data as it was a Json, but it is built on top of SQL (https://dataset.readthedocs.io/)
The lesson, on the other hand, talks about the package called `datasets` (https://pypi.org/project/datasets/) It is possible to share a dataset to the Hub=huggingface (link). Given a dataset, it is possible to stream it: serving it to the training model, without waiting for the dataset download. Moreover, a dataset can occupy all the memory of the computer, so with streaming, only a small part is in the computer memory at the same time (it is called dataset streaming).
Since our data is all text, we have the luxury and the burden of storing raw text data. While we can put this data directly into a Dataset object, there are a few drawbacks to doing so:

Choosing an object from one library (for example the datasets package) may make it difficult to migrate to a different library in the future. The same goes for Pandas tables.
If there is preprocessing or chunking in our Dataset object, we may not be able to reconstruct the full context - this matters because we may want to train a different model with a different context length in the future!
If there is toxicity in our data, it is harder to go context by context in the Dataset object than to delete or edit the source file and rebuild. First, let's talk about how to save a Dataset. Then, we can discuss the raw data.

1.6.4.1 Structuring Raw Data

- Option 1: each content is stored in a separate .txt file. If applicable, store questions/answers pairs, or instruction pairs into separate .csv files.
- Option 2: store everything in one .csv file, or one for each train/validation/test set.

Option 1 : Storing contexts separately in their own .txt files offers several advantages at the cost of additional complexity.

First, if we wish to stream(opens in a new tab) our data, we'll need our data in a format nice enough to do so - putting it all in a single CSV isn't going to do that for us!
Next, from a version control perspective, it's much easier to generate a list of files that have been added, deleted, and modified

than to compare the differences between two files.
Finally, if we are storing our data in the cloud but training offline, we can reduce our egress charges by transferring only missing or updated files rather than retransmitting an entire large CSV. Of course, there are drawbacks!
Chief among the drawbacks is that if we are storing many, many files, we need to have a naming convention that guarantees each file receives a unique name. Otherwise, we risk overwriting our hard-earned data.
Secondly, the directory structure matters - we may want separate directories for train/validation/test data, but this means we have to manage files and track versions across the directories.
Lastly, since there may be many, many files in our dataset, we can run into filesystem issues like exhausting the available inodes(opens in a new tab) on older filesystems.
Option 2: The advantage of the CSV option is, of course, that it is easy to keep track of! Having one file greatly reduces the complexity of managing and moving datasets.
Furthermore, using a single train.csv file to populate a Pandas DataFrame or HuggingFace Dataset is very convenient.
On the other hand, for large datasets, these files can become extremely large and, therefore, challenging to move and share.
Additionally, since all the contexts are stored in a single (large!) file, versioning can be challenging, as we'll need to diff two very large files.
Ultimately, the smaller your dataset is, the more viable the CSV option is. As our dataset becomes larger, a filesystem-based approach becomes necessary, otherwise, store all into one CSV file. Which option you choose should ultimately depend on your comfort managing the raw data files and whether or not you expect your dataset to continue growing.

1.6.5 Datasets for Question Answering

(A question – anwser sample point has: context, question => answer)
Much like with causal language modeling datasets, we'll need to collect data - either from the internet or from some internal process. These contexts will form the basis of our question-answering dataset, so we'll need to ensure they are trustworthy, relevant, and high quality. Context collection is also potentially a double-edged sword here, as the more contexts we collect, the more questions and answers we will want to create. The effort here is around dataset curation and cleaning, identifying quality data sources, and possibly writing a scraper to collect and store the data.
Questions and Answers Given a corpus of contexts, we can begin writing questions and answers. In some rare cases, we may have an existing set of questions and answers, so we can attempt to map questions and answers to contexts by checking for the presence of the answer in a given document.

However, we often have to have humans read documents, generate questions, and identify answers in context.

This can be a very labor-intensive process, and while many labs have leveraged crowdsourcing websites that offer small amounts of compensation to workers who complete tasks, the ethics of such platforms are widely debated(opens in a new tab). Additionally, for tasks that require domain expertise, non-experts may provide poor evaluations that can impact model quality.

1.6.6 Lesson Review

In this lesson, we learned how to construct a relevant, quality dataset for fine-tuning large language models. When building these datasets, it is important to:

1. Identify quality data sources
2. Attempt to collect data from those sources in a way that is minimally disruptive
3. Evaluate the quality of the collected data for the task and clean it where appropriate
4. Store the raw data in a task-appropriate way

We discussed different tasks for language models and appropriate architectures for those tasks:

- Encoder-only models for extractive QA and document similarity
- Decoder-only models for open-ended text generation and summarization
- Encoder-Decoder models for translating text

Finally, we covered how to take the raw data we've collected and, given the task and model at hand, construct a dataset and fine-tune a model using Huggingface transformers and datasets. Now you have the knowledge to fine-tune models for your own tasks, whatever they may be.

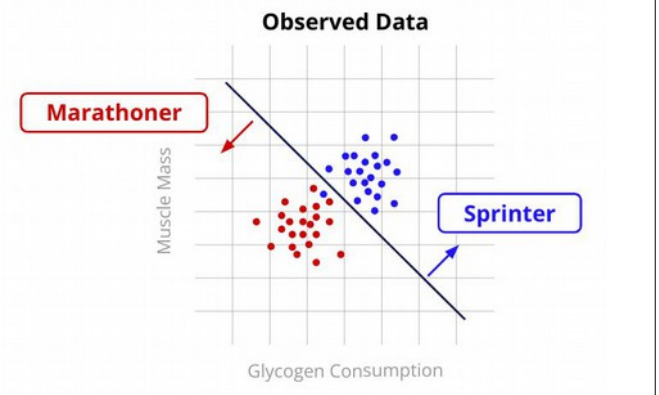
1.7 Image Generation. Course: Introduction to Image Generation.

Generative AI in Computer Vision. Example Changing product background. It can change the picture background, or remove specific objects from the image, still filling the empty space seamlessly. Suppose you wan to creae the visuals for a marketing campaing. With Ge-AI you can generate hundreds of experiments in a few hours. The images, are cheap, in many cases , free, unique and yours. In all cases, the computer generates new content.

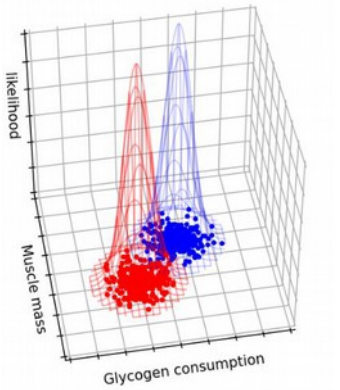
Generative AI is a ML algorithm that can generate new content. Often to generate: images, text, video and audio. Usually based on DL, however, At its core, Generative AI refers to machine learning algorithms capable of generating new content. This includes images, text, video, and audio, primarily based on deep learning techniques. However, it's crucial to note that not all generative algorithms rely on deep learning, and not all deep learning algorithms are generative, so we will look for a better definition.

1.7.1 Discriminative vs Generative models

Suppose we have data about runners. Dataset, with tabular data about runners. Each sample has two features: muscle mass and glycogen consumption of every athlete. Marathon athletes have low mass and low glycogen consumption, whereas Sprinters are the opposite. We can train a discriminative model to classify athletes as marathon runners or sprinters. After the training phase, the model is able to classify athletes. Generative models work in a different way:



Discriminative model



Generative model

Discriminative model: the model learns the boundary between the two classes. After training, the model is able to discriminate between the classes.

Generative model: we plot in 3D, we also add the likelyhood for athlete of that class to have those characteristics. We get a gaussian distribution for marathon runners and sprinters. The two gaussians compose the generative process for our dataset. A generative model uses the training data to learn the shape of both class distributions, and also their sum, the gen-AI model does not learn a decision boundary, but the generative process of our data. So, the Gen-AI model can be used to classify samples and also to generate new data points that represent athletes of the two classes (marathon runner/sprinter). The model just extract a new sample from the distribution it has learned. Generated data points represent realistic, but fake sprinters or marathon runners. Math: X The data; Y : the class.

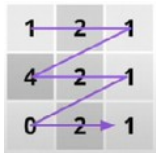
Discriminative model: $m_\theta \rightarrow p(y \mid x)$ if we are solving a classification proble, we can directly ue the output of the model (conditional probability of y given x .

Generative model: $m_\theta \rightarrow p(x, y)$ $\quad p(x, y) = p(y \mid x) * p(x)$ $\quad p(y \mid x) = \frac{p(x, y)}{p(x)} \propto \frac{m_\theta(x)}{p(x)}$

The definition is $p(B \mid A) = \frac{p(A, B)}{p(A)}$

The model given the probability that the pair (tuple, class) is acceptable. Learning the joint probability is harder than learning the conditional probability, so often discriminative models perform better at classification than generative models. When it comes to generating new content, discriminative models fails.

1.7.2 Image Generation

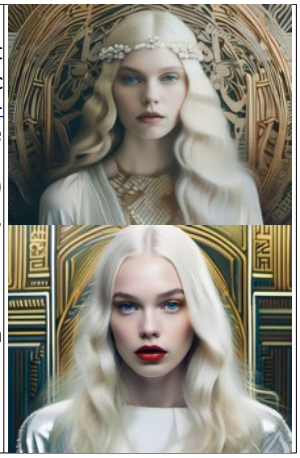
	An image is an ordered collection of pixels, it is a point in a very high dimensional space. For example, a 3x3 image, one channel. The image can be flattened into a vector in R^9 . If we have an image dataset, we can transform the dataset into a table with n rows and 9 columns and train a generative model on this table. As mentioned, an image is just a point in a very-high dimensional space, we can learn a generative model over that space. The problem is that the space of realistic images is very sparse. Suppose that the image is 10x10, with 255 gray levels, What is the probability of generating a particular image, e.g. a smiling face? $255^{-100} = 1.5^{-241}$ Realistic images represent tiny islands in a vast universe of noise.
	Example, the MNIST dataset of hand-written digits. Let us flatten the 28x28 images into vectors and apply a dimensionality reduction technique. There is a lot of empty space between samples. It means that the generative model has a lot to learn to generate images that look like digits (realistic digits). The generative model would neeed at least 10 peaks. It must learn all modes of a multimodal distribution. The previous example had 2 peaks, this generative model must lear at least 10 peaks.

Generative models in computer vision don't just learn to recognize patterns or classify images; they learn the underlying structure and distribution of realistic images within this high-dimensional space. This understanding is crucial for these models to generate new, plausible images. The model's task is akin to finding and focusing on the 'peaks' in a multi-modal distribution, where each peak represents a different aspect of the data, like different digits in the MNIST example. In conclusion, the role of Generative AI in computer vision is not just about creating or altering images. It's about understanding and navigating a complex, high-dimensional landscape where realistic images are rare.

1.7.3 Computer Vision Gen-AI models

Different types of Gen-AI models in CV:

- **Unconditional generative model.** $p(x)$: they generate realistic images without any input from the user. They are trained from an unlabelled dataset. They just generate realistic images. The user has no control over the generation Example (https://this-person-does-not-exist.com/) The model just generates an image similar to the dataset it was trained on. The generation is unconditioned.
- **Conditional generative models** They generate data according to an input ("prompt") $p(x|prompt)$ The prompt can be a textual description, or other types of prompt. Example stable diffusion (Huggingface model).
- **Image to text**, called BLIP. It is the opposite process of what is done by stable diffusion.
- **Text to video model**, for example. Video LDM
- **Multi modal** $p(x|prompts)$ note that it is "*x given prompts*" They can receive inputs in multiple modalities (text, images, audio, …) and answer in multiple modalities. Example: Mini GPT4 and LLAVA (GPT-4V) (LLAVA)



Chain tools together. Generative AI systems can be chained together to produce more powerful tools. For example:

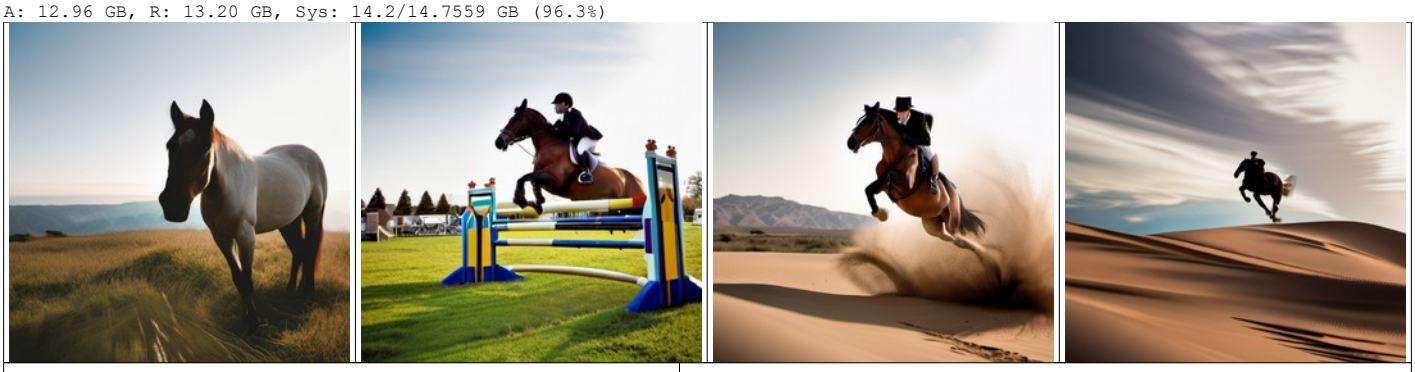
- Create an image using Stable Diffusion, from text
- Pass the image to another model called Dream Gaussian , to generate a 3D asset baased on the 2D image.

This is just a 2 steps pipeline, it goes from text to the 3D asset. More complicated pipelines can be created, the possibilities are endless. Pipelines can also include non generative steps like reinforcement learning, etc.

1.7.4 Generate an image with stable diffusion

Uses the Text2Image model **Stable Diffusion**: GitHub repository: https://github.com/AUTOMATIC1111/stable-diffusion-webui
Set the checkpoint to sd_xl_base_1.0.safetensors
Set the refiner to sd_xl_refiner_1.0.safetensors
Image generation has randomness to it. The random seed is kept at a constant value : we used: 1877613533 . Result:

```
a photo of a horse
Steps: 20, Sampler: DPM++ 2M Karras, CFG scale: 7, Seed: 1877613533, Size: 512x512, Model hash: 31e35c80fc, Model: sd_xl_base_1.0, Refiner: sd_xl_refiner_1.0, Refiner switch at: 0.8, Version: v1.6.0
Time taken: 2 min. 19.6 sec.
A: 12.96 GB, R: 13.20 GB, Sys: 14.2/14.7559 GB (96.3%)
```



Use a fixed Seed to make sure results can be reproduced.

Prompt	Notes
A photo of a horse	Plain prompt to Stable Diffusion XL
A photo of a horse jumping	For example, the horse is displayed in a competition, still low quality and lacking details.
A photo of a horse jumping in the desert	
A photo of a horse jumping in the desert dramatic sky	(He does not use punctuation)
A photo of a horse jumping in the desert dramatic sky by Van Gogh	Styles, again, no punctuation
A photo of a horse jumping in the desert dramatic sky artstation / intricate details / National Geographic	Including the name of a related magazine or web site helps. Add the names of known websites to influence the style, for example, artstation, or add the keyword "intricate details".
Positive prompt: a color photo of a next generation jet fighter while flying high in the sky, combat aircraft magazine, details perfect light bokeh **Negative prompt**: overexposed underexposed	Add negative prompts.

1.7.5 Parameters

The model itself has some parameters that we can tweak, that can meaningfully change the output.
Before we can understand the parameters of Stable Diffusion, we need to add some very high-level detail on how Stable Diffusion XL works. Stable Diffusion is a Diffusion model, which starts from pure noise and iteratively "pulls" the image out of the noise.

There are 2 parameters that are very important:

- **Number of iterations**: too few iterations will generate dull images or images with artifacts, and too many iterations will start corrupting the image. The default value of 20 is usually a good value. You might experiment with increasing it up to 50, but that is not guaranteed to improve results.
- **Classifier-free Guidance scale (CFG Scale)**: this parameter controls how closely SD should follow your prompt. Increasing this leaves less room for SD to make its own decisions. Decreasing it gives more "artistic freedom" to SD.

Version XL 1.0 adds one more complication: there are now two SD models, one "base" model that does the heavy lifting and one "refiner" model that refines the details of the result. This is the Refiner setting we changed at the beginning. This adds the Switch parameter. **Switch**: the fraction of iteractions after which the refiner model takes over from the base model in the diffusion process. https://stable-diffusion-art.com/automatic1111/

The inpaint technique is also described. Select the area that you want to change. The teacher describes the parameters:

- Mask mode: inpaint masked
- Inpant area: only masked
- Only masked, padding pixels: 20
- Sampling method: Euler a
- Sampling steps: e.g. 20
- Refiner switch at: 0.5 = 50%
- Denoising strength: 0.5. Important parameter.
- Batch size: 4. The model produces 4 pictures that we can choose from.

Inpainting is very sensitive to the inpainting mask and the parameters. Links:

- Stable Diffusion examples.
- Stable Diffusion WebUI AUTOMATIC1111: A Beginner's Guide

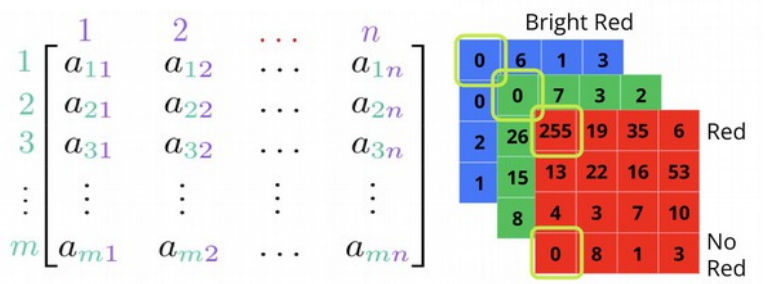
1.8 Course: Computer Vision Fundamentals

Computer Vision allows computers to receive and interpret visual input from the world around them. It mimics human sight. Computers can see and understand visual information. Vision is the primary sense that we use to interact with our environment, recognizing faces and understanding complex scenes.

1.8.1 Image representations

Images as 2D arrays of pixels. Images are represented as 2D arrays or matrices, each element represents a pixel of the image. Each pixel holds information about a tiny portion of an image, its color and intensity. Computers see the image as a collection of individual pixels, while humans see the picture as a whole. In grayscale images, each pixel represents a scale of gray, varying from black to white, the value ranges from 0 to 255, where 0 is black and 255 is white.

Color images are represented as a set of 3 2D arrays, with each 2D array is the set representing the intensity of a primary color channel. This is the RGB model It is like layering three grayscale on top of each other, where each layer captures the intensity of red, green and blue. This RGB model aligns with how we perceive color in the real world. The eyes have receptors for red, green and blue light and the combination of these colors in varying intensities gives us the wide spectrum of color we experience. Each color pixel is represented as three numbers, the intensity of the red, green and blue.



- Images are represented as 2-dimensional arrays, where each cell in the array maps to a single image pixel, which holds information on color and intensity
- Analyzing pixels allows computers to identify patterns, colors, and shapes, helping them to form a broader understanding of the image
- For grayscale pixel values ranging from 0 to 255, 0 represents white and 255 represents black
- Colors are represented in 3 different channels: red, green, and blue

1.8.2 Common Image Processing Techniques

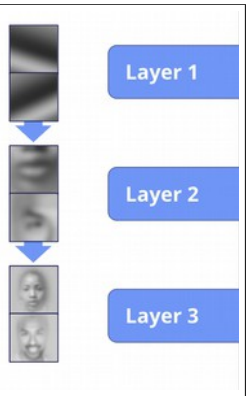
Humans view images holistically, interpreting them as a collection of features. The brain is trained at identifying patterns and shapes and associating them to known objects. Computers use a more methodical and analythical process. The image is broken down into a series of features: edges, corners, colors textures, shapes. To recognize a cat, the computer must be taught to recognize the features that identify a cat. Give the computer many images of cats, each labelled as such. It must identify the common features shared among those images. The computer uses algorithmic and statistical methods. How do computers recognize images? Features are the specific patters or identifyable structures within the image, such as edges, corners, etc. These operations are not applied to the entire image. They are performed on local regions of the image.

Intuitivaly, pixels closer to each other in an image are probably more relevant to each other. So algorithms focus on local pixels of an image. Vieweing all pixels at once may lead to poor understanding of the image. Nearby pixels are more strongly related than distance ones. Math operations leverage this characteristic. They process small areas like patches of an image to detect features like edges, etc.

This process is both layered and hierarchical. The network detects low level features first and then combines them to detect higher level features. The first layer works on low level characteristics, the next layers build upon the previous ones and detect higher features, etc.

- **Layer 1** : Learn to detect edges, textures and color from raw pixels in the first layer.
- **Layer 2** : Use edges to detect simple shapes and patters in intermediate layers: circular patterns , corners, etc.
- **Layer 3** : Combine shapes and patterns to detect abstract higher level features, such as facial shapes, in higher layers.

This hierarchical process is crucial because it mirrors how we humans process the world.



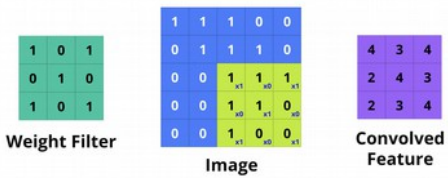
1.8.3 Convolutions

Features extraction using Kernels.

- We take a special matrix (our kernel) and slide it over the image. A kernel is a weight filter. It highlights specific aspects of an image. The result of the convolution is the "Convolved feature"
- We compute a convolution of each batch
- this process essentially compresses the individual pixels of the image into features.

Using different kernels, we can detect different kind of features in the same image. Some kernels may be designed, some other learned.

- The value of the kernel determines what features the kernel detects
- a vertical edge kernel detects vertical edges in an image, for example



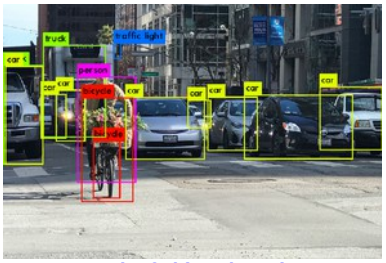


The dot product. The dot product is a similarity score between two vectors.

$$\vec{u}\cdot\vec{v}=\sum_{i=1}^{n} u_i v_i$$

How do kernels extract features? The values of the kernel determine what features the kernel detects. A vertical edge detects vertical edges in an image, for example. High values in the resulting convolved feature matrix indicate a likely presence of a vertical edge in that area. Low values indicate the absence of that feature. In many computer vision tasks, kernel values are explicitly specified. Example, the Canny edge detector is an example of such a kernel.
In modern machine learning kernels are often learned from data.

1.8.4 Computer Vision Tasks



Result of object detection

- **Classification**: It answers the question: "*What is in this picture?*" Assign an image to a specific category. Classification processes the image and assigns it is one of the predetermined labels. Challenging due to the variability of images in the real world. It is the basic step for organizing photo libraries, autonomous driving, etc.
- **Localization**: Now the question is: "*Where is the object in the picture?*" It involves identification of objects as well as their location. It draws a bounding box around the area of interest. The goal is drawing a tight box around the object.
- **Object detection**: builds upon classification and localization, applied to multiple objects in the same image. Usually the result are bounding boxes, but there are also alternatives. In object detection, the system scans the entire image for instances of objects of given categories. Each bounding box is given a label corresponding to the classification of the contained object.

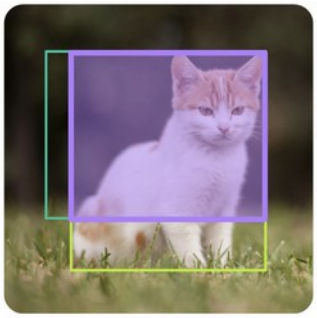
1.8.5 Instance Segmentation

- Mapping the contours and boundaries of an object in an image
- Each pixel is assigned to a particular object or the background: pixel level classification. Each lpixel is labelled.
- This leads to a more precise understanding of the objects in the image
- This is valuable in applications where the shape, size and spatial relationships among objects are crucial. Example: medical imaging.

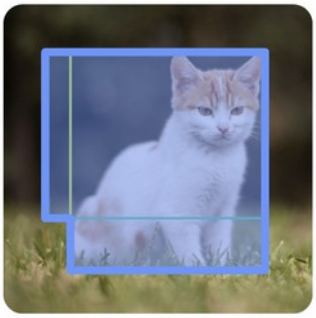
Instance Segementation is about mapping the contours and boundaries of each object in the image.
Each pixel is classified to either the object or the background.
Training a classification model. The training begins by passing the image through a CNN .
At the beginning of the network, there are CNN layers, they extract low level features and the more and more high level features..
At the end of the network, there are fully connected layers (classification stage), the output is a series of nodes, one for each class, provising the probability the the input image contains an object belonging to that class.
Training a simple localization model. Detecting the location of a cat nose in an image. We start by adapting a model that already performs image classification. We add two additional outputs, which give the X and Y coordinates of the nose. To train

this model, we must have a dataset where there is the ground truth of the animal class and the correct X and Y coordinates.
What if now we want to draw a bounding box around the cat? The network must output the X and Y coordinates of the corners of the bounding box. So 4 (X,Y) coordinates instead of one.
A classification model is trained by passing an image through a convolutional neural network (CNN), performing convolutions to extract features, and using these features to output probabilities for each class. During training, the internal parameters are adjusted to improve the accuracy of these predictions. Localization works with a similar process by adding two extra parameters that represent the x and y coordinate of the object. We can draw a bounding box by also adding width and height variables for the box.

1.8.6 Intersection over union (IOU)



Intersection



Union

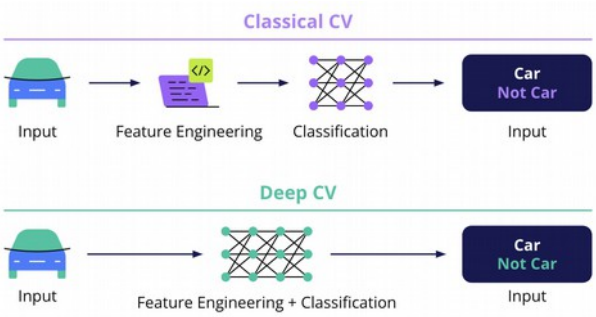
How can we predict if the bonding box predicted by the model is accurate?

- A metric used to quantify the predicted bouding box (BB) accuracy compared to the "Ground Truth" Ground Truth = actual correct answer.
- IOU measures the overlap of the "Ground Truth" and the predicted BB

"How much of the predicted box overlaps with the GT" over "the intersection of BB and GT". IOU ranges in[0,1], where 1 indicates a perfect prediction, while 0 indicates no overlap. Something above 0.5 IOU can be considered a positive sample if you are doing classification. For medical applications this threhold can be moved up, because we need more precision. During training, the model is adjusted to maximize the IOU.

1.8.7 Classical vs Deep CV

Classical CV	Deep CV
Manual feature extraction: algorithms are explicitly programmed to extract some kind of features. Poor adaptability to changes in lighting, scale and orientation Less data reliant, interpretable with rule based decision making Best suited for controlled environments with structured settings, for example industrial automation, where conditions are controlled.	Uses deep learning for automatic feature learning CNN(s) RNN(s) and transformers for hierarchical feature learning Requires significant computing power and large datasets Excels in complex tasks Highly adaptable, but less interpretable: it is difficult to understand why a specific result is generated.



Classical CV relies on manual feature extraction and feature engineering
A limitation of classical CV is its robustness and performance under varying conditions, and an advantage is that it's effective in structured settings and also less reliant on large datasets for training
Deep CV automatically learns the features from data, erasing the need for manually engineered features
Neural network architectures like convolutional neural networks, recurrent neural networks, and transformers are a big part of deep CV
A limitation of deep CV is that it's very computationally intensive, and an advantage is that deep models are much more generalizable and can better handle more complex tasks.
Classcal CV algorithm use less features: so they depend on the quality of the data-

Deep models:

- PRO(s): stonger performance and robustness
- CON(s): black boxes, challenging interpretation

1.8.8 Foundation models

You Only Look Once (YOLO). It is the standard model for object detection. The image is divided into a grid and processed in a single pass. YOLO starts by overlaying a grid over the original image. YOLO tries to figure out whether a known object is is one of the squares. In each square, YOLO tries to figure out the known object contained in it and to draw a BB around the object within the cell. For each box, it also calculates the confidence of the detection and how accurate the the box location and size is.
There is a problem if a cell contains multiple objects. YOLO works around it by outputting multiple classifications and bounding boxes for that particular cell. So, at forst YOLO output, for each cell=square: its class and BB. A cell can contain multiple classes.If in a cell there is the dog and the bike, YOLO will output a classification for bike, with its BB, and a classification for the dog, with its BB.
YOLO paved the way for fully convolutional networks and end-to-end machine learning pipelines in general.
YOLO approach of processing smaller grid cells inspired the patch approach for fully convolutional networks and end-to-end ML pipelines, where data is fed into the model without much preprocessing. YOLO real-time detection capabilities also influenced applications like surveillance sustems and autonomous vehicles, where speed is important. YOLO architeture allows adaptation to new domains, called transfer learning.

1.8.9 Demo

1. Load + process image
2. Load pre-trained model
3. Draw + displays bounding boxes

1.9 Course: Image Generation and GAN(s)

1.9.1 Image Generation and GAN(s)

There are different types of algorithms:

- unconditional algorithms, they start from nothing and generate and image. No specific input or guidance
- text2img: from text prompt to the image; the user describes the desired result with text.
- img2img: user input is through sketches and raw input (img2img, e.g. ControlNet).
- inpainting and Part substitution Another kind of img2img is generative fill, inpainting, etc.

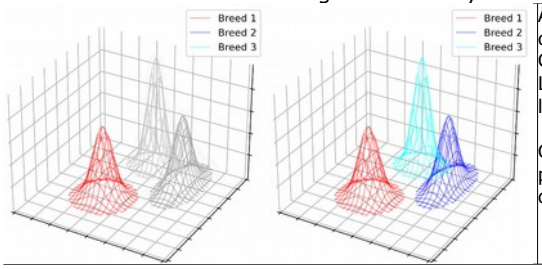
GAN(s) are unconditional generators, in their basic version.

GANs are primarily unconditional image creators, although they can be adapted for conditional inputs. However, another class of Generative AI models, called Diffusion Models, is more commonly used for conditional generation.

How to evaluate image generation algorithms?

- Coverage: the diversity of the generated output. Example generation of dog images: if an algorithm generates the same breed of dog, same pose, environment. etc, it has low coverage wth respect to another that can generate all breeds.

Remember that images are a sparse multi-modal distribution: the generative model samples from a distribution: is it a distribution that covers all dog breeds? Only one breed, etc? The algorithm with little diversity has low coverage.

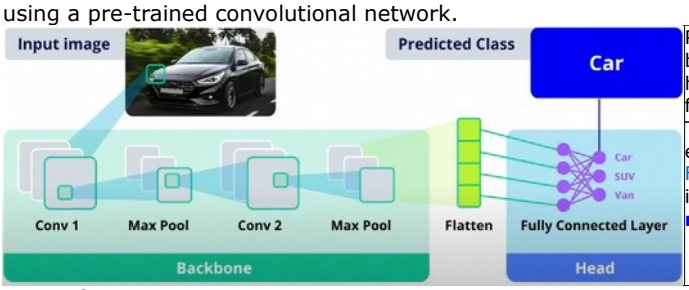


An algorithm that has low coverage for example can only sample from the distribution of dogs Breed 1. Low diversity = low coverage.
Coverage: how many peaks has the model learned, and thus generate images from?
Low coverage algorithm: it can generate from a specific peak: the generated images will look similar in one or more characteristics: breed, pose, environment.

Coverage is a fundamental aspect of image generation algorithms, measuring their ability to produce diverse outputs. Understanding and improving coverage is key to enhancing the quality and variety of images generated by these algorithms.

1.9.2 Evaluating Generative AI in CV – Quality and the Impossible Triangle

Quality is an intuitive concept: better quality means more details. Again, images are samples from a multimodal distribution. Consider one peak of the distribution. Question: how much the distribution of the algorithm resembles the distribution of the real dataset. Good resemblance= high quality. Images are vectors in an high dimensionality space. It is hard to compare the distributions in such a high dimensional space: the idea is to use some sort of dimensionality reduction to go from the image space to a space with fewer dimensions, without destrying information from the images. One way is generating embeddings, using a pre-trained convolutional network.



Pre-trained CNN(s) are used for dimensionality reduction. They have a backbone and a head. The backbone productes a vector of features, the head is a fully connected network that takes the feature vector and uses it for classification. Doing so, it discards irrelevant details.
The result of the Flatten layer contains the summary of the information extracted from the backbone. If we remove the head, we can use the Flatten vector as a low dimensional representation of the content of the image. This representation is called **Embeddings** or **latent representation**.

The **Fréncher inception distance (FID)** is a common method to measure sample quality.

- we take a real-world dataset, e.g. COCO-30K
- we feed it into the backbone of a pre-trained CNN . By convention the FID uses the Pretrained Inception V3 model
- we get the distribution of embeddings from the real-dataset

We do the same for the generated images: so they are fed into the Pre-trained Inception V3 model and the Embeddings Distribution is generated. The two distributions are compared using the Wasserstein distance. The "*W-distance*" is a measure of how close or far the two distributions are. The result of the comparison is the FID: we can thus compare algorithms among each other measuring the FID. **Lower FID is better**. Lower FID => more quality. We have introduced coverage and quality, the other is sampling speed: how much time does it take to generate a sample.

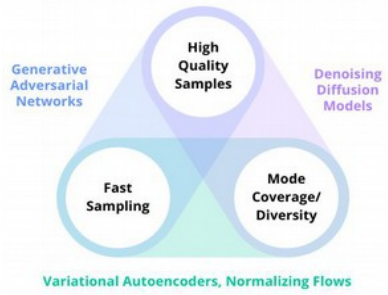


Figure 1: The Impossible triangle

GAN(s):

- Quality high: high quality samples
- Speed high: Fast sampling
- Coverage low. Not good coverage

Denoising diffusion models:

- Quality high: High quality and high coverage,
- Speed low: low sampling speed

Variational autoencoders, normalizing flows, etc.: fast, good coverage, but low quality.

As of today, there is no algorithm that is good at speed, quality and coverage.

1.9.3 Fundamentals of GAN(s)



The Generator: a NN that generates images from noise.
The Discriminator: a NN that classifies an images a either Real=True or Fake=False

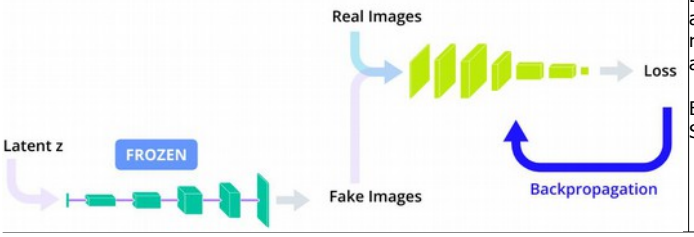
During training: the Generator tries to learn how to fool the Discriminator, while the Discriminator tries to get better at distinguishing between Real and Fake.

We start from a random vector generated from a random distribution (called **Latent vector Z**) dimensionality is e.g. 100, in our example the dimensionality is 2.
The Generator generates an image from the latent vector Z. Usually the Generator, have multiple layers: in each layer the number of channels is decreased until 3 channels. At the beginning shape is (1024,1), at the end (Height, Width, 3).. The channel count is decreased, while increasing the size of the feature map.
The Loss of the Discriminator is used to improve both networks.

How to train a GAN?

- **Discriminator training**
  - Fix G, get some G(z)
  - Train D for 1 step; the D is fed with with either real images or the images generated by G.
- **Generator training**
  - Fix D
  - Train G for 1 step to try to fool D: G tries to optimize so that its samples are considered True (i.e. real) by the D.
- **Repeat the process**, until the Generator does not improve any more.

1.9.4 Discriminator Training



Discriminator training: the G is frozen. G generates some images. They are fed to the D, along with real images. D must distinguish fake from real, so a Loss value can be calculated, with backpropagation the D weights are modified to better distinguish fake fom real.
D output is the probability that the input image is real.
Binary Cross-entropy loss is used as loss function.
Split batch: first real data, then fake data.

1. The Role of the Discriminator
The Discriminator's task in a GAN is to distinguish between real and generated images. During training, this component learns to identify nuances that differentiate authentic images from those created by the Generator.

2. Training Process: The Split-Batch Method. A popular method for training the Discriminator is the 'split-batch' technique. This involves two main steps:

Step 1: Handling Real Images
The Discriminator is fed real images and learns to identify them as authentic.
This process involves a forward pass of real data through the Discriminator, generating a probability score for each image being real.
The Binary Cross Entropy (BCE) loss is then calculated by comparing the Discriminator's predictions against the true labels (real images).

Step 2: Dealing with Fake Images
Next, the Discriminator is presented with fake images produced by the Generator.
These images undergo a similar process, with the Discriminator learning to label them as fake.
The BCE loss is again used to compare the Discriminator's predictions against the true labels (fake images).

3. Updating the Discriminator
After processing both real and fake images, the Discriminator's weights are updated. This is done using the gradients accumulated from both sets of data, ensuring that the Discriminator improves its ability to differentiate real from fake images.

When we call `backward()`, the content of the computation graph is destroyed. `detach()` preserves the information from being destroyed in the backward pass.

```
optimizerD = optim.Adam(D.parameters(), …)
criterion = BCELoss()  # Binary cross-entropy loss.

for data, _ in dataloader:
    # ... omitted, moving data to GPU etc.
    # data contains real images, the ground truth is 1.
    # Probability for data to be real, according to the Discriminator
    D_pred = D(data).view(-1)

    # Ground truth labels: here the images are real
    # so the labels are all 1
    labels = torch.full((size,), 1.0, device=device)

    # Compare Discriminator prediction vs ground truth
    loss_on_real_data = criterion(D_pred, labels) # criterion = BCELoss

    # Compute gradients
    loss_on_real_data.backward()
    # We do not call optimizerD.step() now

    # Get latent vectors and generate fake images using the Generator
    latent_vectors = torch.randn(
        b_size,
        latent_dimension,
        1,
        1
    )
    fake_data = G(latent_vectors)

    # Get predictions from the Discriminator
    # on the fake data
    # NOTE: remember to use .detach()!
    D_pred = D(
        fake_data.detach()
    ).view(-1)

    # Since these are all fake images, the ground
    # truth should be 0 for all labels. We refill
    # the tensor of labels we already have instead
    # of creating a new one
    labels.fill_(0)

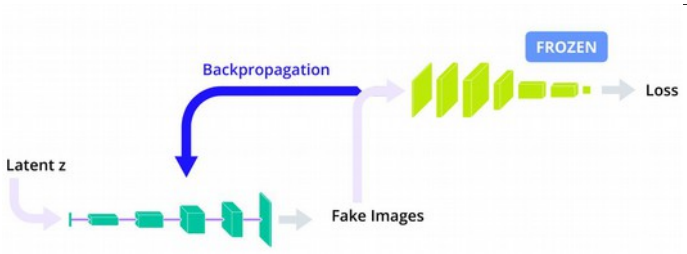
    # Compare Discriminator prediction vs ground truth
    loss_on_fake_data = criterion(D_pred, labels)

    # Add gradients computed on the loss on fake data
    loss_on_fake_data.backward()

    # Finally update the Discriminator
    optimizerD.step()
```

Why the detach? During the forward pass, the autograd keeps track of all computations, to be able to calculate the gradients, later; the computation graph is destroyed when we call .backward(): Detach returns a new tensor, detached from the current graph.

1.9.5 Generator Training and Inference



Generative Adversarial Networks (GANs) have revolutionized the field of AI-driven image generation. A crucial aspect of their success lies in the training of the Generator, which is responsible for creating realistic synthetic images.

1. The Generator's Objective

The Generator in a GAN starts with a random noise vector, known as latent z, and transforms it into a synthetic image. The goal of the Generator is to create images so convincing that they can fool the Discriminator into believing they are real. This is accomplished by trying to maximize the loss of the Discriminator on the fake data. Maximize the loss: we are feeding fake data, but the Generator objective is making sure that the Discriminator classifies them as real => output close to 1 (the Discriminator output is the probability that the input is real.).

2. Training the Generator

The training process of the Generator involves several key steps:

- **Generating Fake Images**: The Generator creates fake images from the latent z vector. We are actually going to reuse the fake images we have generated previously during the training of the Discriminator, but this is just an optimization.
- **Discriminator's Evaluation**: These fake images are then passed through the Discriminator, which is kept frozen during this phase. The Discriminator evaluates these images and assigns a probability score to each, indicating how likely they are to be real.
- **Loss Calculation and Backpropagation**: The Generator then

```
optimizerG = optim.Adam(G.parameters(), …)
for data, _ in dataloader:
    … # Discriminator training

    G.zero_grad()

    # Get a prediction from the Discriminator on the
    # fake data we already generated during Discriminator
    # training
    D_pred = D(fake_data).view(-1)

    # BCE trick: instead of maximizing BCE when
    # y = 0, we minimize the BCE when y = 1. These
    # are equivalent, but minimizing can be done with
    # the normal Gradient Descent algorithm
    labels.fill_(1)
    loss_on_fake_G = criterion(D_pred, labels)

    # Compute gradients
    loss_on_fake_G.backward()
```

adjusts its parameters to maximize the loss derived from the Discriminator's evaluation. This loss reflects how well the Generator is fooling the Discriminator.
3. Binary Cross Entropy Trick
It can be shown mathematically that maximizing the Binary Cross Entropy loss of the Discriminator on fake data (with label y=0) is equivalent to minimizing the same loss assigning y=1 instead of y=0.
	`# Update Generator to maximally-fool the`
	`# Discriminator`
	`optimizerG.step()`

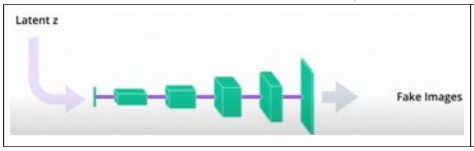
4. Inference: Utilizing the Trained Generator

Once training is complete, the Discriminator is discarded, and the Generator is used for image generation. New images are created by feeding random latent vectors into the Generator, which then produces diverse and realistic images.

In summary, training the Generator in GANs is a delicate and intricate process. By effectively maximizing the loss of the Discriminator on the fake data, the Generator learns to produce increasingly realistic images.

Binary Cross Entropy: $H_p(q) = -\frac{1}{N}\sum_{i=1}^{N} y_i \cdot log(p(y_i)) + (1-y_i) \cdot log(1-p(y_i))$

We have target labels =1 => $y_i = 1$ => BCE trick is the face that $y_i = 1$ means true image.

	How to use the generator. At the end of the training, the D is dropped, from a latent z random vector, a realistic image is produced.

1.9.6 GAN(s) are Tricky to train

GAN(s) are tricky to train: if G or D becomes too good to fast, the training stops working. In classification, we have the validation loss that guides our decision on whether to stop. In GAN(s) there is no stopping point. In general, the feedback from D becomes worse over time, as G becomes better, but w

Mode collapse: G finds a particular type of image that fools D , so starts generating only that, so the diversity decreases: called mode collapse. We can fall into a proble, or another, with a slight change of the parameters.

Challenges in Training GANs

- **Unstable Balance**: Training GANs is delicate; if either the Generator (G) or Discriminator (D) becomes too proficient too quickly, the other lags, disrupting the training process.
- **No Clear Convergence Indicator**: Unlike traditional neural networks, GANs lack a clear metric like validation loss to signify convergence, making it hard to determine the optimal stopping point.
- **Mode Collapse**: A critical issue where the Generator discovers a specific image type that always fools the Discriminator, leading to a lack of diversity in generated images.

W-GAN: Instead of a D that produces a label, we can use a critic network, that assigns a score to a given image. Training becomes also a little faster.

Progressive GAN(s) G generates low-resolution images in the first layers and add details in subsequent layers. This helps convergence and resolution.

Style GAN(s): mapping M(z) → style in G. Style is fed into multiple points of G. Style GAN(s) also add noise to generation in G.

There are extensions to GAN(s) so that they execute conditional generation, conditioned on some input, while the original one were unconditional generators.

1.9.7 GAN(s) Pros and Cons

Generative Adversarial Networks (GANs) have made significant strides in the field of AI-driven image generation. Understanding their strengths and weaknesses is key to leveraging their full potential.

Pros of GANs

- Speed: One of the standout features of GANs is their speed during inference. They require only a single forward pass of the latent vector, resulting in sub-second latency on modern GPUs. This makes them incredibly efficient for generating images quickly.
- High Sample Quality: GANs are renowned for their excellent sample quality. They consistently rank as state-of-the-art in terms of Fréchet Inception Distance (FID) across various datasets, even when compared to newer generation algorithms like diffusion models. The level of detail in synthetic faces and animals created by GANs is often astonishingly high.

Cons of GANs

- Poor Mode Coverage: A notable drawback of GANs is their tendency to have poor coverage. The Generator in a GAN often prefers exploitation over exploration. Once it finds a method to deceive the Discriminator or Critic, it tends to overuse this approach, leading to a lack of diversity in the generated images.
- Training Challenges: GANs are notorious for being tricky to train. They require a deep understanding and implementation of numerous training techniques to function efficiently and produce high-quality results.

In summary, while GANs boast impressive speed and sample quality, they face challenges in terms of mode coverage and training complexity. These factors must be considered when deploying GANs for practical applications in image generation.

1.9.8 Style GAN(s)

https://nvlabs.github.io/stylegan3/

1.10 Transformer-Based Computer Vision Models

1.10.1 Vision Transformers (ViT)

ViT excel at understanding visual data. When working with a transformer, you typically start with text, you tokenize it, text is converted to embedding vectors. Embedding vectors are taken from an embedding dictionary. There is also positional encoding, represenging the order of the word in the sentence. The input data is thus passed to the transformer layers, for training.

How can we apply the transformer developed for NLP, for visual data?

Image GPT explores how to replace text embeddings with an image vector, a simple and effective improvement.

In 2020 OpenAI released image GPT . Pixels are read row by row, left to right as if it was a text).
Image GPT:

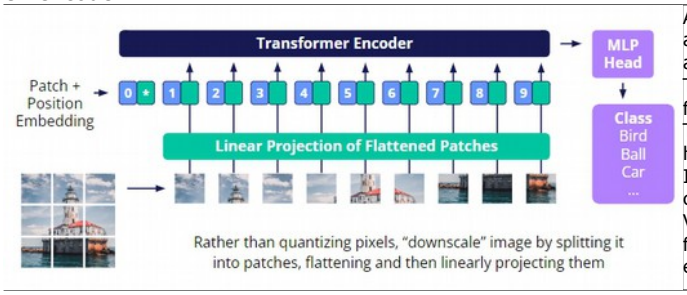
- Pros:
  - Effective at modeling visual information
  - State of the art on semi-supervised classification: a mix of labelled and unlabelled data
- Cons:
  - Substantal computational resources. 2500 V100 days for training, by comparison, other models like MOCO, can be trained in 70 V100 days.
  - Time complexity: these kind of models are limited on the sequence length that they can process . Limited to the resolution of 64x64 pixels, if the image size grows, the number of pixels grows quadratically.

Image GPT is not used anywhere today. It was SOTA for 64x64 pixel image classification in 2020. Image GPT is a precursor to modern Vision Transformers. It paved the way for current state of the art models.
Summary:

- ImageGPT converts images into pixel sequences similar to text tokens, adapting text embeddings for visual data.
- It excels in interpreting complex visuals and is effective in semi-supervised learning with labeled and unlabeled data.
- ImageGPT requires substantial computational power and is limited to a maximum of 64x64 pixel resolution images.
- ImageGPT is a foundational model for modern transformers but sees limited practical use.

1.10.2 Vision Transformer Architecture and Evaluation

Image GPT processes images as sequences of pixels. Modern transformers process images as sequences of patches, fed into an en encoder.



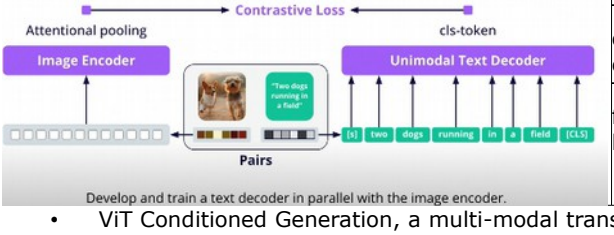
An image is broken down into patches of 16x16. Each patch is analogous to a word in text processing. Each patch is flattened and linearly projected, and used to generate an embedding, similar to text tokens.
Then, the embeddings+position, are fed into the transformer, as it is done for text-
The ViT repurposes the transformer for visual data. The output is a MLP head classifier.
Improvement of ViT over previous CV models: ViT have lower training costs, compared to previous models.
Vision Transformers process images by dissecting them into 16x16 patches, flattening them into embeddings, and using these as tokens in a Transformer encoder to capture complex patterns and bridge language and vision.

- Vision Transformer Pros:
  - ViT scale well, allow us to take advantage of huge datasets.
  - ViT are scalable and versatile for complex tasks
  - They serve as robust foundational models. They can be adapted and fine-tuned for various tasks
- Vision Transformer Cons:
  - Data: they require huge datasets to train . It can be a limiting factor in resource constrained environments.
  - Compute: they require amount of data to achieve optimal performance. Vast amount of labelled data must be performed

Where are ViT used today? They are SOTA in most vision tasks.

1.10.3 Conditioned Generation

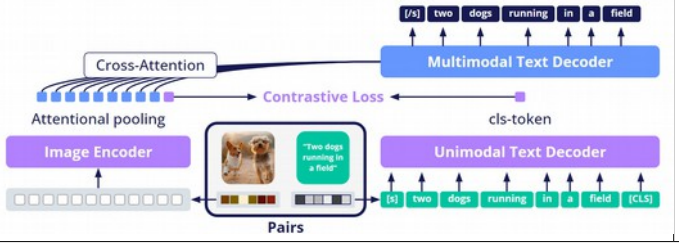
Multi-modality: multiple representations of the same information. Vision & Text: both can be transformed in vector embeddings, separately. Then they are combined: concatenated or added together. Example: merge the image embedding and text embedding.
To train a multimodal vision transformer, first step is merging the text and image embeddings. This results in a composite embedding that encapsulates the description of text and image. The combined embedding is fed into an Image encoder. That produces a set of features. In order to preserve the relationships between model understanding of image and the text representation, the model trains both a text encoder and an image encoder, side by side.. "Develop and train a text encoder in parallel with the image encoder."



The key to this method lies in the contrastive training approach. In contrastive training, contrastive loss is pivotal to align the text and image encoder, to match the corresponding text and image data.
The contrastive loss is low when text and image are aligned: it is a loss function, low value is better. Without contrastive loss, encoding would learn an incorrect image-text association.

- ViT Conditioned Generation, a multi-modal transformer, is used in diverse fields like self-driving cars and crime detection, integrating various data types for complete understanding.
- It converts images and text into vector embeddings, merging them for a unified representation of visual and textual data.
- The model's image encoder processes these merged embeddings, analyzing visual elements and textual context in a manner akin to film direction.
- ViT Conditioned Generation uses contrastive training to align text and image encoders with relevant pairs, ensuring precise text-image matching for effective visual-textual task performance.

Multimodal Vision Transformer Cross attention: the model focuses simultaneously on text and image input the model dynamically adjusts its focus between text and video modality, depending on the context and task. Multimodal Text Decoder.

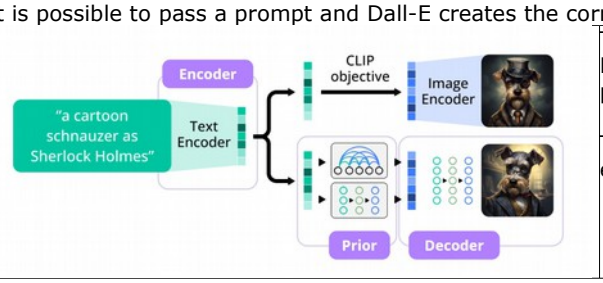


- ViT models use cross-attention to process text and visual data efficiently, adjusting focus contextually to preserve each modality's uniqueness.
- They transform images into patch vectors and merge these with text embeddings for a unified multimodal representation, processed by an Image Encoder and Text Decoder.
- These transformers are versatile, aiding in image description generation, visual question answering, and delivering real-time data for autonomous vehicles, capitalizing on their adept handling of visual and textual data interplay.

Multimodal Vision Transformers integrate visual and textual data by encoding image patches and text embeddings into a shared representation, which is then refined through contrastive learning and cross-attention, enabling the model to generate context-rich interpretations for complex tasks that combine visual and textual analysis.
Applictions: Visual Question Answering

1.10.4 Dall-E model for Image Generation

DALL-E is adept at creating vibrant images from text prompts, used extensively by educators, marketers, artists, and designers for various visual applications.
Its architecture includes a CLIP-enhanced encoder for text comprehension, a Prior model for converting text to image embeddings, and a decoder for image generation.
DALL-E's CLIP component is vital in connecting text and image understanding, analyzing images in the context of text to effectively associate visuals with related concepts and words.
It is possible to pass a prompt and Dall-E creates the corresponding image. CLIP : Contrastive Language Image Pretraining



Test encoder: understands the process semantic meaning.
Prior: generates clip image embeddings based in the text embeddings produced by the text encoder , it generates an encoder.

The Decoder receives the output from the prior , takes image embeddings and translates them into pixel space.

CLIP: a pre trained mode that bridges the gap between text and images

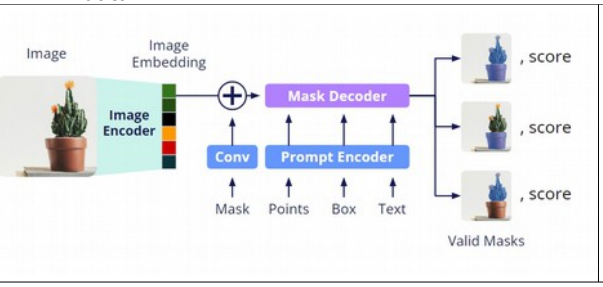
1.10.5 DINO v2

DINO v2: is a vision transformer model, different from Dall-E. Dall-E is known for image generation from text. DINO encodes visual features, for tasks such as depth estimation, semantic segmentation, classification, image classiification Traned with self-supervised learining. DINO can work out of the box without fine tuning. Image of the seal: parts of the animal are colored differently.
DINO v2 can be used straight from Hugging Face. Directly extracts features from the image, it does not use text. It uses Student-Teacher in training

1.10.6 SAM

Segment Anything Model (SAM). SAM uses encodings, to understand specific parts of the image. Can accept as input different types of data, like: Three components:

- task: promptable segmentation task; the task is the segmentation task, any indication about what to segment in an image, example a point= sparse prompt, or a mask=dense prompts
- model
- data



SAM model

1.11 Course: Diffusion Models

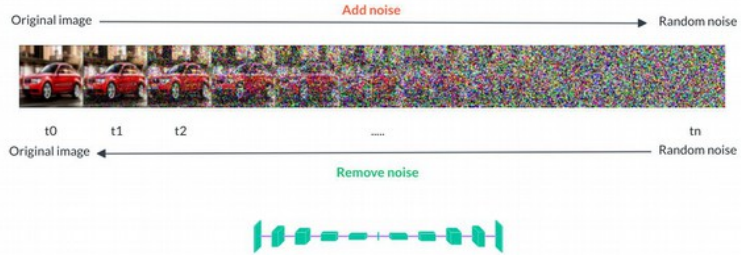
1.11.1 Diffusion Models

In the realm of generative AI, diffusion models have emerged as a cornerstone technology. These models are instrumental in generating diverse media, including images, videos, and audio. This lesson aims to demistify how diffusion models function, offering a concise yet comprehensive understanding of their workings. The Two-Phase Process: Forward and Backward
At the heart of diffusion models lies a two-phase process: the forward phase and the backward phase.

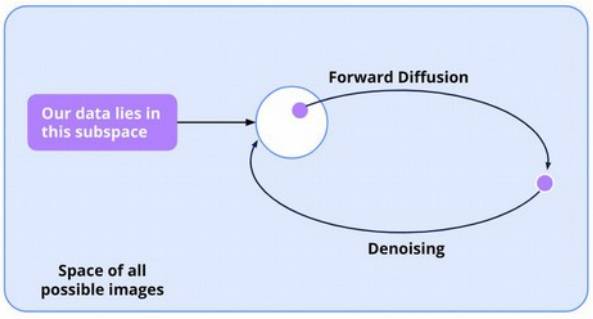
- **Forward Phase**: Here, we begin with an image and progressively add noise until we are left with nothing but pure noise. It's a gradual transformation from structured data to randomness.
- **Backward Phase**: Conversely, in the backward phase, we start from noise and denoise more and more until we're left with a realistic image. This phase essentially reverses the chaos introduced in the forward phase, reconstructing a coherent structure from the noise.

Training phases: forward and backward.

- Forward: start from an image gradually add noise until the image is pure noise
- Backward: start from the noise and try to recover the image , remove it gradually until we are left with a realistic image.



The "diffusion" name comes from the physical process called diffusion. In a liquid, concentrated substances diffuse in the liquid, until the liquid in uniform. Adding noise is like diffusing an image of a kind (car) in the liquid. The liquid is the space of all possible images. Denoising takes back the spread molecule into the original small volume.



The term 'diffusion' in these models is inspired by the physical process of diffusion. Imagine introducing a drop of colored liquid into a container of another liquid. Over time, the molecules of the colored liquid randomly move and spread throughout the other liquid, leading to a mix. This process is analogous to how diffusion models manipulate and generate images in a dataset: the forward process "takes" the images from their subspace (for example, the subspace of all images of cars) to the larger space (the space of all images) by gradually adding noise. The backward process essentially "rolls back time" and remove noise gradually until the image is back into the original subspace.

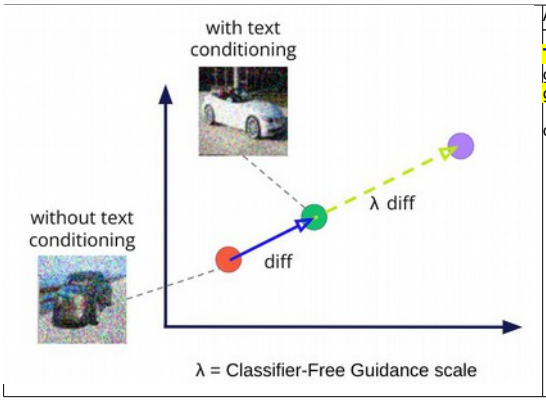
Denoising Diffusion Probabilistic Models (DDPM)
Why many small steps? It is easier to go from a slightly corrupted image to a non corrupted image. Starting from noise and reversing to the original image in one step is much harder.
Lately there has been attempts at building diffusion models that can jump from noise to images in one or very few steps. Despite these early successes, this remains an open field of research.
Generating new images: from Noise to a realistic image.
To generate a new image, we start with random noise and employ the model iteratively to remove the noise gradually. In the initial steps (which remember is time T), the model largely invents as it deals with mere noise. However, as the process progresses, a picture starts emerging, and the model adds finer details until the final, realistic image is achieved at t = 0 (remember we go "back in time"). This process is unconditioned: the result is an image similar the the ones of the dataset.

1.11.2 Conditioning

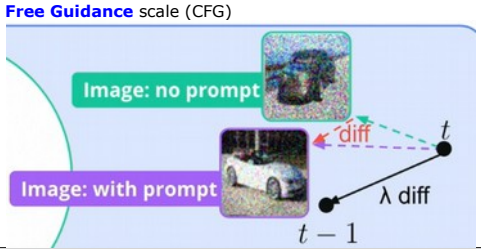
So far we talked about an unconditional diffusion model. Another ingredient is **text conditioning**. We provide a text description. We describe what we want in different ways:
Sketch to Image Conditioning: In this approach, the model is conditioned with a sketch. The user provides a simple sketch or outline of an object or scene. The diffusion model then generates a detailed and realistic image representing the depicted object based on this sketch. This type of conditioning is particularly useful for artists and designers who want to turn their sketches into more detailed and lifelike images.
Image to Image Conditioning: This form of conditioning involves starting with an existing image and asking the model to generate variations of it. For instance, you could begin with a picture of a car and request the model to create a futuristic version of the same car. This method is beneficial for applications such as design modifications, where the user needs variations of a base image, perhaps to see different styles or hypothetical modifications.
Style Conditioning: Style conditioning involves asking the model to generate an image of something based on a given style represented by an input image. For example, you could request the model to generate an image of a car in the style of a famous artist like Van Gogh.
Each of these conditioning methods leverages the core capabilities of diffusion models, adapting and extending their generative powers to specific user needs and creative scenarios. They exemplify the flexibility and potential of diffusion models in generating customized, high-quality content.
Enter Text Conditioning. The most common conditioning by far is text conditioning. Image generation is guided by the text. Text embeddings: we pass text to a language model, which outputs an embedding vector: a numerical summary of the text meaning. How to incorporate the text? When we are denoising, at every step, we pass the image along with the text embedding. We feed the text embedding in different places, deep inside the model using cross attention, (*the power of the embedding vector*

must be strong). During training we remove text embeddings for a random fraction of the training, so the model also learns unconditional generation.
Classifier-Free Guidance: A novel approach called 'classifier-free guidance' is used at the inference stage. This method boosts the importance of the text prompt, ensuring the final image closely adheres to the given text description.

1.11.2.1 Inference

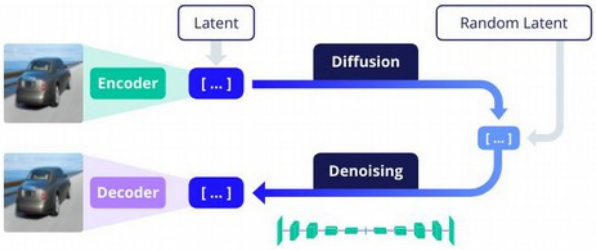


At inference time, we feed the noisy image and the text embedding, at every layer of the network.
This is not enough: the conditioning of the text is not strong enough, this way.
Trick: there is a first generation of an image, without using the text conditioning (orange dot), an a generation of a different image, considering the text conditioning (green dot). The two images must be generated from the same noise. The distance, in the image space, is (**diff**) We boost the step , from the orange dot, into the direction of diff, using the step $\lambda \vec{diff}$
λ is called **Classifier Free Guidance** scale (CFG)



1.11.3 Latent Diffusion Models

The DDPM(s) models operate at pixel level, they operate pixel by pixel and they are very compute intensive. Tricks were used to speed the process, for example: generate at low resolution and then use a separate neural network to upscale the image. This is sub-optimal because it requires training and using two neural networks.
Enter **Latent Diffusion Models**. Autoencoders are used to produce embeddings for images. In the bottleneck, autoencoders generate an image embedding, or **latent vector**.



Diffusion
- From the Original Image to the latent vector
- Add noise to the latent vector, until we get a random latent vector.

Denoising
- From the random latent to the denoised latent
- From the resulting latent to the output image.

Why? Because the size of the latent is far smaller that that of the original image, so the process is faster.

Example: input image can be 512x512 = 262144 values. The latent, in stable diffusion is: 4x64x64=16384 values.
There is reduction factor of 16 between the image and the latent vector.

1.11.4 Diffusion Models: Pros and Cons

Pros:
- Great sample diversity (good coverage): they are creative models.
- Great sample quality.
- Easy to conditon: it is easy to provide a natural way of conditioning the output in many different ways.

Cons:
- They are slow, because many forward passes are required to go from noise to the final image. The number of denoising steps are 10 or 100. In comparison, GAN(s) are faster.

Remember the impossible triangle (Figure 1: The Impossible triangle, page 36) (SDLX Turbo is a new approach (link))

1.11.5 DDPM Implementation in Theory (Denoising Diffusion Probabilistic Models)

Noise schedule: the noise is added as a Normal distribution, with a given mean and standard deviation $q(x_t|x_{t-1})$: calculates the new value of a pixel, given the previous value. $q(x_t|x_{t-1}) = N\left(\left(\sqrt{1-\beta_t}\right)x_{t-1}, \beta_t I\right)$. The value is shifted toward the zero and spread, depending on β_t . The noise distribution shifts and widens as we increase β .
$\beta_t I = \beta_t$ The value of $\beta_t \in [0,1]$ is generated by the noise scheduler, typically it is a linear series from start value to a final value, e.g. $\beta_1 = 0.0001$, $\beta_n = 0.02$.

The larger is the time step, the wider is the distribution. This is a sequential process; we must generate the time t only when we know $t-1$. Mathematically we can see that:

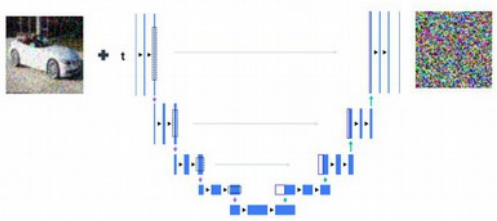
$\bar{\alpha}_t = \prod_{s=1}^{t}(1-\beta_s)$ $q(x_t|x_0) = N\left(\sqrt{\bar{\alpha}_t}x_0, (1-\bar{\alpha}_t)I\right)$: this allows to jump from zero to t in one step; the value of a pixel at time t thus depends only on the value of the pixel at time t_0 .

Backward pass. In the backward pass, we go from the random noise to the original image (*denoising*). The method we use requires the model to predict the noise that must be subtracted from the image to get a slightly less noisy image.
Forward pass at time t : we take and image and we add noise to it. In the backward pass at t , the model tries to predict the noise we added to the original image in the forward pass t , so that removing the predicted noise would give us back the original.
Loss function? $L = E_{t,x_0,\epsilon}\left[\|\epsilon - \epsilon_\theta\|^2\right]$ This is the loss function when training the model that predicts the noise on an input image. We want to minimize the expectation of the error between the predicted noise ϵ_θ and the actual noise ϵ . We want to minimize the expectation of the square loss. We are evaluating the loss over the noise, which is a random variable. We want to

minimize the mean of the difference, over all time steps. We will minimize using stochastic gradient descent, but averaging over the minibatch. So we must sample different timesteps corresponding to different noise levels.
The idea is that there is randomness in this process, so the model will never be able to predict exactly the right noise ϵ for a single time step t of a single image x_0 .
However, if we consider at once a lot of time steps, images, and realizations of the noise, the difference between true noise and predicted noise should on average go down as the model gets better and better. In practice, we approximate the expectation by averaging the simple Mean Squared Error loss $\left[\left\|\epsilon-\epsilon_\theta\right\|\right]^2$ over a mini-batch of many images x_0 where we take a random time step t for each image. For this to work well, it is better to use a relatively large batch size. We can see here that the reparametrization we did earlier, allowing us to generate an image at any time step t directly instead of serially, is crucial for the whole idea to work.

1.11.5.1 Unet: Encoder-Decoder with Skip Connections

What is the model? We need something that, given the image, predicts the noise. Unet, originally created for segmentation tasks. The output has the same size as the input. Receives both the image and the information about the time t at which it is operating, so the model can learn the noise schedule. The model is the same $\forall t$. That is, the model is shared across time.



UNet

How to input the time t ? We use **temporal embeddings**.

Positional encoding: $\sin\left(t\,k^{-\frac{2_j}{n_c-2}}\right)\cup\cos\left(t\,k^{-\frac{2_j}{n_c-2}}\right)$ $for\ j=1\ldots\frac{n_c}{2}$, where for example $k=10000$ and $n_c=256$: is the number of channels; t : timestep. The vector of the $\sin(\ldots)$ and the vector of the $\cos(\ldots)$ have size $\frac{n_c}{2}$, they are concatenated (thus the symbol $\cup$) to get a vector of size n_c . Positional encoding: each time interval has a slightly different vector. We apply a linear layer and an activation function to get the **temporal embedding**. The weights of the linear layer are learned.

1.11.6 DDPM Implementation in Coding

```
n_steps = 512                           # This is called T in the formulas
beta = linspace(start, end, n_steps)    # Linear noise schedule

# Precompute some quantities for the reparametrization
alpha = 1. - beta
alpha_bar = cumprod(alpha, axis=0)

sqrt_alpha_bar = sqrt(alpha_bar)
sqrt_one_minus_alpha_bar = sqrt(1. - alpha_bar)

model = UNet()
optimizer = Adam(model.parameters(), lr=0.001)

for batch, _ in dataloader:
    ... ## move batch to GPU, reset optimizers, then:

    bs = batch.shape[0] # Number of samples in the batch (batch size).
    # Random time steps, one for each image in the mini-batch
    t = torch.randint(0, n_steps, (bs,)).long()

    # Generate noise and add it to the images (forward pass)
    noise = torch.randn_like(batch, device=device)
    x_noisy = (
        sqrt_alpha_bar[t].view(bs, 1, 1, 1) * batch +
        sqrt_one_minus_alpha_bar[t].view(bs, 1, 1, 1) * noise
    )

    # Predict the noise using the model
    # NOTE how the model receives in input the noisy image AND the time step t
    noise_pred = model(x_noisy, t)

    # Approximate taking the expectation by averaging
    # the MSE loss over the minibatch and the random time steps
    loss = F.mse_loss(noise, noise_pred)

    # Compute gradients and optimize the model weights
    loss.backward()
    optimizer.step()
```

This pseudo-code implements the training of the model that predicts the noise, given the time t and the noisy image x .

$$q(x_t|x_0)=N\left(\sqrt{\bar{\alpha}_t}x_0,(1-\bar{\alpha}_t)I\right)$$
$$=\sqrt{\bar{\alpha}_t}x_o+\left(\sqrt{1-\bar{\alpha}_t}\right)N(0,1)$$

This formula is used in the pseudocode to calculate x_{noisy} .

1.11.7 Inference

Sampling and Generation: In the generation phase, the model starts with pure noise and applies the learned denoising process

in reverse, to get the starting image without noise. It is customary to loop over the denoising steps in reverse, starting at T and ending at $t=0$. At each step we subtract the noise predicted by the model from the current best estimate of the image, renormalized appropriately to neutralize the mean shift we applied during the forward process: $x_{t-1}=\frac{1}{\sqrt{\alpha_t}}\left(x_t-\frac{1-\alpha_t}{\sqrt{1-\alpha_t}}\epsilon_\theta\left(x_t,t\right)\right)+\sigma_t z$,

where z is a number extracted from a Normal distribution with zero mean and variance 1.

Note here how we are subtracing the predicted noise ϵ_θ from our best guess so far x_t, but we are also adding more noise!

This is counterintuitive, but it promotes diversity in the samples and reduces correlation between each time step, thereby allowing the model to avoid accumulating mistakes. This new noise term $\sigma_t=\frac{1-\bar{\alpha}_{t-1}}{1-\bar{\alpha}_t}\beta_t$ is large at the beginning of the sampling (at time T) and shrinks as we go towards $t=0$. This encourages the model to focus on the broad picture at the beginning of the denoising, and on the details at the end of the denoising. It also gives the model more freedom to explore.

```
# Compute some quantities we need, contained
# in the formulas reported above
sqrt_one_minus_alpha_bar = sqrt(1. - alpha_bar)
alpha_bar_t_minus_1 = F.pad(
    alpha_bar[:-1],
    (1, 0),
    value=1.0
)
# This is called \sigma_{t} in the formulas
# posterior_variance = (
    beta * (1.0 - alpha_bar_t_minus_1) /
    (1.0 - alpha_bar)
)
# Batch size. For example, generate 8
# fake images
bs = 8
# Generate the random initialization
# (remember, an image filled with random
# values is the start of the reverse process)
x = randn((bs, 3, IMG_SIZE, IMG_SIZE))
# We loop "backward in time"
for ts in range(0, T)[::-1]:

    # This is the noise z that we will add back
    # into the denoised image (except if we are
    # at the very last iteration)
    noise = randn_like(x) if ts > 0 else 0

    # For each image in the batch we consider
    # the time step ts
    t = full((bs,), ts).long()

    # This is the formula for the denoising
    x = (
        sqrt_one_over_alpha[t].view(bs, 1, 1, 1) *
        (
         # Remember: alpha[t] = 1 - beta[t]
         x - beta[t].view(bs, 1, 1, 1)
         /
         sqrt_one_minus_alpha_bar[t].view(bs, 1, 1, 1) *
         model(x, t)
    ) + sqrt(posterior_variance[t].view(bs, 1, 1, 1)) *
        noise
# Cut every value below -1 or above 1
# (this is needed for the conversion back to RGB
# image to work well)
generated_image = torch.clamp(x, -1, 1)
```

1.11.8 Paper: Denoising Diffusion Probabilistic Models (20201211)

J.Ho, A.Jain, P.Abbeel

1.11.9 Hugging Face Diffusers Library

Explains the diffusers library from HF.

Example: filter and select only unconditional image generation, in Libraries select diffusers.

1.12 Course : Introduction to Building Generative Apps

Software Feature: Functionality or that enhances a software product's capabilities. It's a specific, self-contained component or behavior that fulfills a user's need or requirement.
Prompt: How you interact with large language models (LLM). A prompt is the input to which a model is meant to respond. Prompts can include instructions, questions, or any other type of text input.
Context (LLM): The amount of text a large language model (LLM) can process and respond to at once.
Fine-tuning: Adding capabilities to an AI model (new tasks or domain knowledge) by training it on new datasets.
Foundation models, trained on huge datasets. FM handle a variety of tasks, and can be fine tuned for a specific task.
Three layers:

- Application: user facing apps using generative models.
- Model/platform: API, pretrained foundation models, development kits and libraries; deployment (via a webservice or endpoint, embedded in an application, or deployetd in an edge device)
- Infrstructure / HW. The resources to train, fine tune and run generative models. GPU(s), TPU(s), hyperscale compute, cloud providers, Cohere.

1.12.1 The problem with generative models

The only way we can interact with large language models is through natural language. That's it! You're probably used to writing programming functions that return reliable, reproducible results. These models work differently.

- Generative AI models are unpredictable, and you send them natural language input, called a prompt, and receive output called a response that might be what you expected, or not.
- The same prompt might return different responses every time.
- The prompt you send the model will affect the quality of the response.

The other problem you'll face using these models in development is their stateless nature. If you use the ChatGPT web or mobile application, it might appear like the model is keeping a history of your conversation and every time you send a prompt, the model is somehow adding it to a memory of the conversation you are having. This is an illusion.

- Every time you call the API to send a prompt to the model, it responds in isolation, only using the messages you've sent in that API call to generate a response.
- These messages are called the context window of the model, and it's only so large.
- Suppose you want to create a chatbot that has a conversation with the user. In that case, you'll need to send all the previous messages in the entire conversation in your API call, which gives the model context of the entire conversation in order to generate a response that appears to the user like the model is remembering the previous conversation messages.
- At some point, you'll run out of context length in the unit of tokens to send the entire conversation, and you'll only be able to send the most recent messages

Tools and techniques exist to deal with this issue from a development standpoint

1.12.2 Prompt Engineering

The relationship between prompts and prompt engineering bridges both the model layer and the application layer.
Without well-engineered prompts, generative models can provide poor output, resulting in your application not working the way you envisioned it.
Code in the application layer often pre-processes or structures user input into effective prompts for the model.

Let's consider an application that helps users identify potential illnesses based on symptoms they input. The user might input "I have a fever, cough, and tiredness."

- The application interface could also provide checkboxes or buttons for common symptoms to aid users.
- The raw input from the checkboxes the user checked will be too ambiguous for the model.
- Application logic must engineer a prompt based on the selected checkbox values ({fever}, {cough}, and {tiredness}).
- Code pre-processes the checkbox values into this prompt: "*What are the potential illnesses associated with symptoms like {fever}, {cough}, and {tiredness}?*" This is the prompt we send to the model.

The model receives the engineered prompt and outputs a response based on its training. It might respond with "*The symptoms of fever, cough, and tiredness can be associated with conditions like the flu or mononucleosis. It's essential to consult a healthcare professional for a definitive diagnosis.*"

The application post-processes the model's response and displays it to the user in the application, perhaps providing links to more information about each one. It might add disclaimers or advise the user to consult a physician. In this example, prompt engineering in the application layer ensures that the model receives the most effective query to provide a relevant and valuable response.
Prompt: input statements or questions provided to a model to generate a desired output or response.

1.12.3 Generative AI Features and Solutions

How do generative models enable us to create new types of applications and solutions?

Generative models excel at understanding context, filtering relevant information, and synthesizing data into a coherent narrative. We can pass generative models the context behind user queries, enabling them to be effective in tasks like personalizing content recommendations or tailored responses in chatbots.

- Tasks related to natural language processing or NLP become significantly more straightforward. These models can understand, generate, and even translate languages with high proficiency.
- Products and tasks that use images or allow users to manipulate images can leverage generative image models. Integrating computer vision capabilities using generative models is significantly less work than using traditional AI models to accomplish the same tasks.

Designing and developing features, applications, and solutions that leverage generative models is different than developing with traditional AI systems. Instead of just analyzing and predicting data, we can generate entirely new data that's coherent and

contextually relevant. The capabilities of these models open up a realm of possibilities for you to create innovative applications.

- New categories of applications: Generative AI models can draft articles, marketing copy, narrative or analytical content like reports. Consider applications like **Canva**, **Lensa** (https://lensa.app/), or Writer.com. These applications are full-featured solutions for workflows that generate and modify texts, images, and specific types of content for users.
- Natural conversation powered by large language models can handle customer queries, provide information, and engage users in conversation.
- Retrieval augmented generation solutions, known as RAG, combine the ability of models to retrieve information from a database and document sources and then use that information to generate content and contextually relevant responses. You can use RAG to build applications that contain question-answering features, chatbots, document summarization and analysis, or content recommendations.
- It's also possible to fine-tune a generative model and build applications that use the model. This is when you train a generative model, like open-source models, on specific data to make it more suitable for niche tasks. Cloud providers also offer tools for fine-tuning and deploying models. In order to keep the data in fine-tuned models private.
- Existing applications, from content management systems to customer service platforms, can integrate generative AI features to enhance their offerings. For example, an e-commerce platform like Etsy or Wix can use generative AI to automatically draft product descriptions, answer customer queries, or build product web pages. Large software products like Microsoft Office and Google Workspace are integrating LLMs in word processing products or image generation models into presentation software products. Grammarly uses LLMs for an autosuggestion and feature to rewrite texts in its product.

Risks and Challenges

- It's crucial to understand that these models provide outputs based on their training data, which doesn't guarantee correctness. Probabilitic output.
- Generative models are trained on biased datasets, so they can produce outputs that perpetuate or amplify harmful stereotypes or prejudices, leading to skewed perceptions and discriminatory output. This can be an organizational risk if you're outputting biased content to users.
- Developers and businesses should set up evaluation frameworks to assess the accuracy and relevance of the model outputs. Feedback loops where human reviewers can validate the model's responses can be beneficial.
- Be transparent with users about the model's capabilities and limitations.
- Costs involved with leveraging LLMs in your development. Be prepared for the cost of API calls, custom model hosting, and cloud services.

1.12.4 Generative AI Solution Components

- At the heart of your solutions are large language models for text generation or image generation models. Your application will be sending them prompts and receiving and processing their output.
- The user interface is where users interact with the application. It's the bridge between users and AI models.
- Application logic is the controller of the application. It processes user inputs, interacts with AI models, as well as parsing and managing the flow of data.
- Databases can store user information and AI responses or be used alongside OpenAI functions or LangChain tools, or you can use them to fetch data to augment in RAG solutions. You might also have vector databases in your app for semantic search solutions.
- APIs connect our application with external services and platforms. They allow our solution to communicate with other AI models or fetch data from other applications.

How do we know that the response from the model is not biased, toxic, leaking proprietary data, or will damage organizational reputation? There are several software libraries and products available to integrate into our applications that perform this important task.

The initial input, the resulting prompt, and the corresponding response from the large language model can all be stored in a database. This historical record can be used for tracking user interactions, building history features, refining the AI model's performance, or providing audit trails.

1.12.5 Your First Quest: Building a Health Wellness Agent

Tools like OpenAI API function calling and Link Chain enable output from a large language model to trigger actions in external systems, allowing an agent to interact with an external environment and make decisions in an autonomous way.

- Agents use large language models to generate both reasoning traces and task-specific actions.
- The reasoning aspect involves the AI agent thinking through a problem, breaking it down into smaller, manageable parts, and forming a plan or series of steps to solve it.
- The action component allows the agent to interact with external sources like databases or the internet to gather additional information relevant to the task at hand.
- Agents can operate autonomously or semi-autonomously within a specific environment, executing tasks, making decisions, or solving problems without continuous human guidance. Agents can perceive and react to their environment, which might be digital, like data from a database or retrieving data from the internet, and responding promptly to changes. For instance, a chatbot sending an email based on a user goal, or taking actions based on data from a database, or collecting data from the user.

Beyond just reacting to the environment, AI agents often exhibit goal-oriented behavior, taking proactive initiative based on their objectives or planning how to solve a problem. Many AI agents interact with other agents, including humans, to accomplish their tasks. You can even set up LLM-based agents that use other LLM agents.

ReAct Prompting for Agents

A popular prompting method called ReAct can be used to create LLM-based agents. REACT prompt techniques use LLMs to generate both reasoning traces and task-specific actions.

- The **reasoning** aspect involves the AI agent thinking through a problem, breaking it down into smaller, manageable parts, and forming a plan or series of steps to solve it.

- The **action** component allows the agent to make decisions about gathering additional information relevant to the task.
- The **observation** step often involves delivering a final response about a task or recording an event. These observations can be used for further reasoning. REACT-enabled agents actively seek out new information, update their understanding, and refine their reasoning based on observations.

A REACT prompt for a wellness agent might be:

"*Your goal is to improve the wellness of the user by interleaving thought, action, and observation*."

Agents involve an LLM making decisions about which Actions to take, taking that Action, seeing an Observation, and repeating that until done.

LLM(s) are capable of reasoning and planning , so they can be used to create agents. Tools like OpenAI can interact with an external environment and make decisions in an autonomous way.
Key characteristics of agents are: autonomy, reactivity fron the environment e.g. change fron a database, email from a user, etc., proactivity, social ability= interaction with other agents or humans to accomplish their tasks.
ReAct is a popular prompting framework, it can be used to .create LLM-based agents: Use reasoning and actio to solve a problem or accomplish a goal.

- Thought: reasoning and planning (breaking the task into smaller manageable parts)
- Action: select and perform task specific actions
- Observation = response about a tssk or record of an event. Observation can be used for further reasoning

1.12.6 Designing a Project management Assistant Solution

Imagine a helpful assistant that you can interact with to help you with project management tasks. We'll be designing and building a project management assistant.

The first step in designing an assistant solution is to design the tasks that the assistant can perform. We'll identify these tasks that leverage function calls to interact with external systems and data sources:

- task retrieval and status updates
- project reporting and analytics,
- resource allocation and scheduling.

The next step is to define some **user stories** and example conversations the assistant might have. management assistant

- We'll use these stories to identify user intent
- User intent is mapped to functions and parameters.

The last step in our design effort is to define prompts and prompt templates to ensure users understand what tasks the project management assistant is capable of.

- The application should provide clear instructions or an introductory message when the user first interacts with it. This guide should outline the assistant's capabilities and offer examples of how to use it effectively. This also becomes the model instructions or system prompt sent to the model when we start using it.
- Prompt templates are necessary for the application logic to guide the users through the conversation flow and to collect the required data to perform functions. If a user asks, "What are my tasks for today?" we need to identify the user ID and what date today is as parameters.

We'll be designing and building a Project management assistant: a helpful assistant that you can interact with, to help you with project management tasks.
Design the tasks. The first step is to design the tasks that the project management assistant can perform. For example (we chosen tasks that can leverage function call capabilities of the LLM):

- task retieval and status updates
- project reporting and analytics
- resource allocation and scheduling

Define the User Stories and Example conversations to:

- identify intent
- determine when functions must be called
- how to define and collect function parameters by asking follow-up questions.

Define prompts and prompt templates, to make sure that the user understands what the assistant is capable of:

- the application should provide clear instructions and messages, to guide the interaction
- the UI of the gent should outline the capabilities and offer examples
- Model instructions and/or system prompts.

When the user starts the assistant, a welcome message should be presented.

Suppose that the user asks: *What are my tasks for today?* There must be a prompt template like: "`What are {{my}} tasks for {{today}}?`" If user-id and date are not provided, ask for those details.

1.12.7 GPT Function calling

Function calling defines a consistent response format. Instead of receiving raw, potentially diverse outputs, the function ensures that the AI's response adheres to a predefined structure that other systems can easily interpret and utilize.

In the context of applications that leverage LLMs, function calling allows the model to interact with external APIs or databases, enabling the AI to pull in real-time data or trigger actions outside of its own dataset. This means that the AI can now answer questions like "*What is the current weather in Stockholm?*" by integrating with a weather API, something it couldn't do before based on its own training data. In the case of a project management assistant, function calling will:

- Allow the assistant to capture user intent from input and map the intent to defined functions
- Map the input to specific functions and parameters

- Use the identified functions and parameters to call functions in application logic that interacts with external systems
- Return the output from external systems to the model to augment a response

Function calling, in LLM application, is a more structured approach to handling responses from the LLM, acting as the intermediary between its generative capabilties and more consistent, and actionable results. Function calling is similar to ordering from a menu. Instead of receiving raw, potentially diverse outputs, the function ensures that the AI's response adheres to a predefined structure that can be interpreted and utilized by other systems. In LLM applications, function calling allows the model to interact with external API(s) and databases; enabling the LLM to pull-in real time data or trigger actions out side of its own dataset. Note: OpenAI has function calling in Chat Completion and Assistant.

1.12.8 Craft an Interactive User Interface

What are some strategies and methods you can use to design user interfaces and experiences?

- Form fields in interfaces act as intermediaries, translating user inputs into structured prompts for LLMs. The design should prioritize ease of use, clear instructions, and minimizing user error. Each form field can be aligned with a parameter in the LLM prompt to provide the model with clear and unambiguous input.
- UI elements like placeholders, tooltips, and help text are important to guide users on required information.
- Leverage design methods and user experience platforms for developing solutions for Google Assistant, Microsoft chatbots, or Amazon Alexa. These interfaces rely on natural language conversation and are similar in how they interpret various user inputs and intents.
- Educating users on interacting with the system and providing clear feedback and confirmation are crucial in conversational UIs. Recognizing user intent is central to this design process.
- Systems are built to recognize **user intent** through language understanding and map these intents to specific responses or actions. This involves parsing language, understanding semantics, and identifying key actions.
- Avatars add a visual and personal dimension to the chatbot experience, making interactions more engaging and personal. Advances in technology have made avatar-based interactions more lifelike and dynamic.

1.13 Building Solutions with Vector Databases

Vector DB are long term menory for Generative AI. They allow us to extend the knowledge of the model without retraining. Vector DB(s) are used in multiple ways:

- search engines and recommender systems
- computer visions
- Generative AI: instead of sending the user query to the model directly, we first retrieve the most relevant information from the Vector DB. We combine the retrieved information (called context) with the user query to form a "context aware prompt". We use the LLM to answer the question using the context aware prompt, it reduces the allucination and get data that is not part of the training corpus-

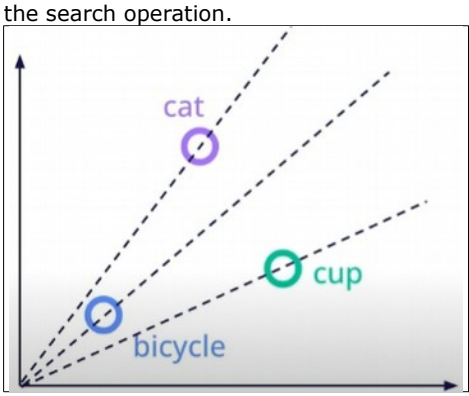
Allucination is in general when the response is invented or not accurate. Vector DB(s) are the long term memory

1.13.1 Vector Search Basics

Embedding: a list of floating point numbers. Anything (video, text, images, etc) can be turned into an embedding vector. The length of the vector is the number of dimensions that this vector has. All vectors in a vector column (of the DB) must have the same number of dimensions; for vector search, the query vector must have the same number of dimensions as the vectors in the searched column. An embedding vector is a point in a high dimensional space.

Distance can be straight line or Euclidean distance. $d(x,y)=\sqrt{\sum_{i=1}^{n}(y_i-x_i)^2}$: the general case with n dimensions.

KNN= K Nearest Neighbours = finding the k vectors nearest to a given vector. k : the number of results that are retrieved in the search operation.



Euclidean distance is not the only distance.

There is also cosine distance: it measures the angular distance between vectors.

Cosine similarity = $\cos(\theta)=\frac{A\cdot B}{\|A\|\|B\|}$

Cosine distance = 1 - cosine similarity.

Depending on the chosen distance measure, we get different results. The distance function affects which item is considered closer.

1.13.2 Embeddings

In search engines, using embeddings, we retrieve items related to the user query. Classification, embeddings to get the most similar label. Embeddings can be used in clustering and de-duplication. Anomaly detection: to find edge cases that are far from the normal distribution. BERT is used to generate embeddings from text.

- We are given the raw text
- Split the sentence into tokens; the tokens supported by the model, e.g.BERT.
  - BERT has sentence start= CLS and sentence separator = SEP.
- Generate the embeddings for each token. There is an Embedding lookup table (LUT). In BERT the

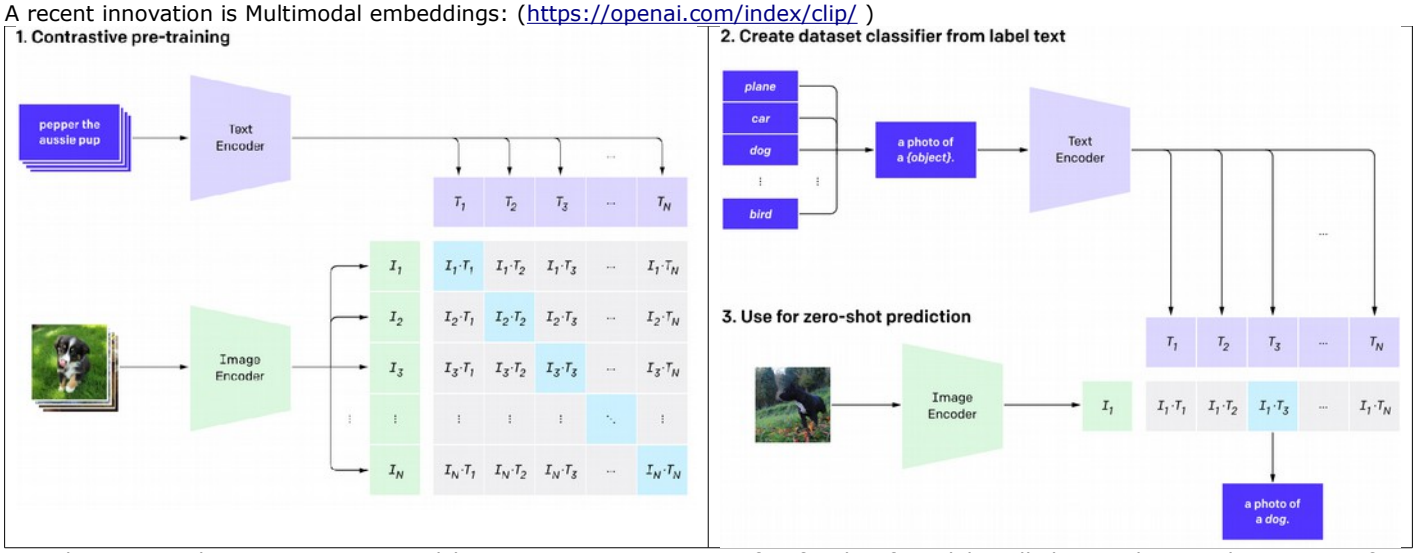
Embeddings in recommenders: a common technique is called matrix vectorization. Suppose we have a matrix of movie view history = for each user which movie he/she has watched. The matrix is factorized into two matrixes, representing users and the movies. Matrix factorization can be done using techniques like QR, SVD, etc. The result is, for example:

- embedding length is 768, there are about 30K tokens.
- How to generate the embeddings for the whole sentence? We take all the embeddings, we compute the element-wise average, for each item. That is the sentence embedding.



The result is a movie embedding and a user embedding. We can use the movie embedding to recommend a user a movie similar to one that he has watched in the past.

A recent innovation is Multimodal embeddings: (https://openai.com/index/clip/)



CLIP learns visual concepts via natural language. CLIP is part of a family of models called zero-shot. The Hugging face implementation of CLIP allow to extract embeddings from the text or the image. CLIPProcessor allows to create the embeddings from an image.
https://huggingface.co/docs/transformers/model_doc/clip

1.13.3 Vector Databases

Vector search tools:

- **Vector index libraries**: light weight libraries for fast vector search: e.g. FAISS, Annoy, etc. They miss the data management features of a database. E.g filtering, metadata management etc.
- **Traditional DB(s)** started adding features of Vector databases. They are traditional DB(s) with vector search. Pgvector, Elastic search, etc. They tend not to be scalable in the performance amd not optimized.
- **Pure-play Vector Databases**: complete data management solution for unstructured data. LanceDB, Milvus (https://milvus.io/), Pinecone, etc. They are more complete offerings.

Vector databases support:

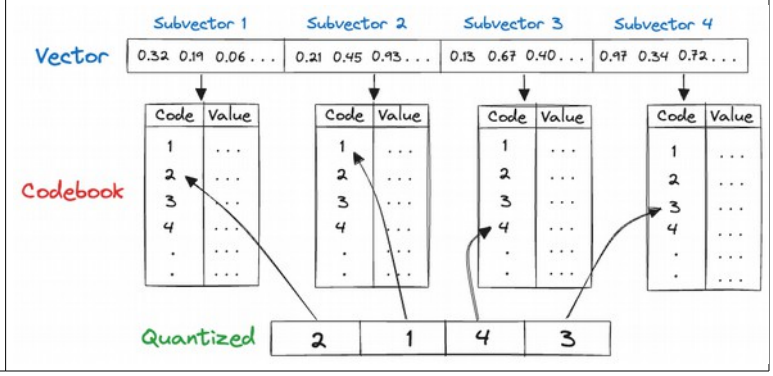
- indexing, updates to add and remove vectors incrementally, versioning.
- Querying and filtering to limit the results. Post processing:
- Separate the compute from the storage. E.g. use a cache and Object storage.
- Add metadata to support categorical search and filtering.
- Being able to add images and text along with the vectors (called raw data)
- We need flexibility on the query side. Real time search. Keyword search and SQL. *Keyword search*= searching for a word in text.

1.13.4 Lance DB

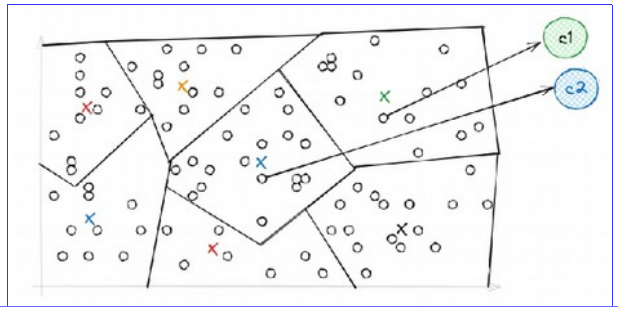
1.13.4.1 Indexing

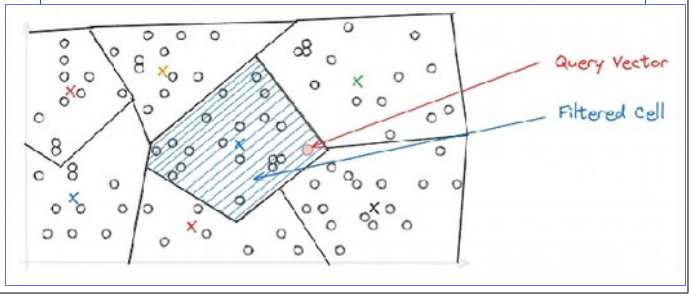
Approximate Nearest neighbors (ANN): faster but less accurate than kNN.https://lancedb.github.io/lancedb/concepts/index_ivfpq/

Product quantization (PQ) divides the embedding into subvectors.. Each subvector is assigned a reproduction value ID that maps to the centroid of points for that subvector.
Quantization is a lossy process.
PQ reduces the size of the index.
Suppose that the vector has `128D`, `32-bit` floats. The size is `128 * 32 bits = 4096`.
Quantizing to `8-bit` vector with `4` dimensions results in a vector of size= `4*8= 32 bits`.
Each subvector is `128/4= 32` floats.



Inverted File Index (IVF) During query time, depending on where the query lands in vector space, it may be close to the border of multiple Voronoi cells, which could make the top-k results ambiguous and span across multiple cells. To address this, the IVF-PQ introduces the `nprobe` parameter, which controls the number of Voronoi cells to search during a query. The higher the `nprobe`, the more accurate the results, but the slower the query.
There is also the `refine_factor`, makes the search read extra elements and re-rank them in memory.
The chosen number of sub-vectors divides the vector dimension into Subvector1, Subvector2, etc., in the picture above.





1.14 Developing AI Solutions with LangChain

LLM(s) are currently context free, each request is stateless and processed independently of all others. Sidered in isolation. Moeover, output data is not in a structured format like JSON (it does not considers functions)

1.14.1 Introduction to LangChain

There is an abstraction called LLM, it abstracts an LMM from OpenAI, HuggingFace, Anthropic, etc. Two flavours:

- **Completion**: pass a string and LLM returns text that semantically extends the string.
- **Chat oriented**: pass-in a conversation between user and AI to get semantic completion or answers based on it.

Both flavours are accessed using the same interface.
Prompt templates: similar to formatted strings, but they allow partially filled placeholders. A predined pattern that helps generaing the input for interations with the language model. Partial evaluation and composability. Partial formatting: some placeholders are left to be completed later.
Pipelining: Example. **Prompt**: What is the weather like next week? **Response**:
Templates provide consistency: consistency is crucial to ensure that the LLM responds in a predictable and reliable way. Templates save time and effort. Flexibility: templates can be rigid or flexible as you need.
Prompt templates:

- Functionality - Similar to string formatting in Python, prompt templates act as stencils to guide the generation of inputs for conversations with language models. They enable more dynamic interactions and nuanced control over the input.
- Construction and Usage - Prompt templates are systematically constructed to feed into language models.
- Partial Formatting - This feature allows for the prefilling of certain parts of a template, like a name, with the flexibility to add other details, such as location, later.
- Prompt Pipelining - This involves assembling different pieces of text in a specific order, and is particularly useful in chat prompts where each message might need individual crafting.

LangChain: a framework aimed at developers. One of the most used components is an abstraction called LLM. Hides the particular LLM, like OpenAI, HuggingFace, etc. LLM comes in two flavours: completion and chat oriented.

- **Completion**: pass a string and LLM returns text that semantically extends the string. They are useful for tasks where the goal is to expand or complete a given text input.
- **Chat oriented**: pass-in a cinverstation between user and AI, and get a semantic completion or answers based on it. They are particularly suited for interactive applications.

Both flavours can be used through the same interface.
Prompt Templates: a generalization of string formatters. They allow partial formatting, when you have partial information to fill the template; other information to complete the template can be passed down the line in the code..
Pipelining: you put together different pieces, in a particular order, to create something complex and detailed. You can create a template for each peace of converstation, and then connect them one after another.
Why using templates? They provide consistency. They save time and effort. Templates:

- Are reusable.
- Allow users to define specific changes.
- Are easily adapted for multiple scenarios and applications.

1.14.2 Chains

- Chains are standalone components that allow for composition of multiple components into a coherent whole.
- A chain is just a sequence of calls to various components. These components can even include other chains.
- **LLMChain**, the combination of prompt template thd LLM call is very popular when developing LLM applications. LLMChain combines prompt and execution of the LLM.

1.14.3 Document Loaders

Document Loader is an abstraction to load a LangChain document from a source of data, namely: database, CSV file, Wikipedia and more.
The document Loader contains a `page_content` field and `metadata. CSVLoader()` is the loader of CSV data.

1.14.4 Output parsers

Once the data is loaded, it is passed to the LLM. The LLM can produce:

- Unstructured Text: the raw output of LLMs, which is typically not in a format that's easy for developers to work with.
- Structured Output: the preferred format for developers, such as JSON or CSV. This format is easier to use and integrate into applications.
- Parsing: the process of converting the unstructured text output from LLMs into structured objects.

Langchain integrated Pydantic to parse unstructured data.
In the exercise:

- Loads the CSV data using a CSV loader.
- Creates a ChanOpenAI llm object.
- Sets up a Pydantic Parser to parse the Json output. The Pydantic Output Parser can generate formatting instructions for the LLM.
- Picks 3 random reviews, setup the prompt to classify the review. The prompt contains: question, format instructions from Pydantic, Context: the reviews to classify.

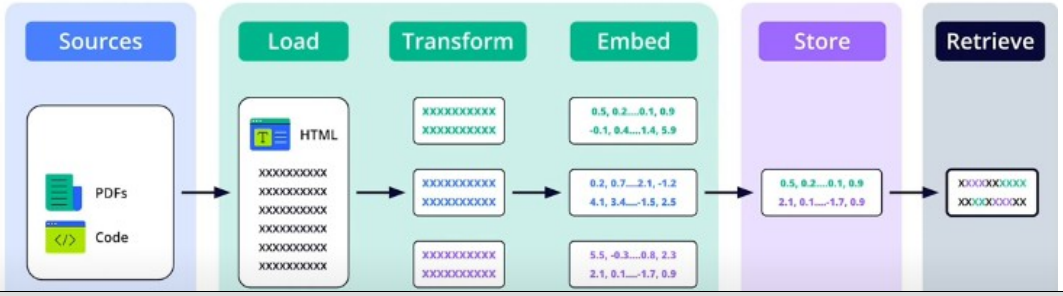
1.14.5 Retrieval Augmented Generation (RAG)

Retrieval Augmented Generation (RAG), is a technique that enhances the capabilities of large language models (LLMs). RAG can integrate a company's data, like a knowledge base, with LLMs. This allows applications to leverage both the power of LLMs and the specific information contained in the company's own data.
RAG: query semantic search over your own data, and use them to query the LLM

How It Works

- **The process begins with a user query**, which is used to search a vector database. Vector databases are used to store data. These databases are essential for adding additional, semantically relevant information to the LLM
- The system then retrieves documents that are semantically closest to the query.
- The retrieved documents are passed along with the original query to the LLM. This provides the LLM with extra context and up-to-date information, resulting in a more informed and accurate response.
- **Document transformers** are used to prepare data by breaking it into smaller chunks. This is beneficial for indexing large documents and achieving a more precise match between the user's query and the document content.
- **Text Embedding Models** convert document chunks into embeddings that capture the semantic meaning of the data.
- Vector Storage is where the embeddings are stored, ready for retrieval.
- Retrievers fetch the semantically relevant chunks for the LLM to process.

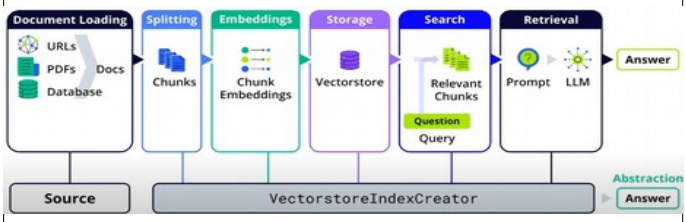
The RAG system allows for efficient use of LLMs by not overwhelming them with data and ensures precise and accurate output.
In LangChain, Document Builders allow you to load various kind of documents (HTML, PDF, etc) into the system.
Document Transformers allow to chop the data into smaller, more manageable chunks. This chunking is a useful stratecy when indexing large documents as it allows you to find a more precise match between the user query and the document content.




```
from langchain.indexes import VectorstoreIndexCreator
from langchain.document_loaders.csv_loader import CSVLoader
loader = CSVLoader(file_path='./tv-reviews.csv')
index = VectorstoreIndexCreator().from_loaders([loader])
query = "Based on the reviews in the context, tell me what people liked about the picture quality"
index.query(query)
```

1.14.6 Vector Stores, Embeddings, Retrievers

Vector Store Index Creator: integrates vector stores, embeddings, and retrievers, streamlining the process of setting up an information retrieval system.



Key Steps in the Process

- Document Loading - The first step involves loading data into the system using various loader components.
- Document Splitting - Documents are split into smaller chunks for more precise matching between user queries and document content.
- Embedding Transformation - Each document chunk is transformed into an embedding. Embeddings are numerical representations of text in a high-dimensional space, where similar meanings are placed close together.
- Vector Database Storage - Embeddings are stored in a vector database like ChromaDB, designed for high-dimensional vectors and semantic search operations.
- Semantic Search - This system uses ChromaDB to perform semantic searches, finding documents based on context and meaning.
- Question-Answering (QA) Chain - The system uses the output from the semantic search to contextualize and generate responses to queries, utilizing semantically relevant documents from the knowledge base.

The interplay of character splitters, embeddings, and vector databases allows the system to mimic human context and nuance understanding.

1.14.7 Memory

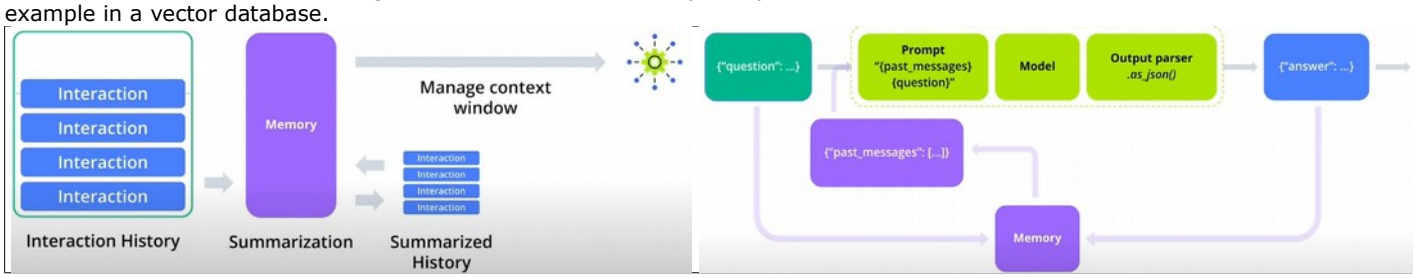
LLM(s) are stateless. To create continuity, we use memory components. The stateless nature of LLM(s) presents two primary challenges:

- the limited context length, that the LLM can handle in an interaction (you cannot copy all the previous converstation in the current context).
- The absence of conversation history in sequential interactions

Vector databases are used to overcome both limitations. LangChain has a Memory component:

- introduces strategies for maintaining a conversation's state by remembering or summarizing the past dialogue
- Applictions can recall the entire converstation history, as long as it remains in a manageable size.
- History can be summarized to compress the information as the history grows

The summarization process is a vital aspect of the LangChain Memory module: it retains relevance and context.
When a conversation reaches a given threshold, the memory component can summarize the essentials and store them, for example in a vector database.



LangChain Memory Component

The LangChain memory component is a solution designed to address the limitations of LLMs in maintaining conversational continuity. LLMs are stateless and they typically process each interaction in isolation, lacking memory of past exchanges. This presents challenges:

- A limited context window.
- The absence of conversation history in sequential interactions.

Memory helps overcome context limitations by storing and retrieving context as needed, enabling LLMs to remember important details beyond their immediate context window.

Simulating Maintained State

The LangChain Memory Component addresses both the issue of maintaining ongoing conversation state and summarizing dialogue.

- Allows for recalling the entire conversation within a manageable size or summarizing it as it grows.
- Retains relevance and context in long interactions.
- Ensures LLMs can access summarized versions to maintain continuity without losing context.
- Enhances user input with context before executing core logic and to store interactions for future reference after the response.
- The memory system is engaged at two points during interaction with an LLM.
- **After user input**. The memory is used after receiving user input, and before executing the core logic, the system reads from the memory to enhance the user input with context.
- **Before response**. After the core logic is executed and before the response is relivered, the interaction is written into memory to be available for future reference.

Memory can be customized:

- variables from memory can be structured to align with the chain's expectations
- there is control over which part of the interaction is saved, using parameters like `input_key` and `output_key`

1.14.8 Memory in Chains

The component that is always present in chains is memory.
ConversationBufferMemory.

- It maintains a list of chat messages to be used within the prompt template ensuring dialogue continuity
- It is the chatbot memory of the ongoing conversation

Both the user query and the LLM response are added to the Conversational Buffer Memory

1.14.9 Examples of Chains

Router Chain

- Purpose - Allows dynamic, non-deterministic sequences of operations based on previous step outputs.
- Application - Adapts interactions with a large language model based on context or question type.
- Example - A chatbot handling customer inquiries about electronics, software, and home appliances. It identifies the inquiry type and routes to the appropriate response mechanism. It determines how to route a customer inquiry based on its content. In this case the routing is done based on a classification function. The routing selects a destination chain. There is also a default chain.

Sequential Chain

- Purpose - Facilitates a series of operations where the output of one is the input for the next. (deterministic chain)
- Application - Useful for processes needing multiple language model calls or functions in a specific order.
- Example - Creating a product description followed by generating a review using that description.

Transformation Chain

- Purpose - Involves altering or processing inputs at various points in an operational sequence.
- Application - Used to create pipelines for specific tasks.
- Example - Extracting video transcripts, stripping timestamps, and then summarizing the text.

Difference between Sequential and transformation: In transformation, the input is altered multiple times, in sequential, the output of a stage is the input of the next.

1.15 Capstone project

https://python.langchain.com/v0.2/docs/how_to/vectorstores/

2 Build a LLM From Scratch

Sebastian Raschka – 2024, version 8 -- Manning Publications. 408 pages. Read up to page 22.
Github repository: https://github.com/rasbt/LLMs-from-scratch Forum:https://livebook.manning.com/forum?product=raschka

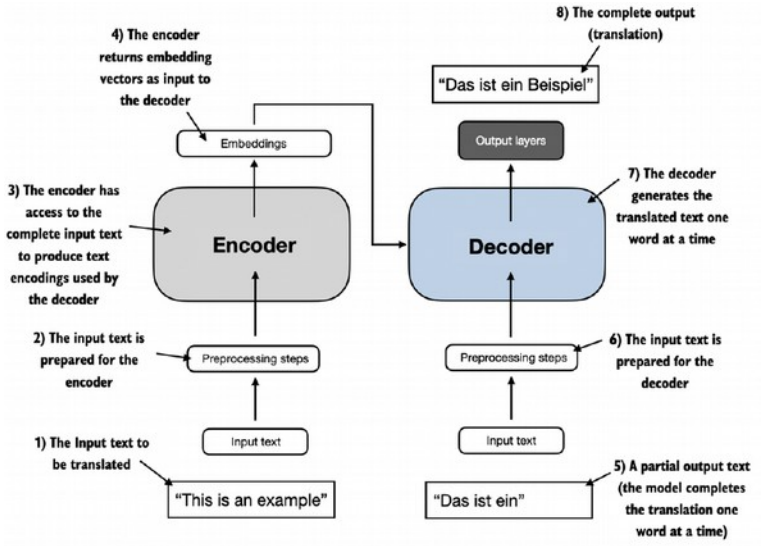
2.1 Understanding Large Language Models

Previous models were designed for specific tasks, LLM(s) are for a wide range of tasks. Success of LLM(s) is due to the amount of data and the transformer architecture.
LLM: large are both the model size and the amount of data used for training. Next word prediction harnesses the sequential nature of the language. LLM(s) use the transformer architecture, it pays selective attention to the parts of the input when making predictions. Note: deep learning does not need feature extraction. Application: knowledge retrieval from vast volumes of text. LLM can automate tasks tat require parsing and generating text [9]. The process of creating an LLM includes:

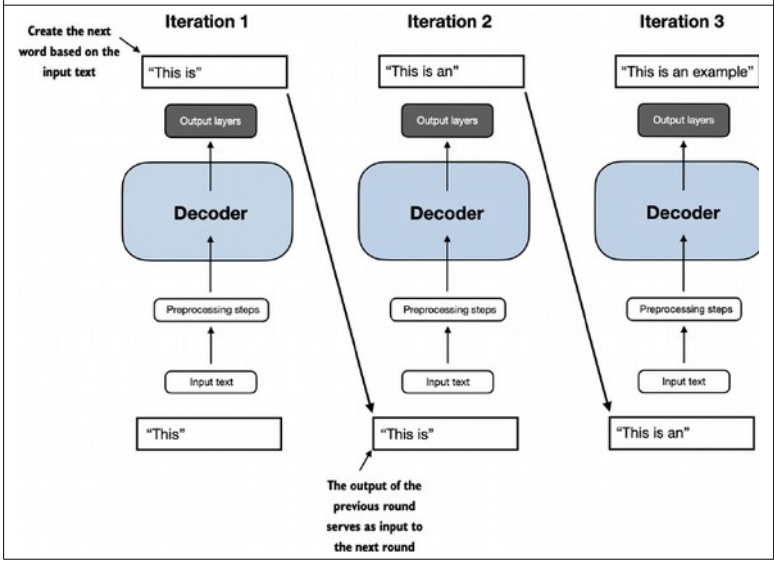
- pretraining: the LLM is trained with a large, diverse dataset, to develop an understanding of the language. The pretrained model is foundational : pretraining uses large dataset, training involves text completion and few shot capabilities. Pretraining uses raw data, filtering may be applied.
- finetuning: after the pre-training, the model is trained on a narrower dataset that is more specific to the task domain. The specific task can be classification, summarization, etc. Popular tasks are:
  - Instruction-finetuning: dataset consists of instruction-answer pairs
  - Classification task: dataset consists of text-label pairs

A custom built LLM can outperform the general purpose LLM, additional advantages are data privacy, local execution.

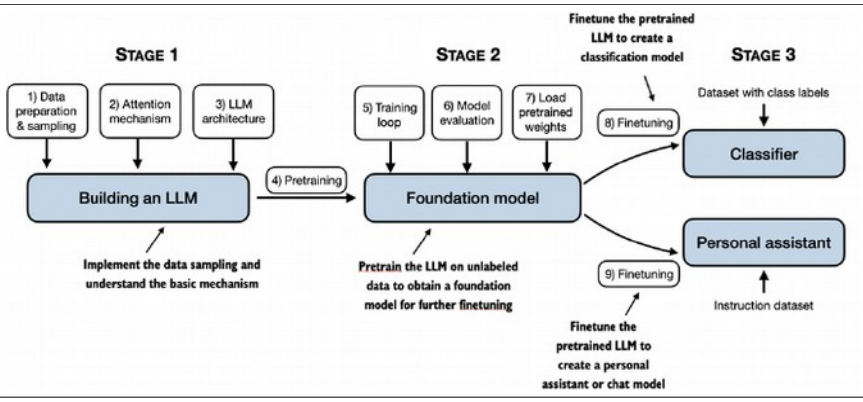
2.1.1 The transformer architecture



The Encoder produces one embedding for each token.
The decoder receives both the encoder output and the decored previous output.
GPT is trained for generative tasks. BERT and similar models are specialized on masked word prediction.
Next-prediction task is a form of self-supervised learning e.g. self-labelling.
Decoder receives the embedding of the current token, plus the embedding of the previous generated token.



GPT architecture: autoregressive model.
Autoregressive models use their previous outputs as input for the next prediction. The GPT architecture uses the decoder. It is designed for unidirectional, left-to-right processing. Text is generated one word at a time. (See also Sect.1.3.11 "NLP Models and Handling sequences")
The original transformer was designed for translation. GPT was designed for text prediction. However, with large models, GPT-like models are also able of performing translation tasks, due to their size. This is called emergent behavior.


STAGE 1
STAGE 2
STAGE 3
1) Data preparation & sampling
2) Attention mechanism
3) LLM architecture
Building an LLM
Implement the data sampling and understand the basic mechanism
4) Pretraining
5) Training loop
6) Model evaluation
7) Load pretrained weights
Foundation model
Pretrain the LLM on unlabeled data to obtain a foundation model for further finetuning
Finetune the pretrained LLM to create a classification model
8) Finetuning
Classifier
Dataset with class labels
Personal assistant
9) Finetuning
Instruction dataset
Finetune the pretrained LLM to create a personal assistant or chat model

3 Generative AI with LangChain – B.Auffarth

361 pages. Read up to page:20

4 Agents

4.1 Available products

4.1.1 Crew AI

https://www.crewai.com/ an agent performs tasks, makes decisions, communicates with other agents

4.1.2 AgentPy

https://agentpy.readthedocs.io/en/stable/index.html A simple script , not a product.

4.1.3 LangChain

https://langchain.com/

4.2 Papers on Agents

4.2.1 AgentLite: A Lightweight Library for Building and Advancing Task-Oriented LLM Agent System

Z.Liu, W.Yao, Silvio Savarese, et al. 23 Feb 2024
ReAct and Reflection reasoning strategy. ReAct: high execution correctness when one-step
LLM agent augments a LLM with tool usage and environment interaction ability
AutoGPT: https://news.agpt.co/ AutoGen: https://microsoft.github.io/autogen/

5 Quantization Fundamentals – DeepLearning.AI

5.1 Handling Big Models

https://learn.deeplearning.ai/courses/quantization-fundamentals/lesson/2/handling-big-models
Bigger and bigger Models tend to become larger and larger. 2023: the larger models have 70B parameters, it would need 200GB.
Different ways to compress the model.
Model compression – pruning= remove connections that do not improve the model. A critera can be the magnitude of the weight, and other metrics.
Model compression – Knowledge distillation: train a smaller model (student = the target, compressed, model) using the original model (instructor). You still have to run the original, big, model, and you have to get the predictions from the original model, which can be costly.
Note: in a NN, you can represent the weights $\vec{w}$ and the activations a as floats, they are the values that propagate through the layers of the NN. $g()$ = activation function. For example, in a linear layer $a=g(\vec{w}\cdot\vec{x}+b)$ $g(z)=\frac{1}{1+e^{-z}}$. $\vec{w}$ are the weights, b is the bias. You can quantize the weights, or the activation values.
Quantization: represent the model weights in a lower precision. If 9 weights and F32 => 36 bytes to store the matrix. If we quantize in 8bit precision, it takes 9 bytes. This comes at a price: the quantization error. The challenge is lowering the quantization error to avoid any performance degradation. See Quanto library from HF

5.2 Data types and Sizes

Unsigned integers. Unsigned Integer, N bits: Range `[0, 2^N -1]`. N=8 =>`[0, 255]`.
Signed integers. To represent negative values: two's complement representation. With N bits, the range is $[-2^{N-1},2^{N-1}-1]$; With N=8 `[-128,127]` The bit in the last position, the e.g. 2^7 is negative, so $-2^7 = -128$. Does this representtion work? The Carry of the last (heaviest bit) is dropped, in general. **NOTE: bits on the left are more significant**.
Torch data types : torch.int16, torch.int32, etc ; limited support for unsigned data types.
Floating point representation:

- Sign: +/-
- Exponent (range): impact the prepresentable range of the number
- Fraction (precision): impact on the precision of the number: E.g. 0.4976 is represented as 0.497599999, or 0.5?

FP32, BF16, FP16, FP8 are floating point formats with specific number of bits for the exponent and fraction.
FP64: Sign: 1bit, Exponent: 11bits, Fraction:52 bit =>32 bits Best^2 precision
FP32: Sign: 1bit, Exponent: 8bits, Fraction:23 bit =>32 bits Best precision
BF16: Sign: 1bit, Exponent: 8bits, Fraction:7 bit => 16 bit Good precision
F16: Sign: 1bit, Exponent: 5bits, Fraction:10 bit. Less range but more precision than BF16. Better precision. `torch.half`
Downcasting: when a higher precition data type, such as a float, is converted into a lower precision data type, such as an integer. It usually results in a loss of data . With size 100, 128 and 512, the loss of precision after dot product is 0.18%.
Downcasting:

- Reduced memory footprint: more efficient use of GPU memory, allows larger model, allows larger batch sizes
- Increased compute speed: computation can be ffaster because it uses less memory, it depends on the hardware
- Less precise: we are using less memory., thus the compution is less precise.

One of the use-cases of downcasting is Mixed precision training: do the computtion in smaller precision (F16, BF16, FP18), but store and update the weights in higher precision

5.3 Loading ML models with different data types

In an ML model, each layer contains some weights, each layer is a matrix of learnable parameters , which can be represented with different precisione precision. For example, 12 layers. In PyTorch, by default models have float32 parameters.

By callong model.to(target_dtype), it is possible to typecast the original model into a different data type. Example model= model.half() . The inference fails: some operations are not implemented for float16.
But Bfloat16 exists : `model_bf16 = DummyModel().to(torch.bfloat16)`
The difference, using bfloat16 is 0.23% HF Transformer, by default, loads models in float32 precision.
`fp32_mem_footprint = model.get_memory_footprint()`. Loading the model as bloat16, halves the memory footprint, as expected.
This way the model is first loaded in full precision, then downcasted.
For Hugging Face Transformers library, the default data type to load the models in is float32. You can set the "default data type" as what you want.

```
desired_dtype = torch.bfloat16
torch.set_default_dtype(desired_dtype)
```

By setting the default type, the model is created with the target precision.

5.4 Quantization Theory = Linear Quantization

Linear Quantization. Quantization: the process of mapping a large set to a smaller set of values.
(-bit linear example. Map the highest Fp32 number to 127, and the most negative to -128. Other values are mapped using linear mapping. You convert the original values, and also keep track of the values that you used to quantize $s = scale$, $z = zero\ point$; they are called linear mapping parameters. We delete the original tensors, so we save a lot of memory. You cannot get beack the original values, but you can perform de-quantization, to transform 8-bit back in float32. The quantization-dequantization error is not zero, but it is not too bad either. Linear quantization is used in many SOTA methods:

- AWQ: Activation – aware weight quantization
- GPTQ: GPT Quantized
- BNB: Bits and Bytes Quantization.

Libraries:

```
!pip install transformers==4.35.0
!pip install quanto==0.0.11
!pip install torch==2.1.1

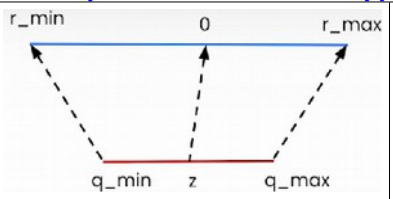
from quanto import quantize, freeze
import torch
tokenized = AutoTokenizer.from_pretrained(model_name)
model = AutoModelForCausalML.from_pretrained(model_name, low_cpu_mem_usage=True )
quantize(model, weights=torch.int8, activations=None)
freeze(model)
```

In the model, linear layers are replaced by Qlinear layers. To run the inference:

```
input_text = "Hello, my name is "
input_ids = tokenizer(input_text, return_tensors="pt").input_ids

outputs = model.generate(input_ids)
print(tokenizer.decode(outputs[0], skip_special_tokens=True))
```

Linear quantization. Linear mapping



r = the original tensor values. q = quantized values

$$r = s(q - z) \qquad q = \frac{r}{z} + z$$

$$r_{max} = s(q_{max} - z) \qquad r_{min} = s(q_{min} - z)$$

$$s = \frac{r_{max} - r_{min}}{q_{max} - q_{min}} \qquad z = \text{int}\left(round\left(q_{max} - \frac{r_{min}}{s}\right)\right)$$

Quantization aware trainig: training in a way that controls how the model performs once it is quantized.

- Intermediate start hold both:
  - a quantized version of the weights
  - original unquantized weights
- Forward pass (inference)
  - Use quantized version of the model weight to make predictions
- Back propagation uses the unquantized version of the model weights to update the model

5.5 Quantization of LLM(s)

QLora AWQ, GPTQ, SmoothQuant ; AQLM, HQQ, QuIP# : 2-bit quantization
Some of them require calibration: : run inference on a dataset, optimize quantization parameters to minimze the quantization error.
Some quantization methods work out of the box: they can be applied to any model without adjustement.
Ready to use are:

- Linear quantization-dequantization
- LLM.INT8
- Qlora 4-bit
- HQQ (up to 2-bit) (link)

Quantized models are easier to distribute: Llama 2 7B is 28GB inf32, becomes about 4GB in 4-bit precision in GGUF format In Hugging face, there is TheBloke who distributes quantized models.
Is it possible to fine-tune a quantized LLM? PEFT + Qlora can be used

5.5.1 Videos

- https://youtu.be/WocHhzv-Ijo
- https://youtu.be/Sq5wwHbP3Js
-

https://www.youtube.com/watch?v=4A-E5RchG4o

6 Papers

6.1 Encoder-Decoder

6.1.1 Neural Machine Translation by Jointly Learning to Align and Translate (Y.Bengio et al, 201409)

Dzmitry Bahdanau, Kyunghyun Cho, Yoshua Bengio, Last revised 19 May 2016 (v7)

Most of the proposed neural machine translation models belong to a family of encoder-decoders, where the encoder reads and encoders a source sentence into a fixed length vector. Problem with long sentences.

The proposed solution is an extension to the encoder-decoder which learns to along and translate jointly.

Translation. x =source sentence. y :translated sentence $argmax_y\, p(y|x)$. In NN machine translation, typically two components: an encoder of the source sentence, to a fixed kength vector, and a decoder to the target sentence y . This approach is called RNN Encoder-Decoder.

Input sequence $x=(x_1, x_2, \dots x_{T_x})$; each stage of the RNN is $h_t = f(x_t, h_{t-1})$.

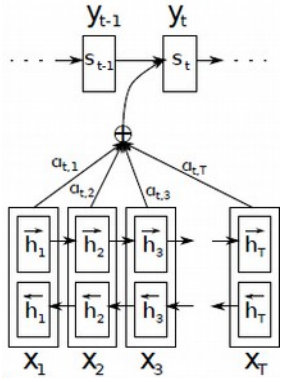
$c=q(\{h_1, h_2, \dots, h_{T_x}\})$ is the sequence of the hidden states.

$f()$, $q()$ are non linear functions. Usually, $q(\{h_1, h_2, \dots, h_{T_x}\}) = h_{T_x}$: only the last hidden vector is used.

The decoder creates $y=(y_1, y_2, \dots y_{T_y})$, using the previous output and the hidden state $p(y_t|\{y_1, y_2, \dots y_{t-1}, c\}) = g(y_{t-1}, s_t, c)$

6.1.1.1 Learning to align and translate

Decoder: $p(y_t|\{y_1, y_2, \dots y_{t-1}, \boldsymbol{x}\}) = g(y_{t-1}, s_t, c_i)$; the context vector is different on every step. Hidden vector items are called "annotations" here.

$e_{ij}=a(s_{i-1}, h_j)$	$\alpha_{ij}=softmax(e_{ij})=\frac{\exp(e_{ij})}{\sum_{k=1}^{T_x}\exp(e_{ik})}$	$c_i=\sum_{j=1}^{T_x}\alpha_{ij}h_j$	

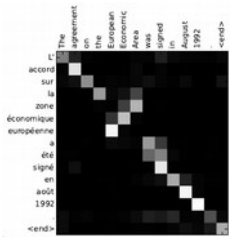
- i : we are trying to produce the output at the i[th] step
- j : index of the hidden state of the already encoded vectors h_j
- $a()$ is the "alignment model", a feedforward neural network.
- h_j : hidden state of the encoder = annotation.
- s_i : hidden state of the decoder.

The weighted sum of all annotations $c_i=\sum_{j=1}^{T_x}\alpha_{ij}h_j$ is called expected annotation. The probability α_{ij} reflects the importance of h_j with respect to the previous hidden state of the decoder s_{i-1} . "*This implements a mechanism of attention in the decoder*".

Encoder:a bidirectional RNN is used (BiRNN)

BiRNN: a forward RNN reads the input sequence from first to last, and calculates the sequence of forward hidden states $\vec{h}_i$.

The backward RNN reads the sequence in the reverse order resulting in a sequence of backward hidden states $\bar{h}_i$. Each hidden state is the concatentation of the forward and backward hidden state. $h_i=[\bar{h}_i, \vec{h}_i]$

Results. These plots show the values of α_{ij} : the weight that each item of the input sentence has, when producing a given output vector.	

6.1.2 Deep contextualized word representations - ELMo 201806

M.E.Peters, M.Neumann, M.Iyyer, M.Gardner, C.Clark, K.Lee, L.Zettlemoyer, 201806
Each token is assigned a representation that is a function of the entire input sentence.
EMLo

6.2 Transformers

6.2.1 Attention is all you need: 20170712, revised 20230823

(Link to arXiv)

6.3 Fine-tuning

6.3.1 LoRA: Low-Rank Adaptation Of Large Language Models. 20211016

Version2. E.Hu, Y.Shen, P.Wallis, Z.Allen-Zhu, Y.Li, SheanWang, L.Wang, W.Chen

Lora can be applied to any dense layer in deep learning models. Usually weight matrices have full rank. When adapting to specific tasks, weights have low intrinsic dimension. ΔW is added to the original weight matrices to adapt it to the new task.

$$W_0+\Delta W=W_0+B*A \quad B\in \boldsymbol{R}^{d\times r} \quad A\in \boldsymbol{R}^{r\times k} \quad d\ll min(d,k)$$

$\alpha\in N$ is a metaparameter of the LoRA algorithm, usually $\alpha=2*r$
In the parper, they only fine tune the attention weight for downstream tasks. MLP modules are frozen.
Typical values for r are 2, 4 and 8. Typical $\lambda=2*10^{-4}$ $Lora\ dropout=0.1$

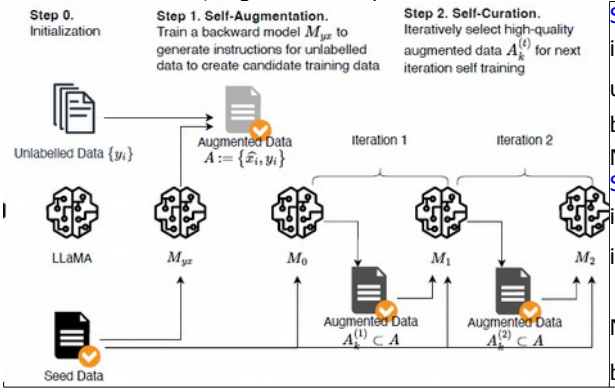
6.4 Instruction following

6.4.1 Self-Alignment with Instruction Backtranslation (20230814)

Xian Li, Ping Yu, Chunting Zhou, Timo Schick, Luke Zettlemoyer, Omer Levy, Jason Weston, Mike Lewis (arXiv)
Instruction backtranslation: inspired from the classic backtranslation method
Start with an instruction following model and a web corpus. For each document, the model tries to predict the prompt that would be answered by the document. With the couple it creates a (request/response) training sample.
In case the set of samples are used directly, the result is poor. Instead, use the model itself to self-curate the set of newly created augmentation dfata by predicting their quality. Then train using only the high quality samples. The procedure is iterated, the improved model is used to curate the instruction data, etc.
Start with the base language model and a collection of unlabelled examples. Assumptions:

- 1st assumption: there exist some subset of the corpus that is suitable as gold generation for some user instructions.
- 2nd assumption: we can predict instructions for the candidate gold answers.

The process starts with **seed data**: a set of human annotated (instruction, output) that is used to fine-tune models to give predictions in both directions: predict an output given an instruction, predict the instruction given the output. The process starts ith unlabelled data, e.g. a web corpus as source of unlabelled data.



Self-augmentation: fine tune the base language model with (output, instructions pairs) to obtain a backward model $M_{yx}:=p(x|y)$. y: unlabelled example, $\hat{x}$: candidate instruction created using the backward model. Candidate augmented paired dataset $A:=\{(\hat{x}_i,y_i)\}$
Note that some of the items on the Candidate data set are of low-quality.
Self Curation (selecting high-quality examples) the language model is instructed to rate each $(\hat{x}_i,y_i)$ pair on a 5-point scale. The dataset A is filtrated by removing the samples that have score $a_i<k$

Next loop: $A^{(1)}$ is used to train $M^{(1)}{}_{yx}$, $M^{(1)}{}_{yx}$ is used to create a better paired A, that is then filtered to create $A^{(2)}$.

They used the Open Assistant dataset.
Typical learning parameters: $\lambda=10^{-5}$, linear decay to $9*10^{-6}$, weight decay 0.1, batch size 32, dropout 0.132

7 Biography

ID	Title	Version

8 Links

- CivitAI https://civitai.com/ run the WildCardX-XL-Fusion
- Hugging Face:
  - Transformers: https://huggingface.co/docs/transformers/index
- ONNX: with C++ in Linux
  - https://github.com/microsoft/onnxruntime ONNX runtime for CPU on Linux (tutorials)
- ONNX-MLIR: transforms a model into C++ code https://github.com/onnx/onnx-mlir https://github.com/openxla https://github.com/onnx/onnx-mlir/blob/main/docs/BuildOnLinuxOSX.md

https://towardsdatascience.com/7-things-you-didnt-know-about-the-transformer-a70d93ced6b2
https://pytorch.org/tutorials/beginner/transformer_tutorial.html
https://github.com/karpathy/nanoGPT

9 Terms and acronyms

Term	Description
logits	the vector of raw (non-normalized) predictions that a classification model generates, which is ordinarily then passed to a normalization function. If the model is solving a multi-class classification problem, logits typically become an input to the softmax function. The softmax function then generates a vector of (normalized) probabilities with one value for each possible class.